

《计算机组成原理实验》

(第一次)

厦门大学信息学院软件工程系
2026年3月

目录

• 第一部分 高级语言中的数据类型

- C语言数据表示实例
- C语言运算溢出实例
- C语言整型数据类型转换（8位转8位）
- C语言整型数据类型转换（8位转32位）
- C语言整型数据类型转换（32位转16位）
- C语言中的浮点数据类型（例2.9）

• 第二部分 汉字编码实验

- 在LED点阵上显示“华中科技大学计算机学院谭志虎”
- 在LED点阵上显示“厦门大学信息学院欢迎您！”（设计实验）

• 第三部分 校验码实验

• （一）奇偶校验码实验

- 16位偶校验传输测试实验
- 16位奇校验传输测试实验（设计实验）
- 8位偶校验传输测试实验（设计实验）

• （二）海明校验码实验

- 16位扩展海明码传输测试实验1
- 16位扩展海明码传输测试实验2
- 8位扩展海明码传输测试实验（设计实验）

• （三）循环冗余校验码（并行）实验

- 16位循环冗余校验码（并行）实验1
- 16位循环冗余校验码（并行）实验2
- 8位循环冗余校验码（并行）实验（挑战性实验）

• （四）循环冗余校验码（串行）实验

- 循环冗余校验码（串行）实验1
- 循环冗余校验码（串行）实验2

对应教材2.2.5小节“计算机中的数据类型”

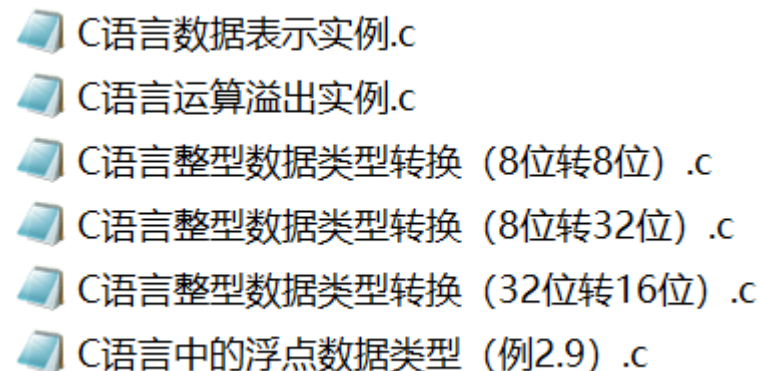
第一部分 高级语言中的数据类型

参考程序

-  C语言数据表示实例.c
-  C语言运算溢出实例.c
-  C语言整型数据类型转换 (8位转8位) .c
-  C语言整型数据类型转换 (8位转32位) .c
-  C语言整型数据类型转换 (32位转16位) .c
-  C语言中的浮点数据类型 (例2.9) .c

高级语言中的数据类型

- 1、C语言数据表示实例（教材P35-36程序）
- 2、C语言运算溢出实例（教材P36程序）
- 3、C语言整型数据类型转换（8位转8位）（教材P37程序）
- 4、C语言整型数据类型转换（8位转32位）（教材P37程序）
- 5、C语言整型数据类型转换（32位转16位）（教材P38程序）
- 6、C语言中的浮点数据类型（例2.9）（教材P38-39程序）



-  C语言数据表示实例.c
-  C语言运算溢出实例.c
-  C语言整型数据类型转换（8位转8位）.c
-  C语言整型数据类型转换（8位转32位）.c
-  C语言整型数据类型转换（32位转16位）.c
-  C语言中的浮点数据类型（例2.9）.c

- 第一步：解压 “**devc++.zip**”
- 第二步：在解压后的文件夹中运行“**devcppPortable.exe**”
- 第三步：在Dev-C++5.11中打开你所编写的C语言代码文件(课前布置)
- 第四步：点击“运行”->“编译运行”，即可得到程序的运行结果

1、C语言数据表示实例.c

C语言程序——数据表示实例.c - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
#include "stdio.h"
```

```
union
```

```
{
    int i; unsigned int ui; float f;
    short s; unsigned short us;
    char c; unsigned char uc;
};
```

```
void hex_out(char a)
```

```
{
    const char HEX[]="0123456789ABCDEF";
    printf("%c%c",HEX[(a&0xF0)>>4],HEX[a&0xF]);
}
```

```
void out_1byte(char *addr)
```

```
{
    hex_out (*(addr +0));
}
```

```
void out_2byte(char *addr)
```

```
{
    hex_out (*(addr +1));
    hex_out (*(addr +0));
}
```

```
void out_4byte(char *addr)
```

```
{
    hex_out (*(addr +3));
    hex_out (*(addr +2));
    hex_out (*(addr +1));
    hex_out (*(addr +0));
}
```

```
main()
```

```
{
    t.i=0xC77FFFFFFF;

    out_4byte(&t.i);
    printf(" = %d \n",t.i);

    out_4byte(&t.ui);
    printf(" = %u \n",t.ui);

    out_4byte(&t.f);
    printf(" = %f \n",t.f);

    out_2byte(&t.s);
    printf(" = %d \n",t.s);

    out_2byte(&t.us);
    printf(" = %u \n",t.us);

    out_1byte(&t.c);
    printf(" = %d \n",t.c);

    out_1byte(&t.uc);
    printf(" = %d \n",t.uc);
}
```

程序运行结果

D:\计算机组成原理\实验\计算机组成原理实验1\计算机组成原理实验1.c

```
C77FFFFFFF = -947912705
C77FFFFFFF = 3347054591
C77FFFFFFF = -65535.996094
FFFF = -1
FFFF = 65535
FF = -1
FF = 255
```

```
-----
Process exited after 0.7444 seconds with return value 0
请按任意键继续. . .
```

2、C语言运算溢出实例.c

C语言程序——运算溢出实例.c - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
#include "stdio.h"
```

```
void main()
```

```
{
```

```
    short s1=32767,s2=-32768,s;
```

```
    unsigned char uc1=128,uc2=255,uc;
```

```
    s=s1+1;
```

```
    printf(" %d + 1 = %d\n",s1,s);
```

```
    s=s2-3;
```

```
    printf(" %d - 3 = %d\n",s2,s);
```

```
    uc=uc1+uc2;
```

```
    printf(" %d + %d = %d\n",uc1,uc2,uc);
```

```
    uc=uc1-uc2;
```

```
    printf(" %d - %d = %d\n",uc1,uc2,uc);
```

```
}
```

程序运行结果

```
D:\计算机组成原理\实验\计算... × + v
32767 + 1 = -32768
-32768 - 3 = 32765
128 + 255 = 127
128 - 255 = 129
-----
```

3、C语言整型数据类型转换（8位转8位）.c

C语言程序——相同字长的整型数据转换.c - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
#include "stdio.h"
```

```
void hex_out(char a)
{
    const char HEX[]="0123456789ABCDEF";
    printf("%c%c",HEX[(a&0xF0)>>4],HEX[a&0xF]);
}
```

```
void out_1byte(char *addr)
{
    hex_out (*(addr +0));
}
```

```
int main()
{
    unsigned char uc1=255,uc;
    char c1=-127,c;

    c=(char)uc1;

    out_1byte(&uc1);
    printf(" = uc1 = %u \n",uc1);

    out_1byte(&c);
    printf(" = c = %d \n",c);

    uc=c1;

    out_1byte(&c1);
    printf(" = c1 = %d \n",c1);

    out_1byte(&uc);
    printf(" = uc = %u \n",uc);

}
```

程序运行结果

D:\计算机组成原理\实验\计算1

FF = uc1 = 255

FF = c = -1

81 = c1 = -127

81 = uc = 129

4、C语言整型数据类型转换（8位转32位）.c

📄 C语言程序——小字长转大字长.c - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
#include "stdio.h"
```

```
void hex_out(char a)
{
    const char HEX[]="0123456789ABCDEF";
    printf("%c%c",HEX[(a&0xF0)>>4],HEX[a&0xF]);
}
```

```
void out_1byte(char *addr)
{
    hex_out (*(addr +0));
}
```

```
void out_4byte(char *addr)
{
    hex_out (*(addr +3));
    hex_out (*(addr +2));
    hex_out (*(addr +1));
    hex_out (*(addr +0));
}
```

```
int main()
{
    unsigned char uc=254;
    char c=uc;
    int i; unsigned ui;

    i=uc;
    ui=uc;

    out_1byte(&uc);
    printf(" = uc = %d \n",uc);

    out_4byte(&i);
    printf(" = i = %d \n",i);

    out_4byte(&ui);
    printf(" = ui = %u \n",ui);

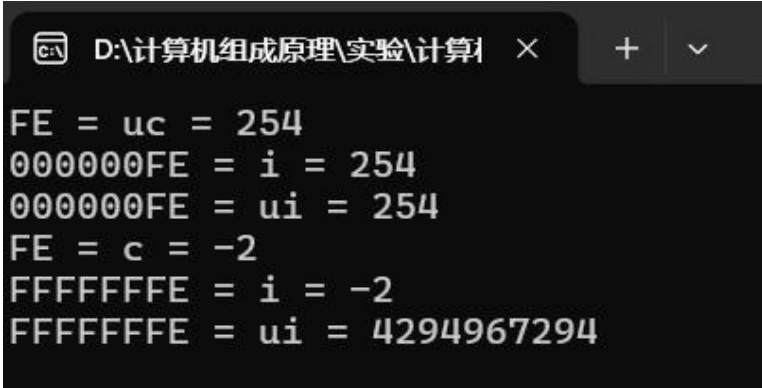
    i=c;
    ui=c;

    out_1byte(&c);
    printf(" = c = %d \n",c);

    out_4byte(&i);
    printf(" = i = %d \n",i);

    out_4byte(&ui);
    printf(" = ui = %u \n",ui);
}
```

程序运行结果



```
D:\计算机组成原理\实验\计算
FE = uc = 254
000000FE = i = 254
000000FE = ui = 254
FE = c = -2
FFFFFFFFFE = i = -2
FFFFFFFFFE = ui = 4294967294
```

5、C语言整型数据类型转换（32位转16位）.c

C语言程序——大字长转小字长.c - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
#include "stdio.h"
```

```
void hex_out(char a)
{
    const char HEX[]="0123456789ABCDEF";
    printf("%c%c",HEX[(a&0xF0)>>4],HEX[a&0xF]);
}
```

```
void out_2byte(char *addr)
{
    hex_out (*(addr + 1));
    hex_out (*(addr + 0));
}
```

```
void out_4byte(char *addr)
{
    hex_out (*(addr + 3));
    hex_out (*(addr + 2));
    hex_out (*(addr + 1));
    hex_out (*(addr + 0));
}
```

```
int main()
{
    int i=0xFFFF1001;
    short s; unsigned short us;

    s=i;
    us=i;

    out_4byte(&i);
    printf(" = i = %d \n",i);

    out_2byte(&s);
    printf(" = s = %d \n",s);

    out_2byte(&us);
    printf(" = us = %u \n",us);
}
```

程序运行结果

```
D:\计算机组成原理\实验\计算机组成原理实验1\计算机组成原理实验1.c
FFFF1001 = i = -61439
1001 = s = 4097
1001 = us = 4097
-----
```

```
#include "stdio.h"
#include <math.h>
```

```
int f1(unsigned n)
{
    int sum=1,power=1;
    for(unsigned i=0; i<=n-1; i++)
    {
        power *= 2;
        sum += power;
    }
    return sum;
}
```

```
float f2(unsigned n)
{
    float sum=1,power=1;
    for(unsigned i=0; i<=n-1; i++)
    {
        power *= 2;
        sum += power;
    }
    return sum;
}
```

```
int f3(int n)
{
    int sum=1,power=1;
    for(int i=0; i<=n-1; i++)
    {
        power *= 2;
        sum += power;
    }
    return sum;
}
```

6、C语言中的浮点数据类型（例2.9）.c

```
int main()
{
    unsigned n;
    int n1;

    n=0;
    printf("f1(%d)=%d\n",n,f1(n));
```

```
// n=0;
// printf("f2(%d)=%d\n",n,f2(n));
```

```
n1=0;
printf("f3(%d)=%d\n",n1,f3(n1));
```

```
n=23;
printf("f1(%d)=%d f2(%d)=%f\n",n,f1(n),n,f2(n));
```

```
n=24;
printf("f1(%d)=%d f2(%d)=%f\n",n,f1(n),n,f2(n));
```

```
long long int f31=pow(2,32)-1;
n=31;
printf("f(31)=%lld f1(%d)=%d\n",f31,n,f1(n));
```

```
long long int f30=pow(2,31)-1;
n=30;
printf("f(30)=%lld f1(%d)=%d\n",f30,n,f1(n));
```

```
n=127;
printf("f2(%d)=%f\n",n,f2(n));
```

```
n=126;
printf("f2(%d)=%f\n",n,f2(n));
}
```

屏蔽这两行

程序运行结果

n=0，f1出现死循环

```
#include "stdio.h"
#include <math.h>
```

```
int f1(unsigned n)
{
    int sum=1,power=1;
    for(unsigned i=0; i<=n-1; i++)
    {
        power *= 2;
        sum += power;
    }
    return sum;
}
```

```
float f2(unsigned n)
{
    float sum=1,power=1;
    for(unsigned i=0; i<=n-1; i++)
    {
        power *= 2;
        sum += power;
    }
    return sum;
}
```

```
int f3(int n)
{
    int sum=1,power=1;
    for(int i=0; i<=n-1; i++)
    {
        power *= 2;
        sum += power;
    }
    return sum;
}
```

6、C语言中的浮点数据类型（例2.9）.c

屏蔽这两行

```
int main()
{
    unsigned n;
    int n1;

    // n=0;
    // printf("f1(%d)=%d\n",n,f1(n));

    n=0;
    printf("f2(%d)=%d\n",n,f2(n));

    n1=0;
    printf("f3(%d)=%d\n",n1,f3(n1));

    n=23;
    printf("f1(%d)=%d   f2(%d)=%f\n",n,f1(n),n,f2(n));

    n=24;
    printf("f1(%d)=%d   f2(%d)=%f\n",n,f1(n),n,f2(n));

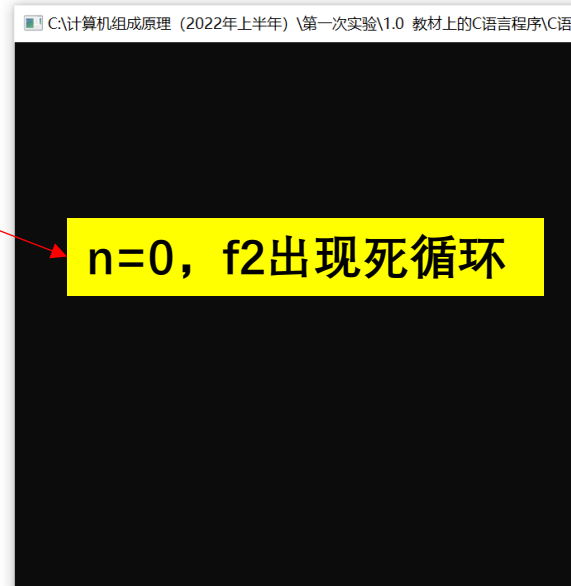
    long long int f31=pow(2,32)-1;
    n=31;
    printf("f(31)=%lld   f1(%d)=%d\n",f31,n,f1(n));

    long long int f30=pow(2,31)-1;
    n=30;
    printf("f(30)=%lld   f1(%d)=%d\n",f30,n,f1(n));

    n=127;
    printf("f2(%d)=%f\n",n,f2(n));

    n=126;
    printf("f2(%d)=%f\n",n,f2(n));
}
```

程序运行结果



屏蔽这四行

```
int f3(int n)
{
    int sum=1,power=1;
    for(int i=0; i<=n-1; i++)
    {
        power *= 2;
        sum += power;
    }
    return sum;
}
```

```
int main()
{
    unsigned n;
    int n1;

    // n=0;
    // printf("f1(%d)=%d\n",n,f1(n));

    // n=0;
    // printf("f2(%d)=%d\n",n,f2(n));

    n1=0;
    printf("f3(%d)=%d\n",n1,f3(n1));

    n=23;
    printf("f1(%d)=%d   f2(%d)=%f\n",n,f1(n),n,f2(n));

    n=24;
    printf("f1(%d)=%d   f2(%d)=%f\n",n,f1(n),n,f2(n));

    long long int f31=pow(2,32)-1;
    n=31;
    printf("f(31)=%lld   f1(%d)=%d\n",f31,n,f1(n));

    long long int f30=pow(2,31)-1;
    n=30;
    printf("f(30)=%lld   f1(%d)=%d\n",f30,n,f1(n));

    n=127;
    printf("f2(%d)=%f\n",n,f2(n));

    n=126;
    printf("f2(%d)=%f\n",n,f2(n));
}
```

程序运行结果

```
D:\计算机组成原理实验\计算l × + ∨  
f3(0)=1  
f1(23)=16777215      f2(23)=16777215.000000  
f1(24)=33554431      f2(24)=33554432.000000  
f(31)=4294967295     f1(31)=-1  
f(30)=2147483647     f1(30)=2147483647  
f2(127)=1.#INF00  
f2(126)=17014118346046923000000000000000000000000000.000000
```

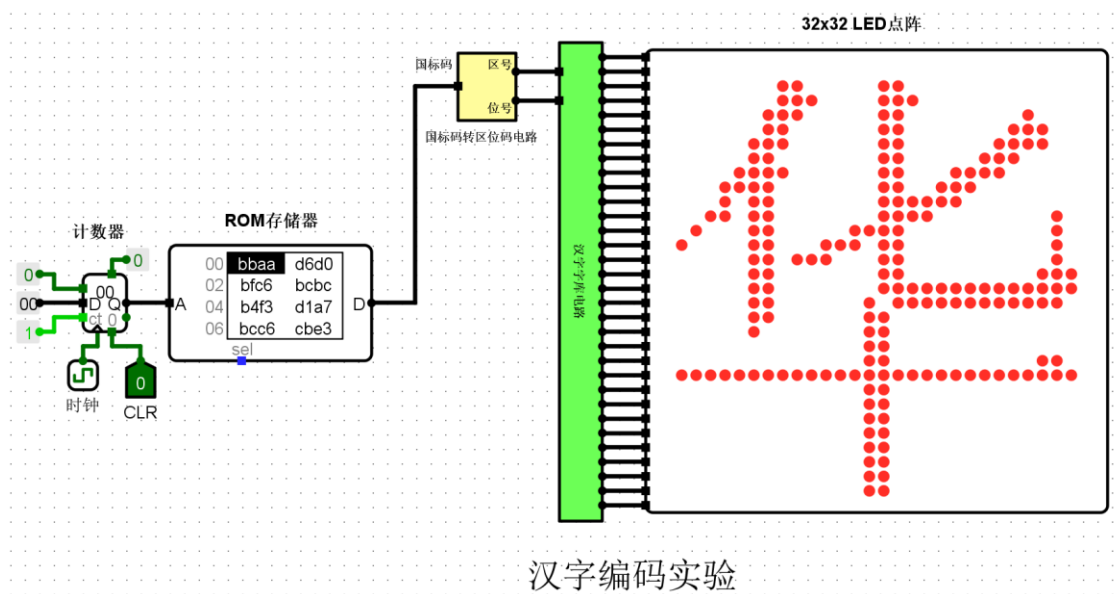
对应教材2.3.2小节“汉字编码”

第二部分 汉字编码实验

参考设计文件：汉字编码实验.circ

汉字国标码：ROM内容——华中科技大学计算机学院谭志虎.txt

- 运行“Logisim”，在Logisim中打开“**汉字编码实验.circ**”。
- 也可以直接双击“**汉字编码实验.circ**”，打开该设计文件。

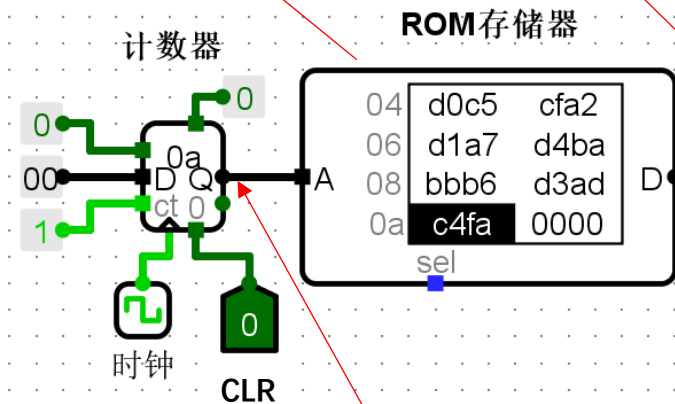


由“国标码转区位码电路”，将国标码转换为区号（7位）和位号（7位）

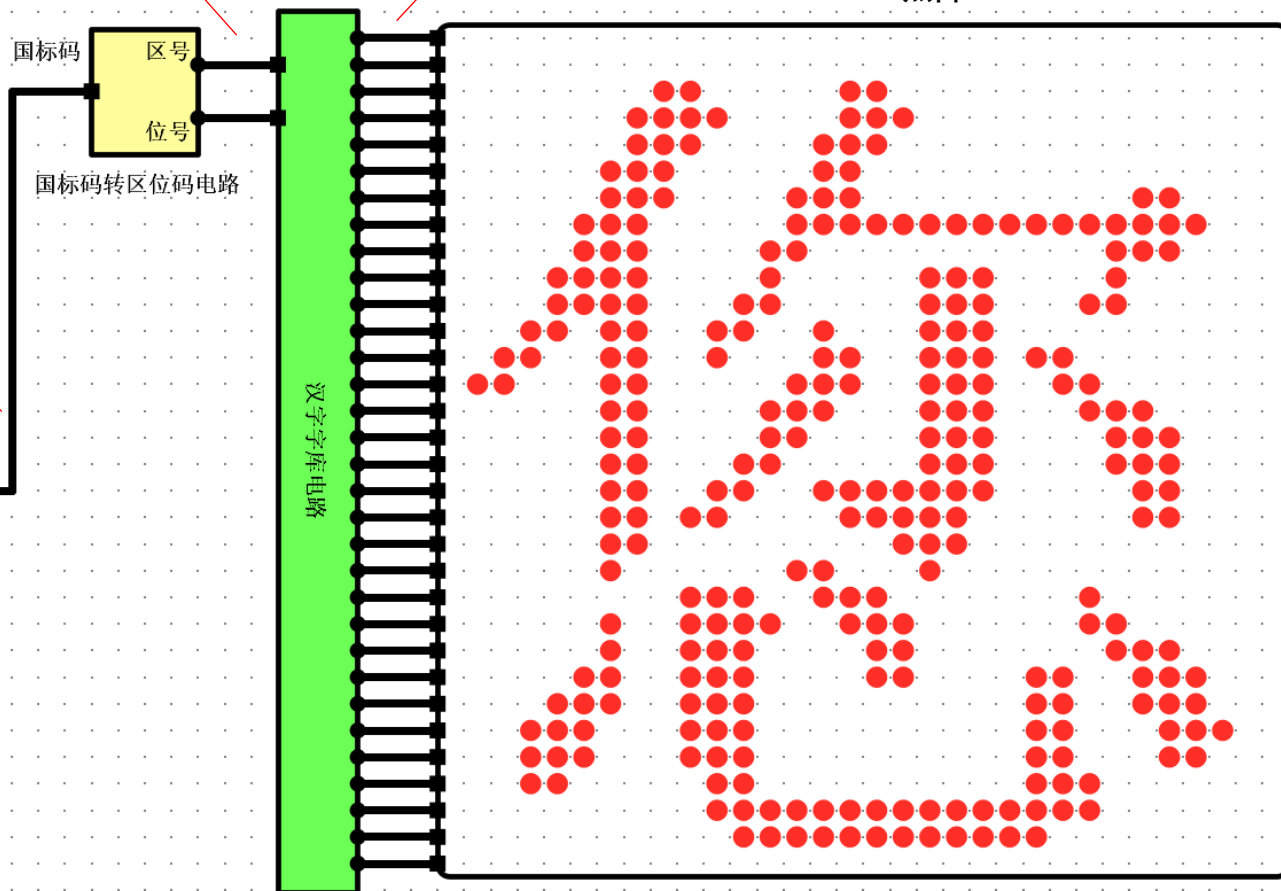
根据区号（7位）和位号（7位），从“汉字字库电路”中得到汉字的点阵码（32x32），并在LED点阵上显示

从ROM存储器中取出汉字的国标码（16位）

ROM存储器中存放汉字的国标码（16位）



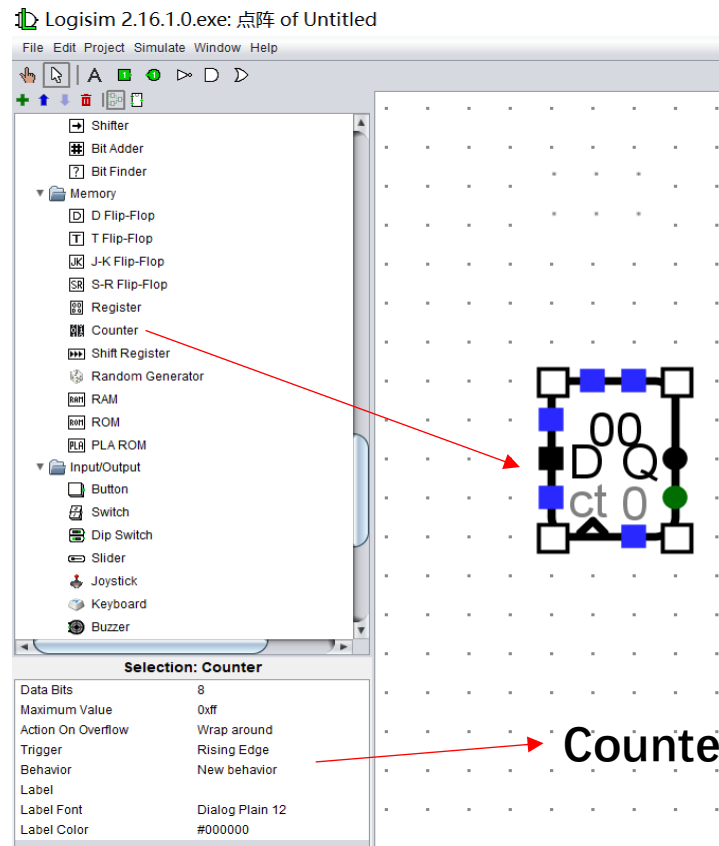
计数器给出ROM存储器的地址（8位）



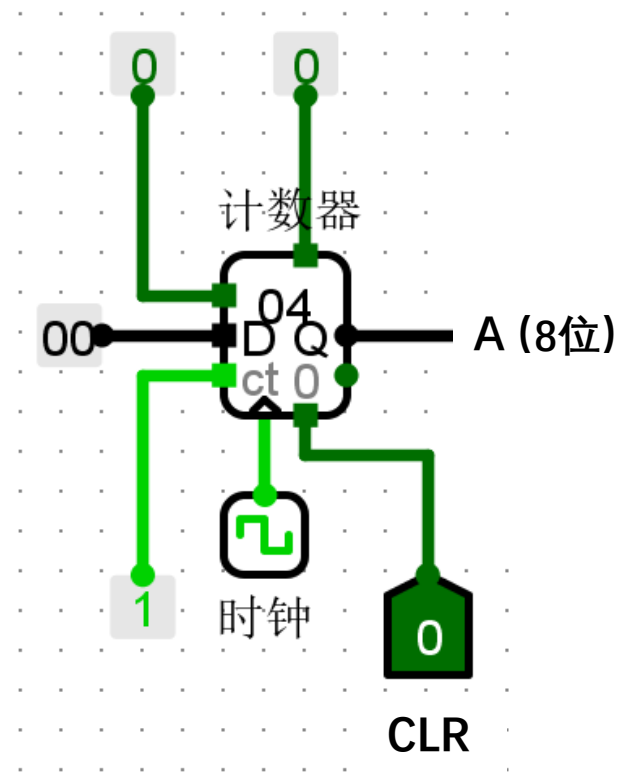
汉字编码实验

• 1、计数器 (Counter)

- “CLR”=0，按”时钟“2次，计数器输出A加1
- “CLR”=1，计数器输出A=0
- 计数器输出A为ROM存储器的地址（8位）

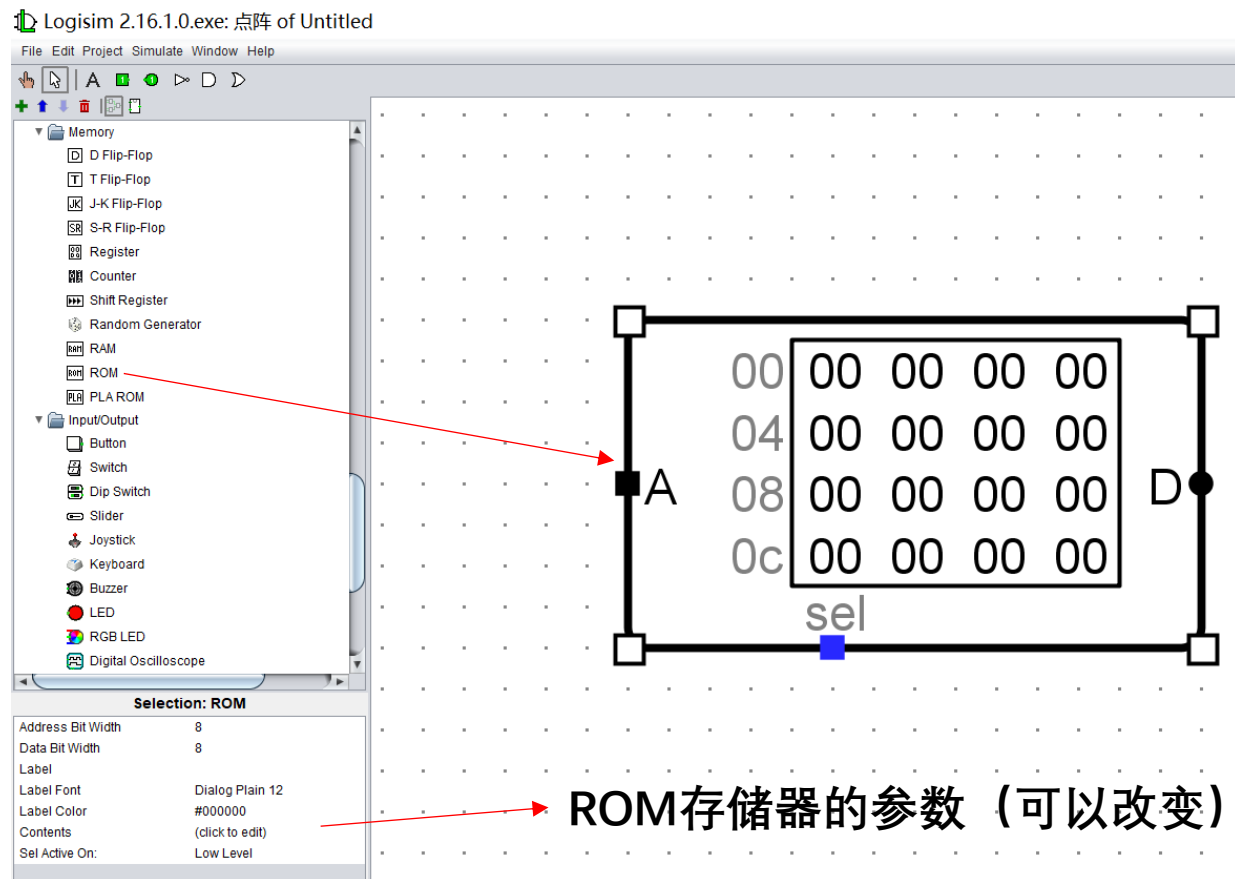
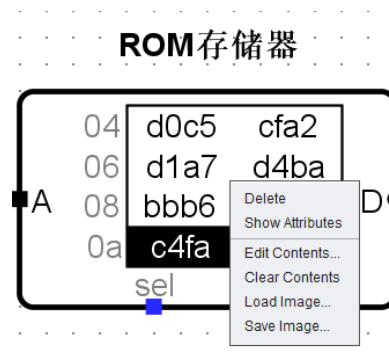


Counter的参数（可以改变）



• 2、ROM存储器 (ROM)

- 输入A: 存储器的地址 (8位二进制数)
- 输出D: 存储器的数据 (16位二进制数, 即汉字的国标码)
- ROM存储器中的内容可以装入 (存放在文本文件中)



ROM内容——华中科技大学计算机学院谭志虎.txt - 记事本

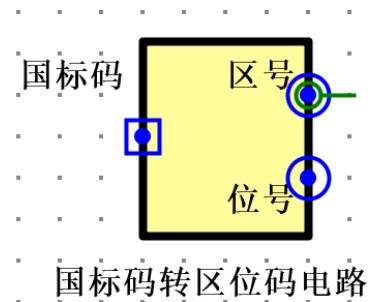
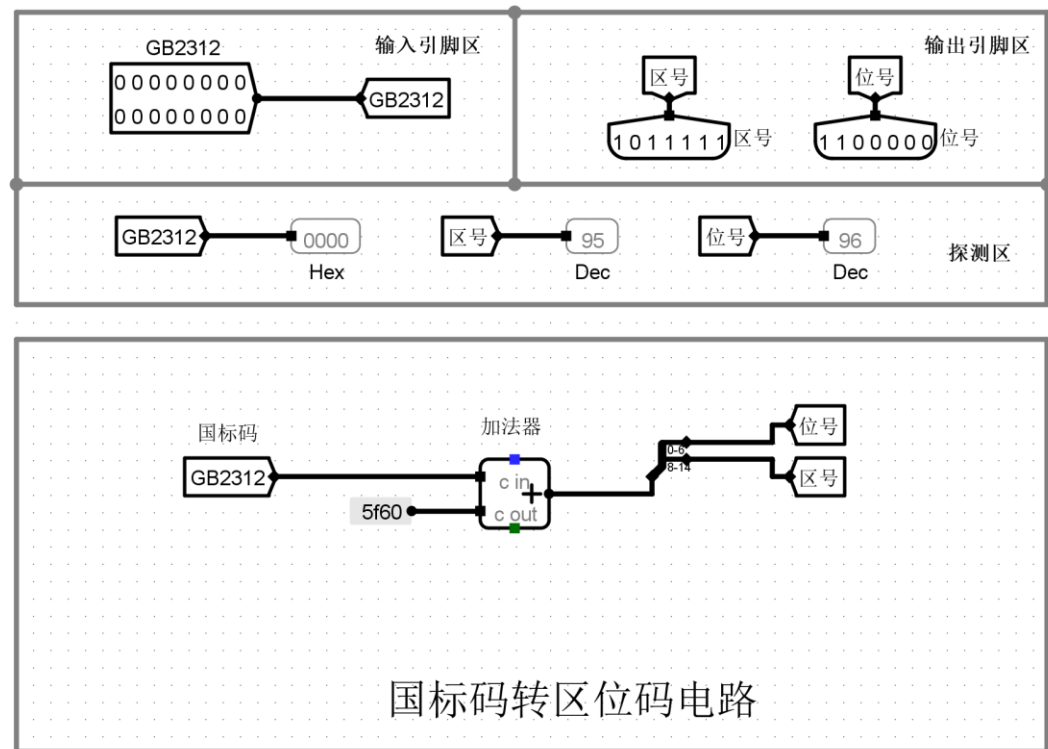
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

bbaa d6d0 bfc6 bcbc b4f3 d1a7 bcc6 cbe3
bbfa d1a7 d4ba ccb7 d6be bba2

• 3、国标码转区位码电路

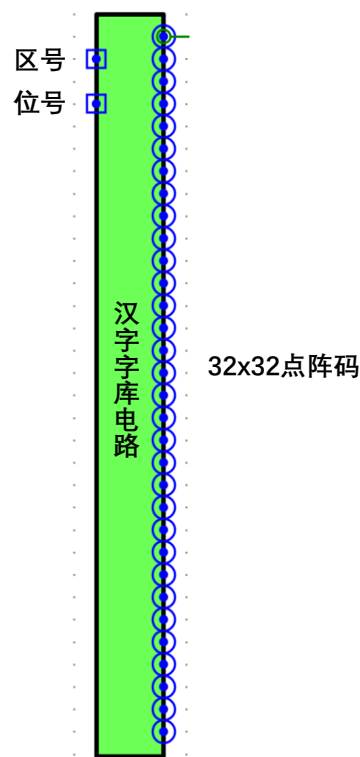
- 输入：16位国标码(GB2312H)
- 输出：区号(7位)、位号(7位)
- 国标码(GB2312H)=区位码+A0A0H
- 区位码=国标码(GB2312H)-A0A0H
- A0A0H=1010 0000 1010 0000
- 取反加1=0101 1111 0110 0000=5F60H
- 因此：区位码=国标码(GB2312)+5F60H



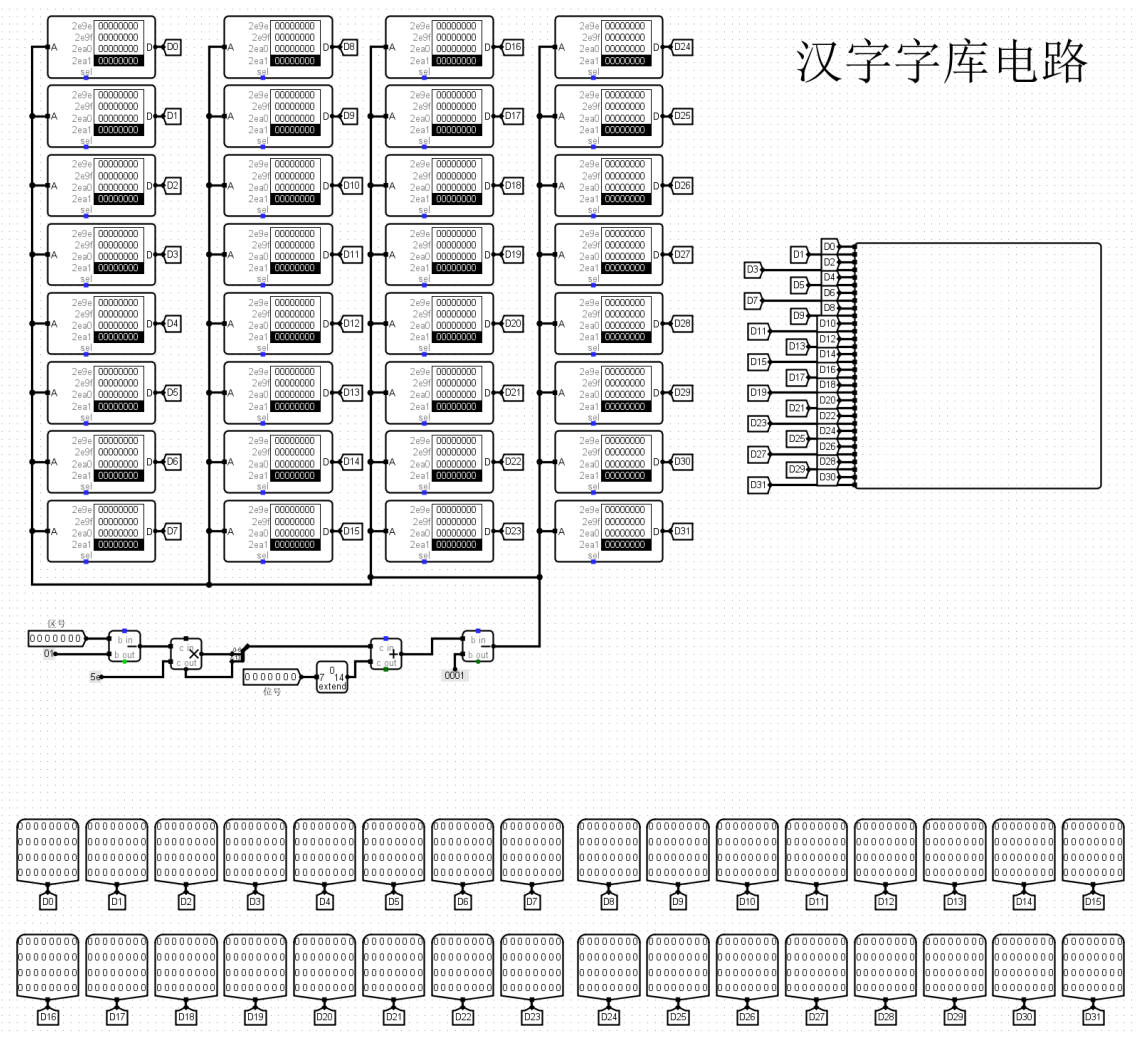
封装后的国标码转区位码电路

• 4、汉字字库电路

- 输入：区位码(区号、位号)
- 输出：汉字的32x32点阵码



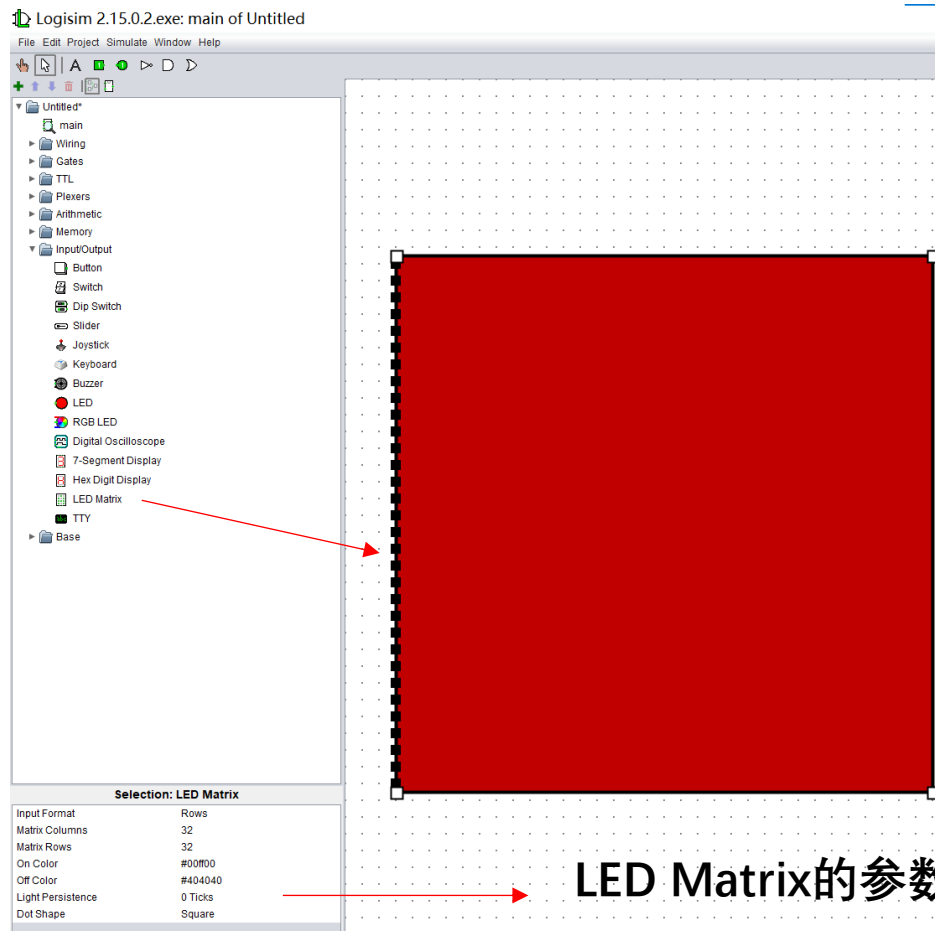
汉字字库电路



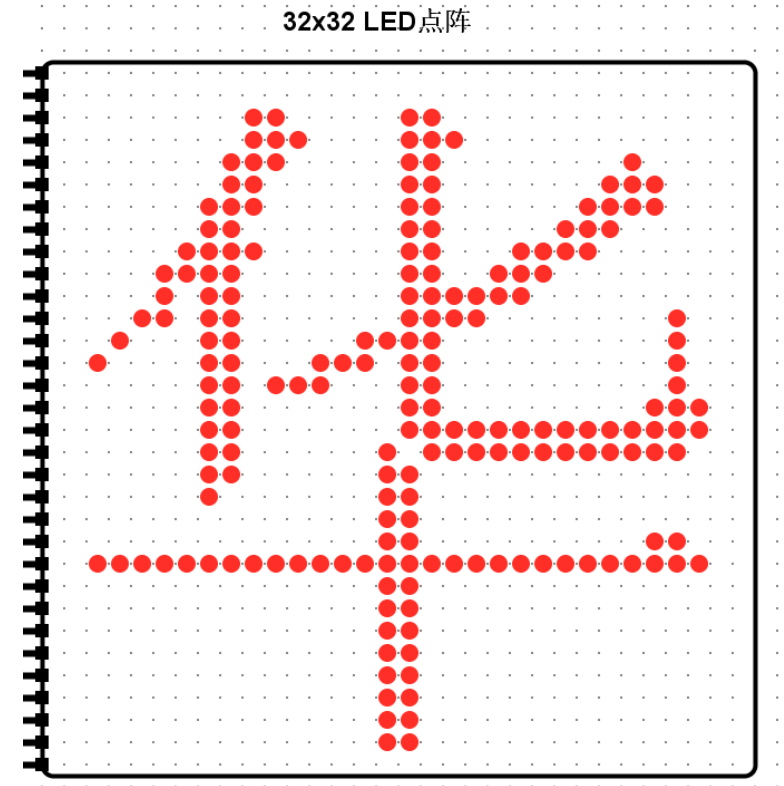
封装后的字库字库电路

• 5、LED点阵 (LED Matrix)

- 输入：汉字的32x32点阵码
- 输出：显示对应的汉字

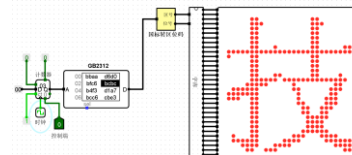
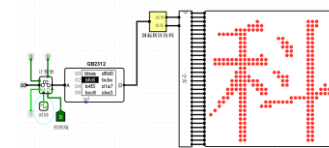
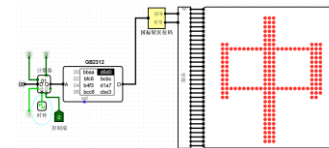
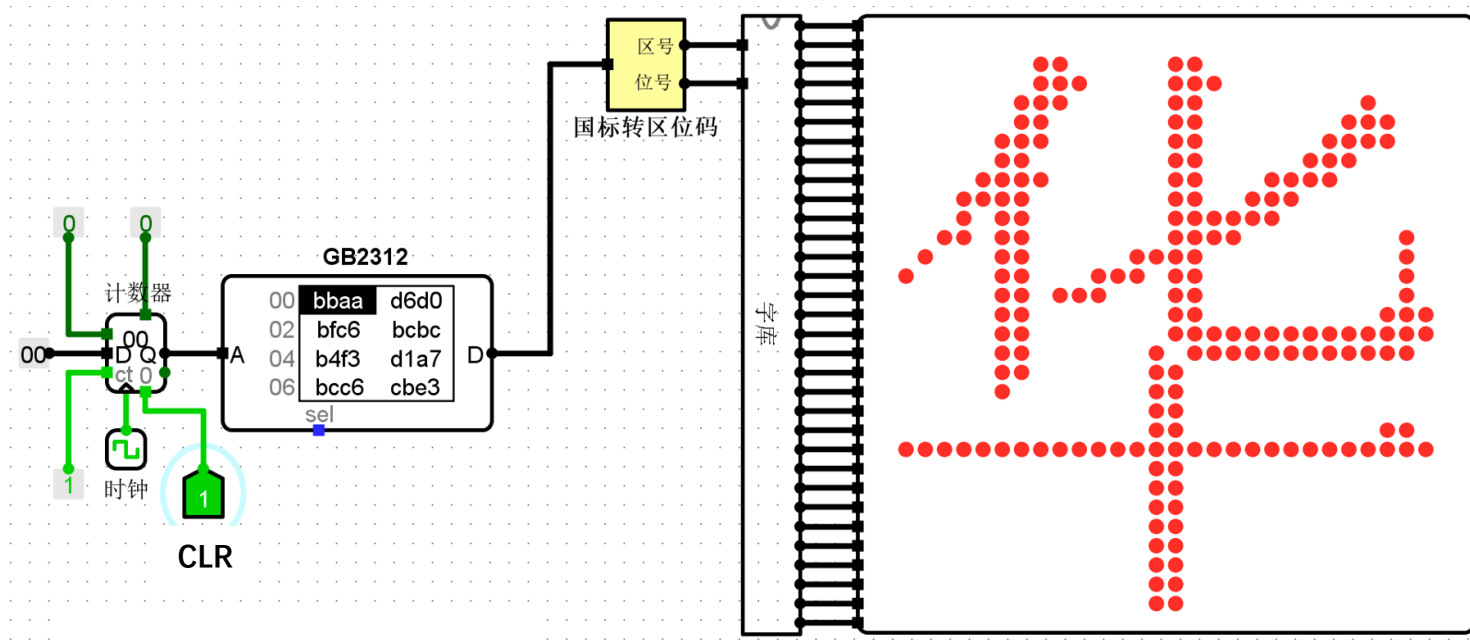


LED Matrix的参数 (可以改变)



• 实验步骤:

- 将文本文件中的汉字国标码（ROM内容——华中科技大学计算机学院谭志虎.txt）装入到ROM存储器中
- 鼠标点击计数器的“CLR”按钮，使其变为1，则计数器清零（从头开始显示汉字）
- 鼠标再次点击计数器的“CLR”按钮，使其变为0，则计数器处于工作状态
- 鼠标每点击2次“时钟”按钮，计数器加1，从ROM存储器中读出1个汉字的国标码，转换为区位码，经字库，得到汉字的点阵码，并在LED点阵上显示



- 本实验的要点：

- 1、汉字的国标码（16位）
- 2、汉字的区位码：区码（7位）、位码（7位）
- 3、汉字的点阵码：例如，32x32点阵码
- 4、国标码转区位码：区位码=国标码(GB2312)+5F60H
- 5、计数器（输入：时钟、CLR；输出：ROM的地址）
- 6、ROM存储器（输入为地址；输出为汉字的国标码）
- 7、**国标码转区位码电路**（输入为国标码；输出为区位码）
- 8、**汉字字库电路**（输入为区位码；输出为汉字的点阵码）

• 请实现：在LED点阵上显示任意汉字字符串，比如：欢迎加入厦门大学信息学院！

- 通过以下的网站查找汉字的国标码：
- http://www.mytju.com/classcode/tools/encode_gb2312.asp

查看字符编码（简体中文）-----GBK内码查询

请输入文字（可以输入多个）：

如：“春眠不觉晓，处处闻啼鸟。”

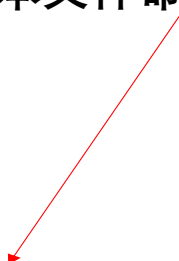
请输入编码（只可输入一个）：

如：“&HB4BA”，“65”

字符	GBK编码10进制	GBK编码16进制 (GBK内码)	Unicode编码10进制	Unicode编码16进制
厦	53187	CFC3	21414	53A6
门	50117	C3C5	38376	95E8
大	46323	B4F3	22823	5927
学	53671	D1A7	23398	5B66

将国标码拷贝到文本文件中，文本文件的最前面增加“v2.0 raw

然后将该文本文件装入到 ROM存储器中（文本文件命名为：**ROM内容——欢迎加入厦门大学信息学院!.txt**）



 ROM内容——华中科技大学计算机学院谭志虎.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

bbaa d6d0 bfc6 bcbc b4f3 d1a7 bcc6 cbe3

bbfa d1a7 d4ba ccb7 d6be bba2

对应教材2.4小节“数据信息的校验”

第三部分 校验码实验

(一) 奇偶校验码实验

(二) 海明校验码实验

(三) 并行循环冗余校验码 (CRC) 实验

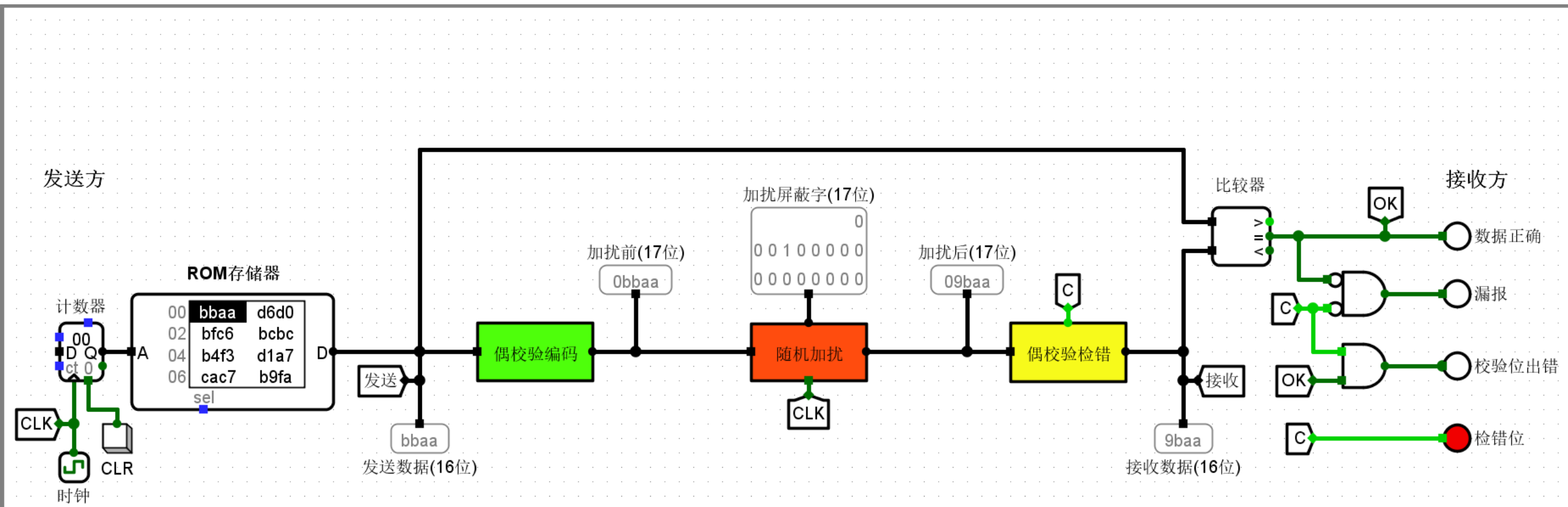
(四) 串行循环冗余校验码 (CRC) 实验

对应教材2.4.2小节“奇偶校验”

(一) 奇偶校验码实验

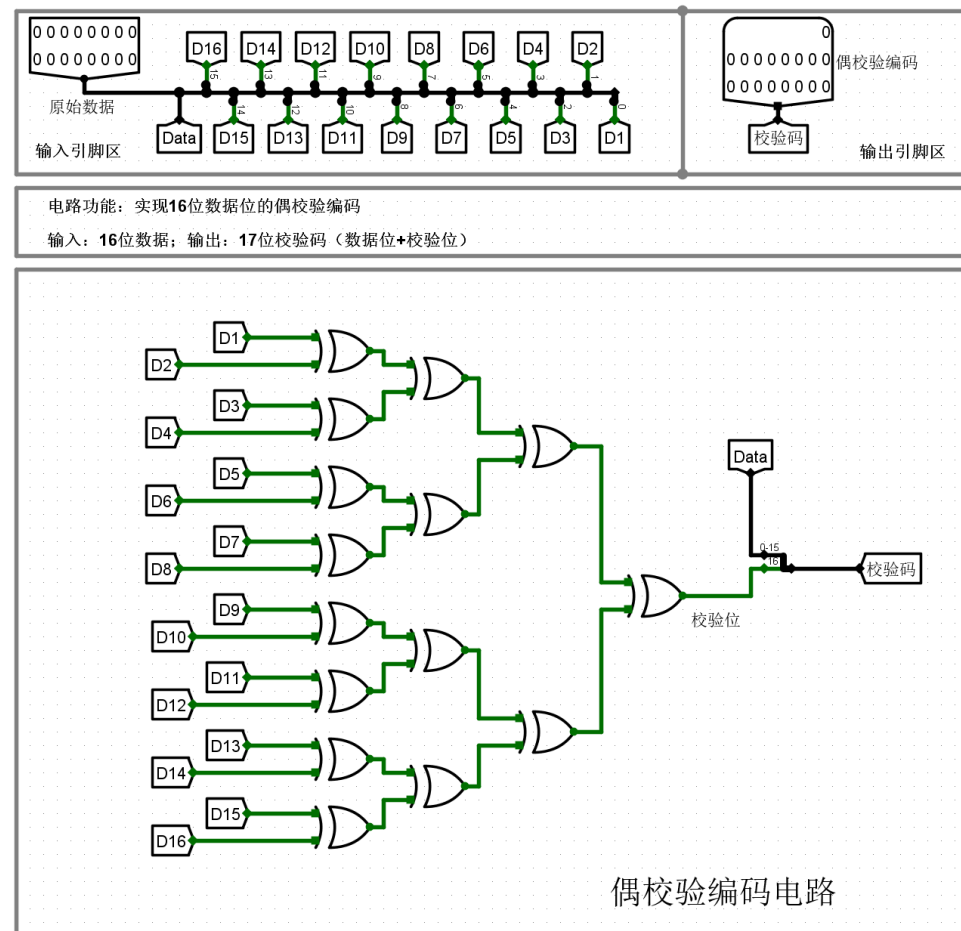
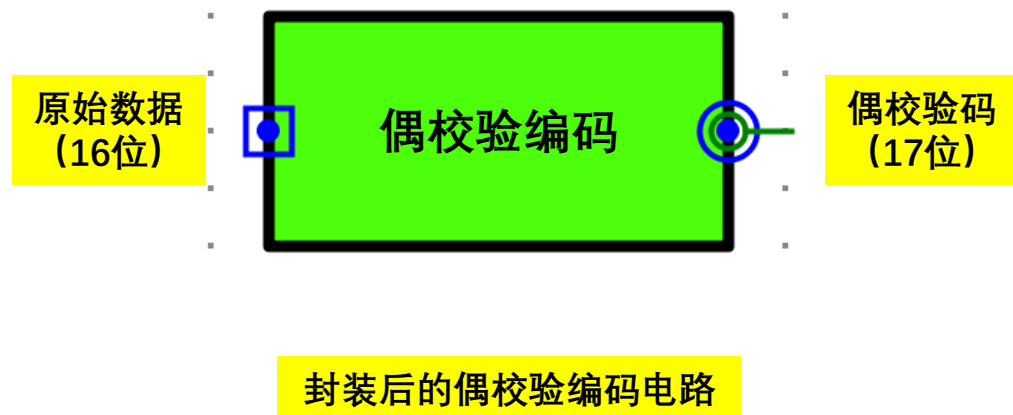
参考设计文件：16位偶校验传输测试实验.circ

- 运行“**Logisim**”，在Logisim中打开“**16位偶校验传输测试实验.circ**”。
- 也可以直接双击“**16位偶校验传输测试实验.circ**”，打开该设计文件。



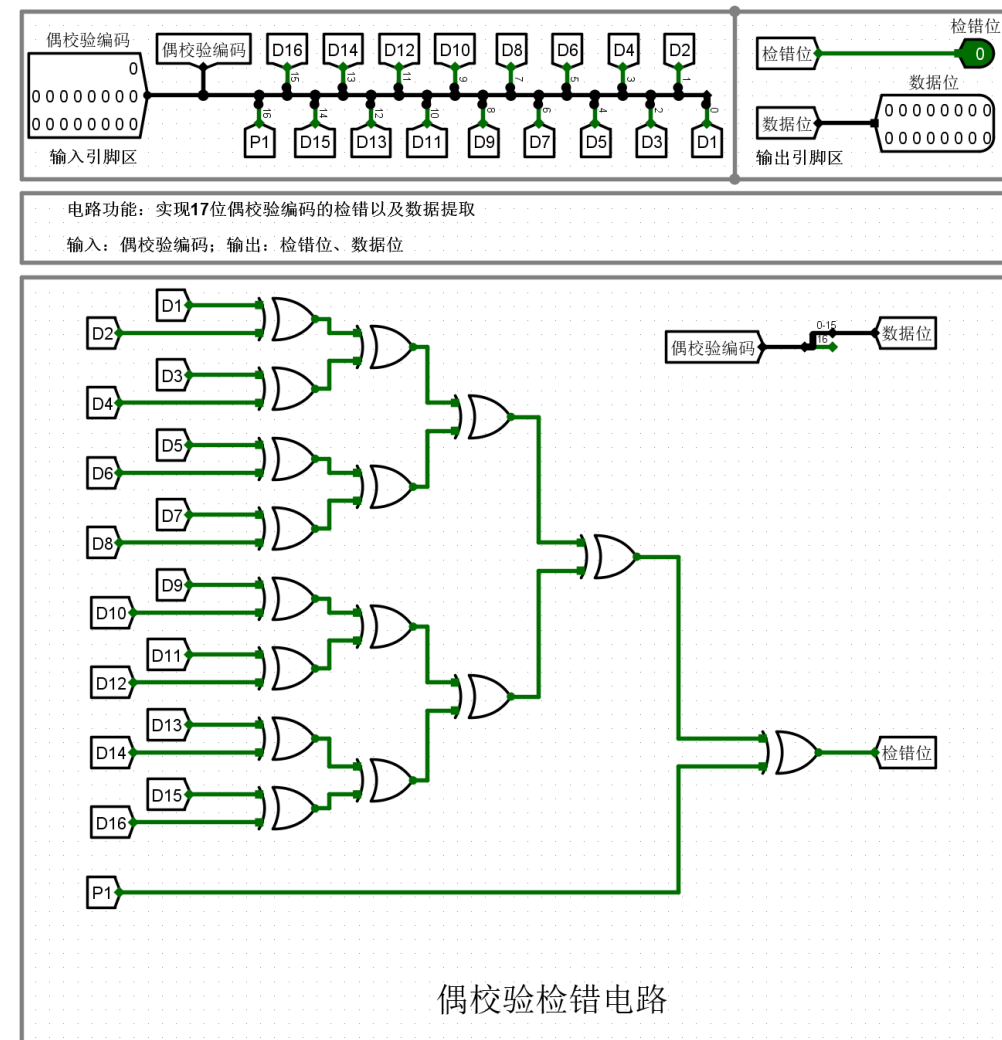
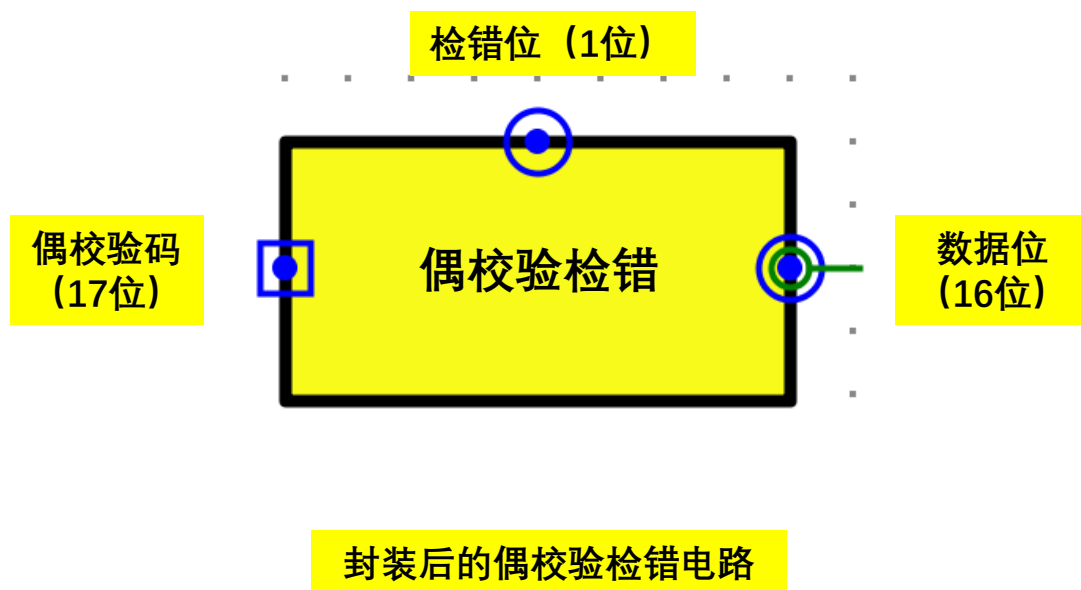
• 1、偶校验编码电路

- 输入：16位原始数据
- 输出：17位偶校验码 = 16位原始数据 + 1位校验位



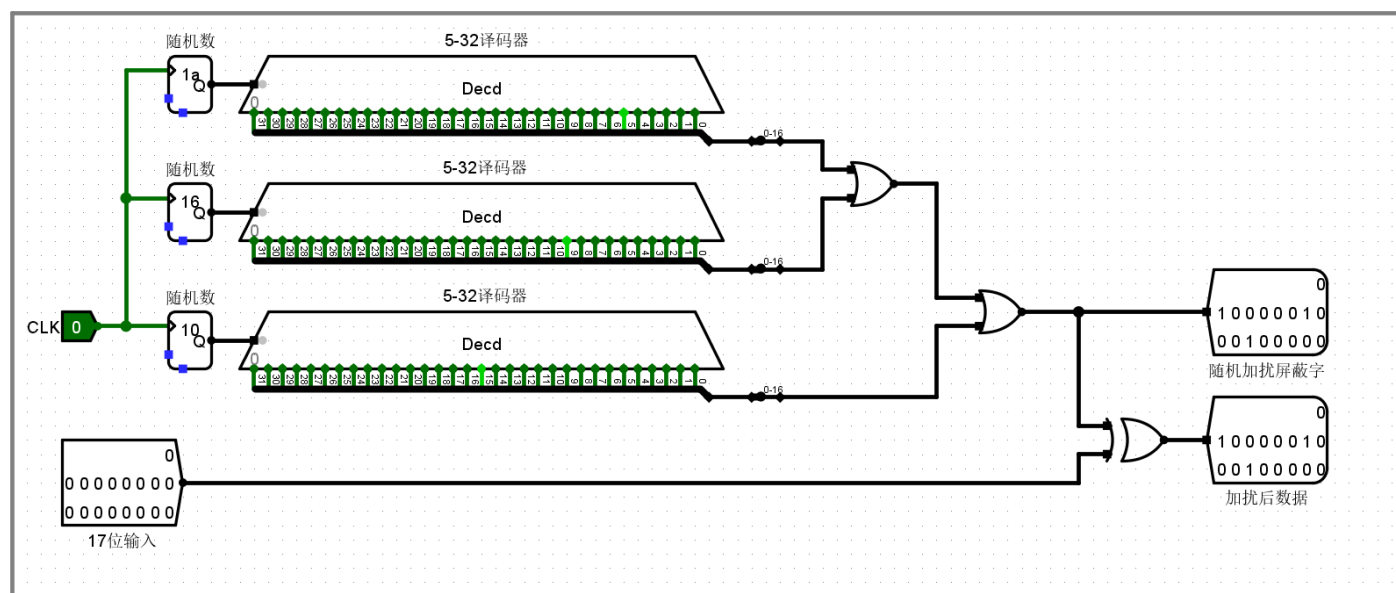
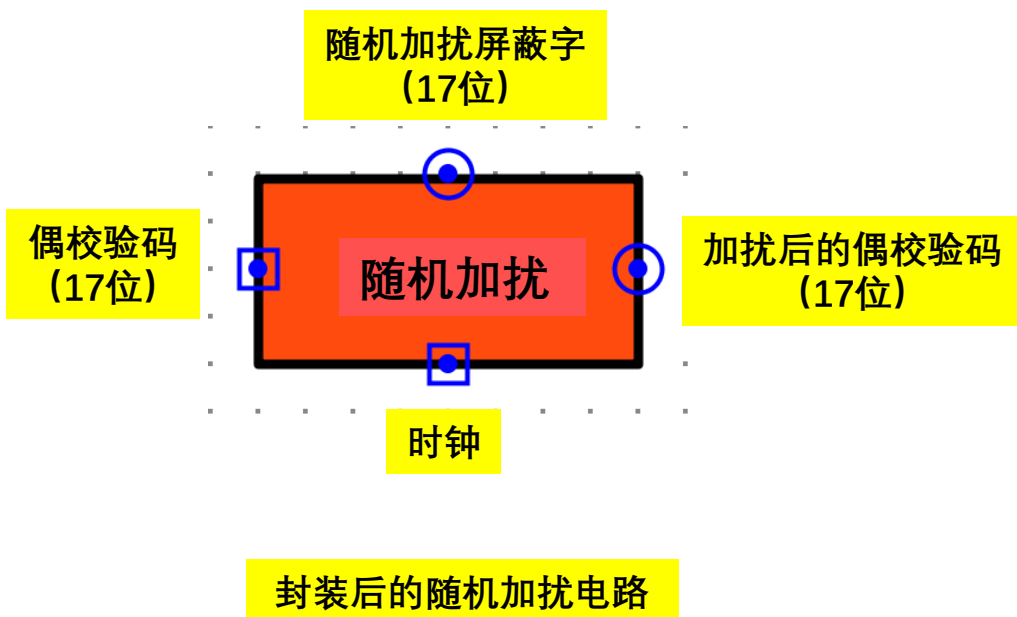
• 2、偶校验检错电路

- 输入：17位偶校验码 = 16位数据位 + 1位校验位
- 输出：1位检错位；16位数据位



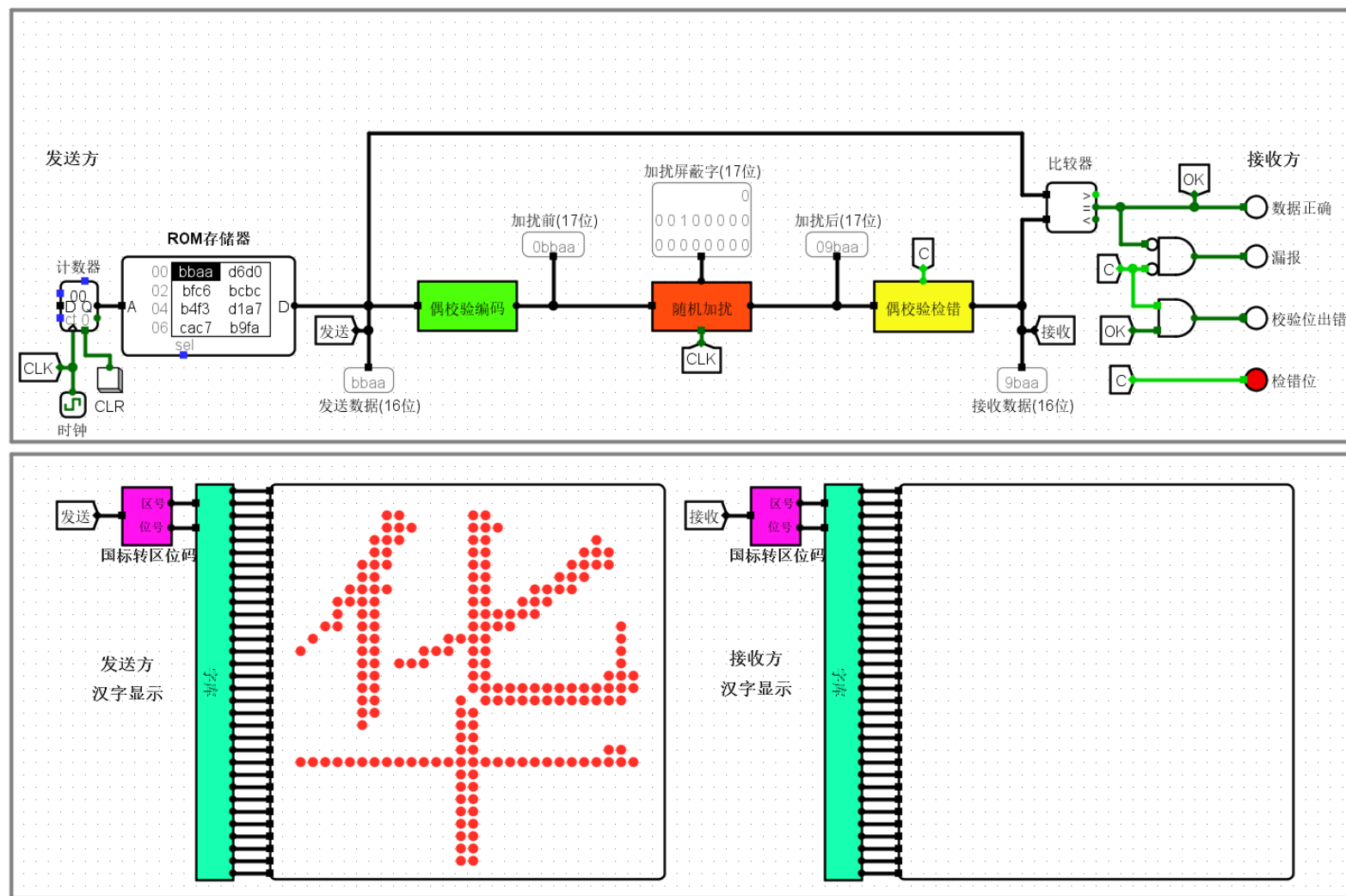
• 3、随机加扰电路

- 输入：17位偶校验码、时钟（CLK）
- 输出：17位加扰后的偶校验码、17位随机加扰屏蔽字
 - 加扰后的偶校验码 = 偶校验码 $\oplus$ 随机加扰屏蔽字
 - 每按2次“时钟”按钮，会产生1个新的随机加扰屏蔽字



随机加扰电路

• 4、16位偶校验传输测试实验步骤



实验步骤：

1、按“CLR”按钮清零计数器

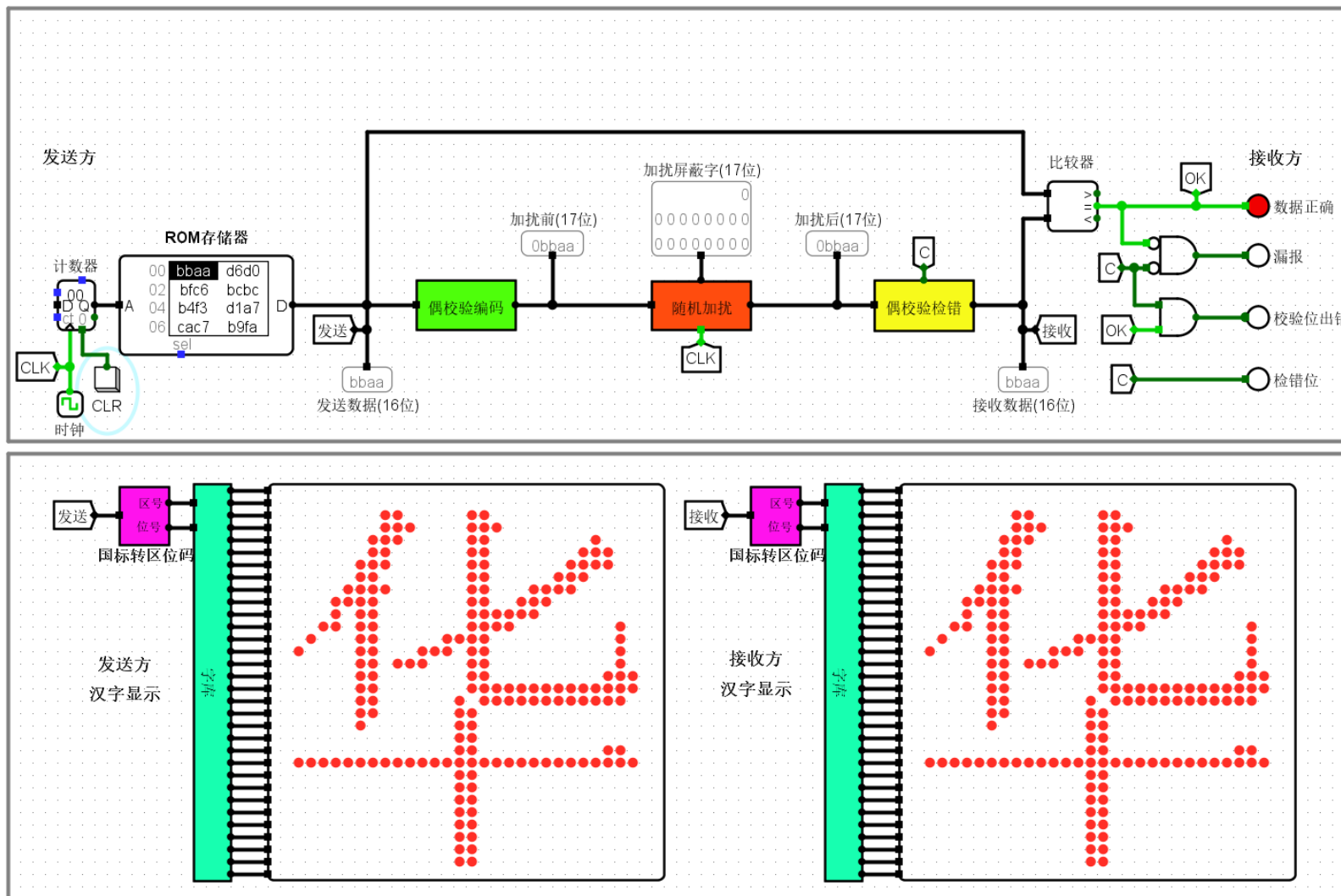
2、每按2次“时钟”按钮使计数器加1，ROM存储器输出1个汉字国标码

3、观看右上方的4个指示灯和下面的发送方与接收方显示的汉字

4、有4种情况：

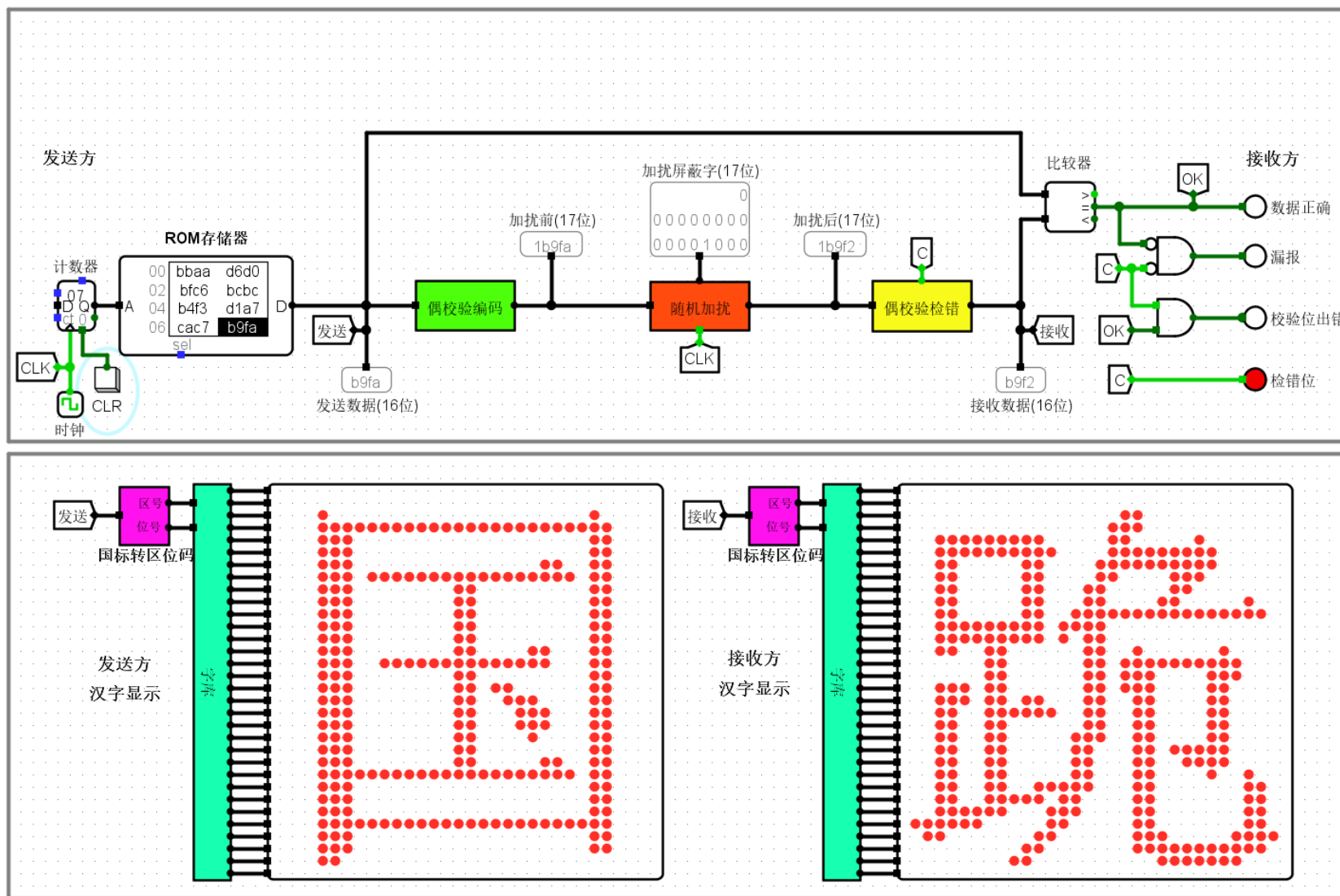
16位偶校验传输测试实验

- 第1种情况：加扰屏蔽字=0 0000 0000 0000 0000，**无错**；数据正确=1（红灯）、漏报=0（白灯）、校验位出错=0（白灯）、检错位=0（白灯），发送方与接收方显示的汉字相同。



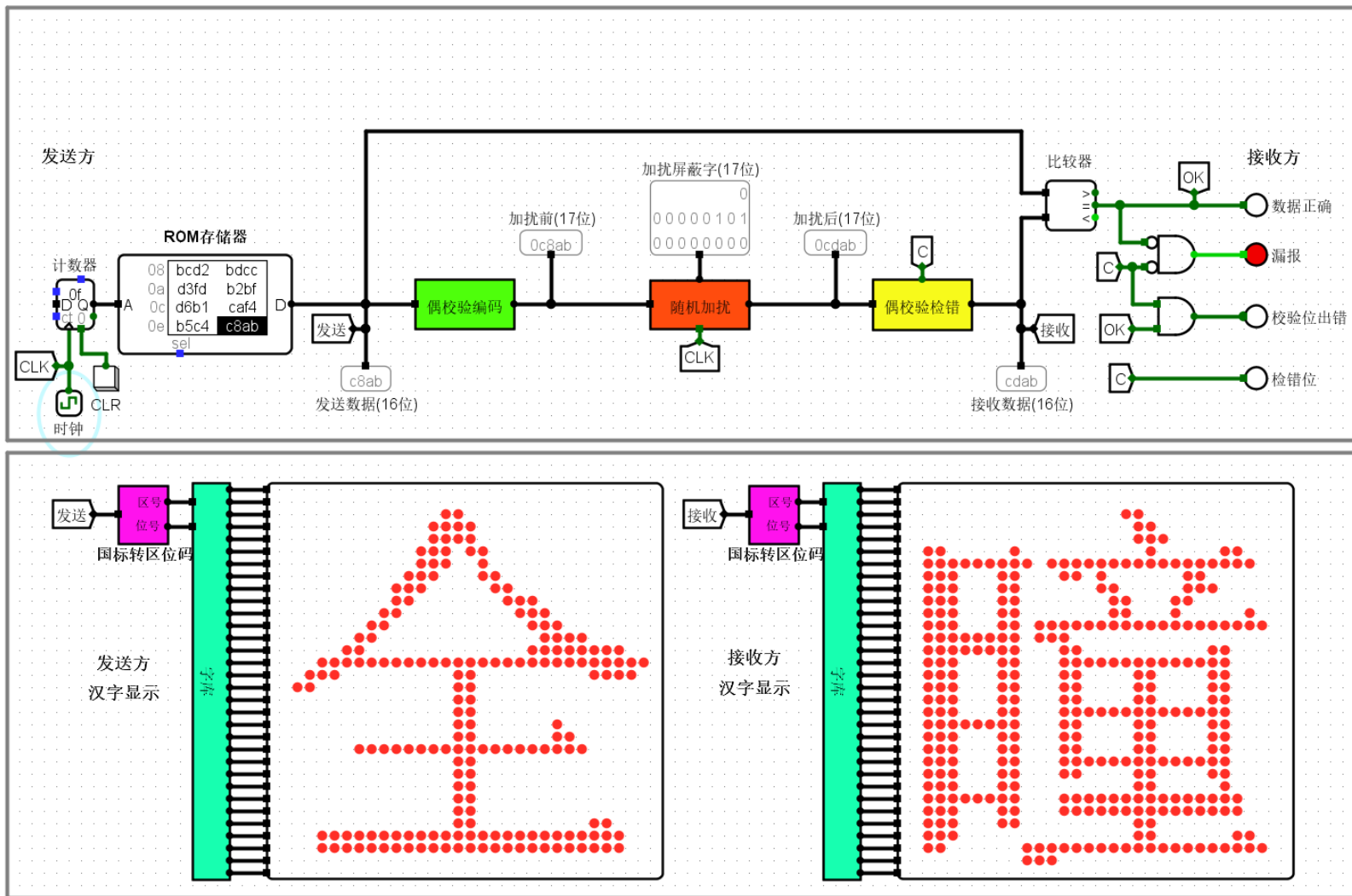
16位偶校验传输测试实验

- 第2种情况：加扰屏蔽字=0 0000 0000 0000 1000，**1位错**；数据正确=0（白灯）、漏报=0（白灯）、校验位出错=0（白灯）、检错位=1（红灯），发送方与接收方显示的汉字不同，表示**奇偶校验码可以发现1位错、但不能纠正1位错**。



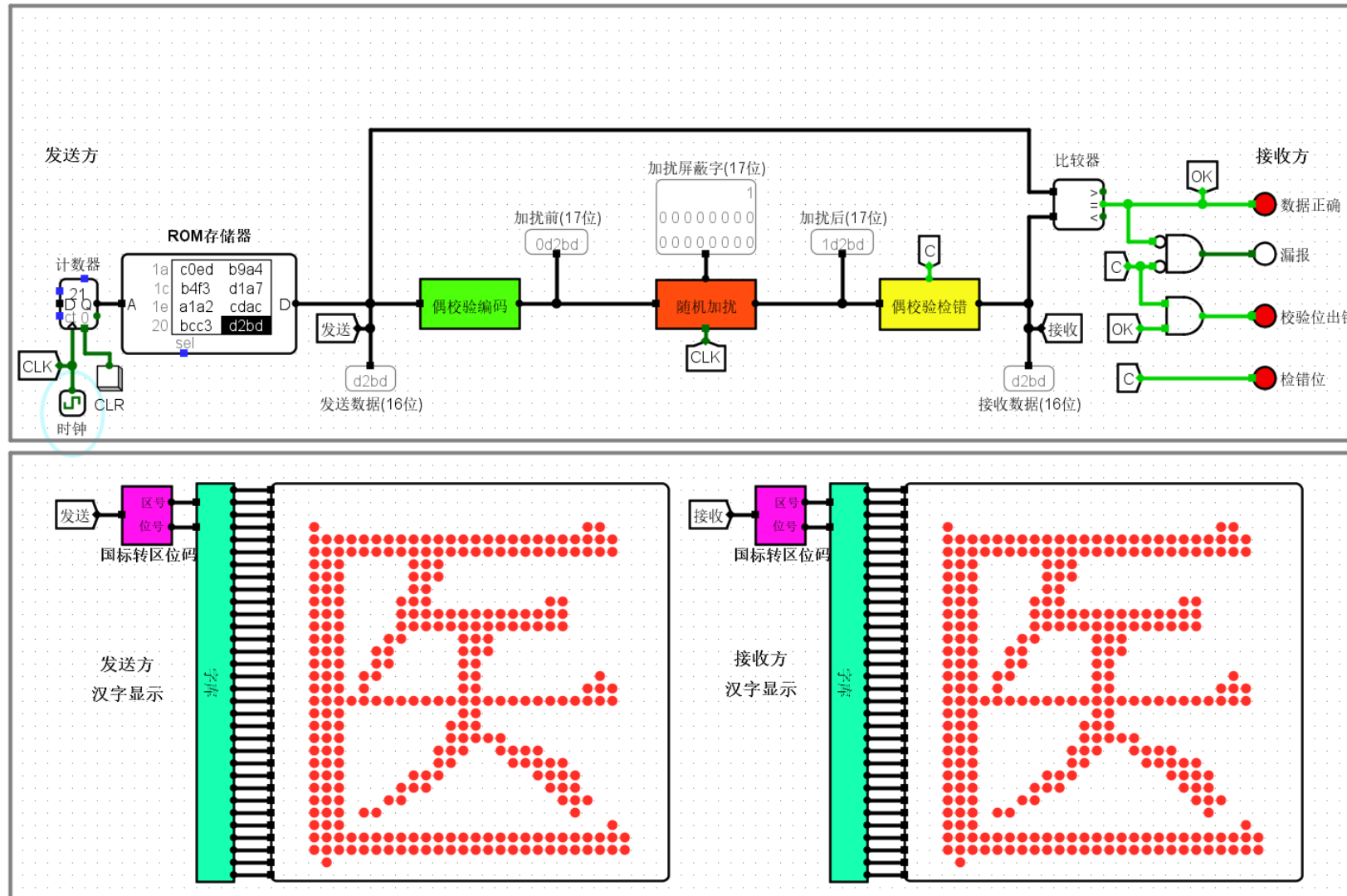
16位偶校验传输测试实验

- 第3种情况：加扰屏蔽字=0 0000 0101 0000 0000，**2位错**；数据正确=0（白灯）、漏报=1（红灯）、校验位出错=0（白灯）、检错位=0（白灯），发送方与接收方显示的汉字不同，表示**奇偶校验码不能发现2位错、也不能纠正2位错**。



16位偶校验传输测试实验

- 第4种情况（这种情况比较少，需要多按几次时钟按钮才会出现）：加扰屏蔽字=1 0000 0000 0000 0000，**1位错**；数据正确=1（红灯）、漏报=0（白灯）、校验位出错=1（红灯）、检错位=1（红灯），发送方与接收方显示的汉字相同，表示**校验位出错、数据是正确的**。



16位偶校验传输测试实验

- 本实验的要点：

- 1、由16位原始数据，怎么计算1位偶校验位（偶校验编码电路）？如果要计算1位奇校验位，电路怎么修改？（下面的设计实验内容）
- 2、由17位偶校验码，怎么计算1位偶检错位（偶校验检错电路）？如果要计算1位奇检错位，电路怎么修改？（下面的设计实验内容）
- 3、为什么会出现误报现象（第2种情况）？
- 4、为什么会出现第4种情况（检错位=1，但是数据正确=1，即接收数据=发送数据）？
- 5、如果原始数据是8位，电路怎么修改？（下面的设计实验内容）

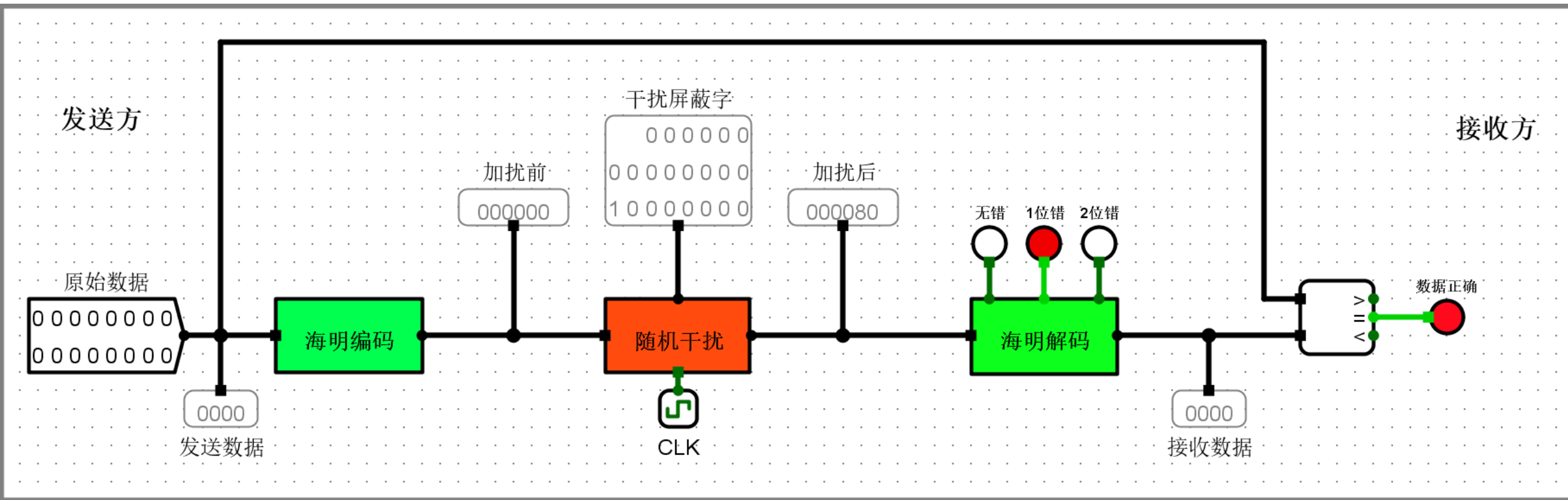
- 1、请同学们修改前面的“16位偶校验传输测试实验”的电路，使其变为“16位奇校验传输测试实验”电路（设计文件命名为：**16位奇校验传输测试实验.circ**）。
- 2、请同学们修改前面的“16位偶校验传输测试实验”的电路，使其变为“8位偶校验传输测试实验”的电路（设计文件命名为：**8位偶校验传输测试实验.circ**）。

对应教材2.4.3小节“海明校验”

(二) 海明校验码实验

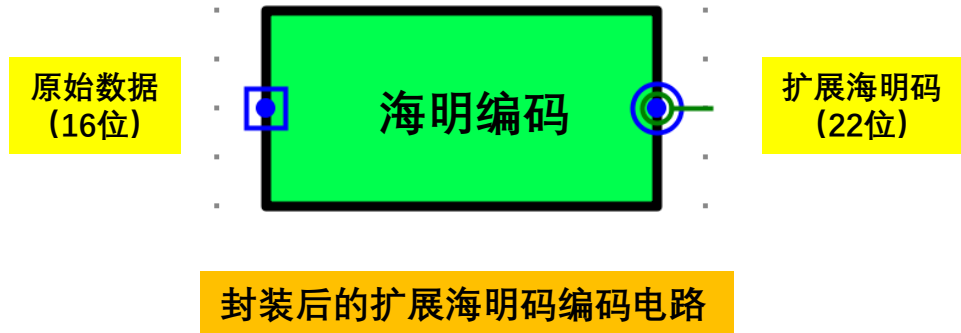
参考设计文件：16位扩展海明码传输测试实验.circ

- 运行“Logisim”，在Logisim中打开“16位扩展海明码传输测试实验.circ”。
- 也可以直接双击“16位扩展海明码传输测试实验.circ”，打开该设计文件。

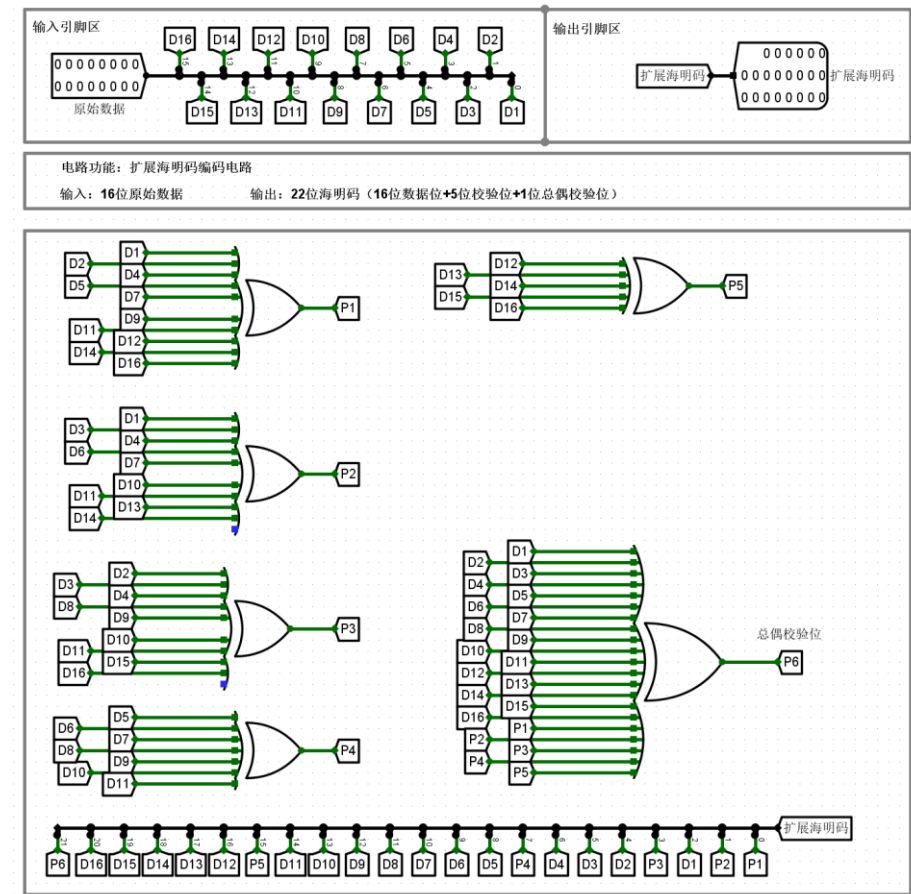


1、扩展海明码编码电路

- 输入：16位原始数据
- 输出：22位扩展海明码=16位原始数据+6位校验码
- 6位校验码=5位校验码+1位总偶校验位
- $n=k+r \leq 2^r - 1$
- $k=16, r=5, n=21 \leq 2^5 - 1 = 31$
- $k=16, r=4, n=20 > 2^4 - 1 = 15$



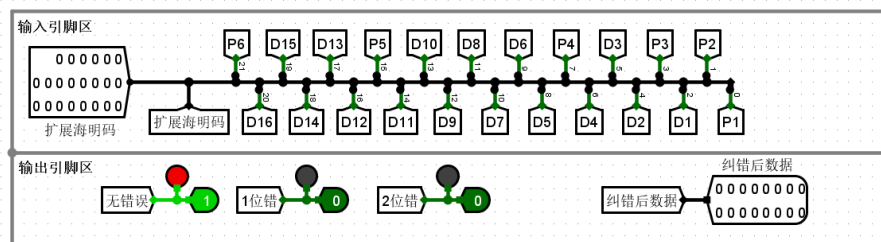
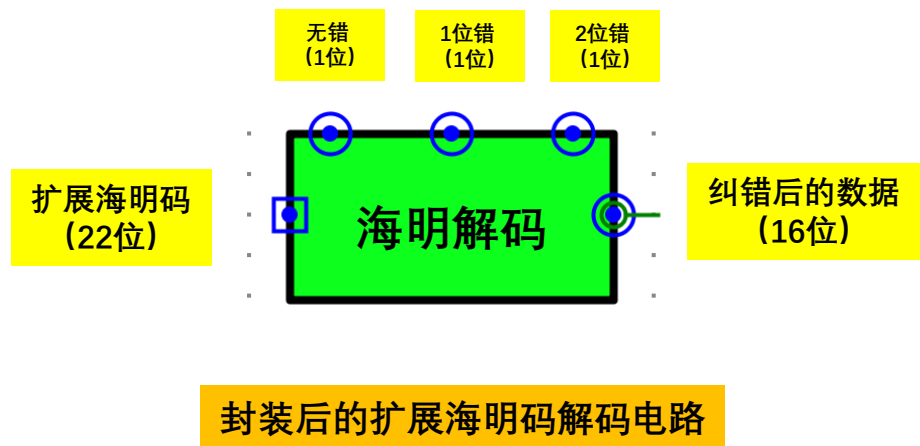
封装后的扩展海明码编码电路



	P1	P2	D1	P3	D2	D3	D4	P4	D5	D6	D7	D8	D9	D10	D11	P5	D12	D13	D14	D15	D16
G1	1		1		1		1		1		1		1		1		1		1		1
G2		1	1			1	1			1	1			1	1			1	1		
G3				1	1	1	1					1	1	1	1					1	1
G4								1	1	1	1	1	1	1	1						
G5																1	1	1	1	1	1

• 2、扩展海明码解码电路

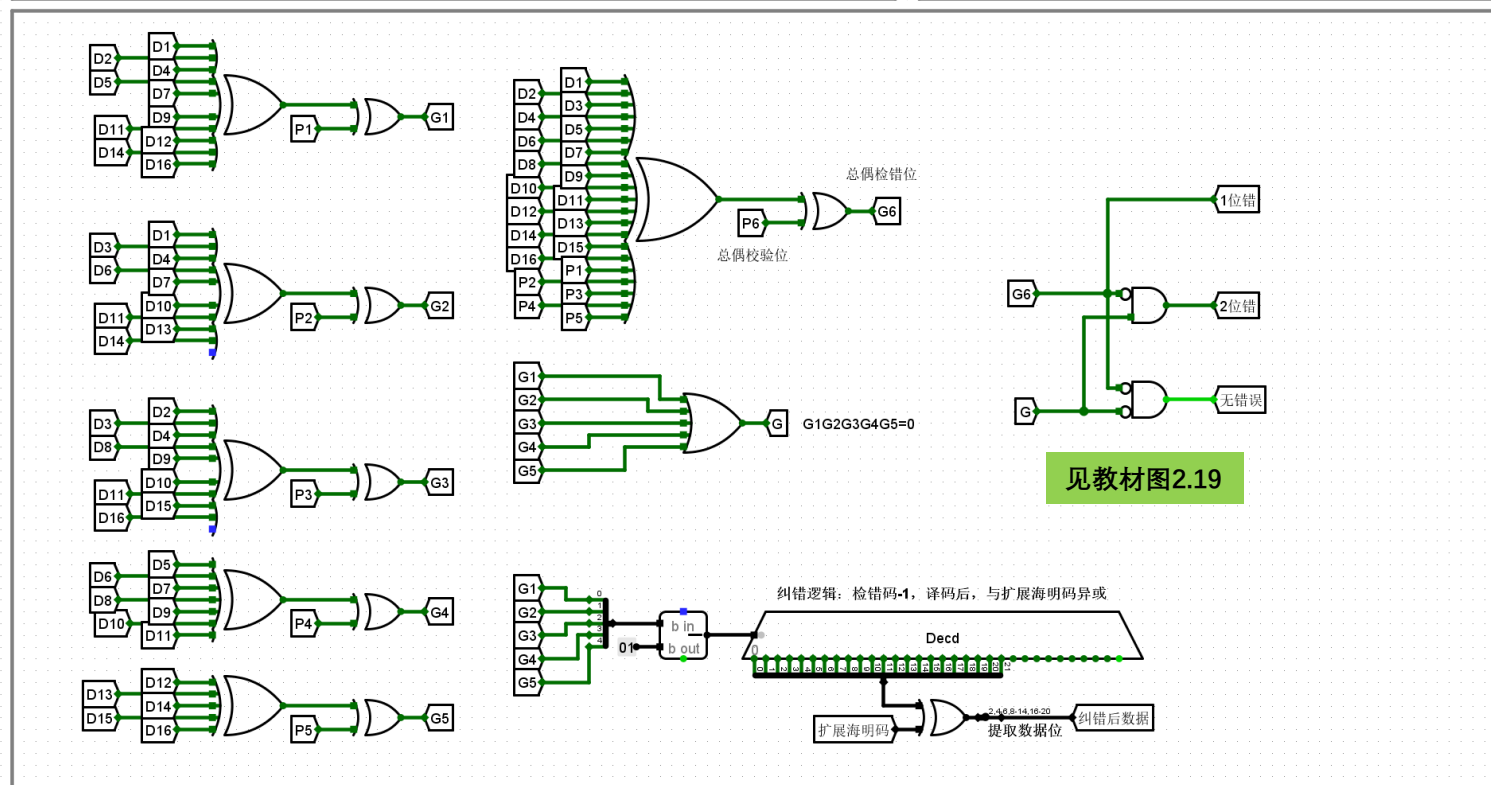
- 输入：22位扩展海明码
- 输出：
 - 无错误标志（1位）
 - 1位错标志（1位）
 - 2位错标志（1位）
 - 纠错后的数据（16位）



电路功能：22位海明码的数据解码，能判断无错误、1位错、2位错，1位错能自动纠错

输入：扩展海明码（22位）

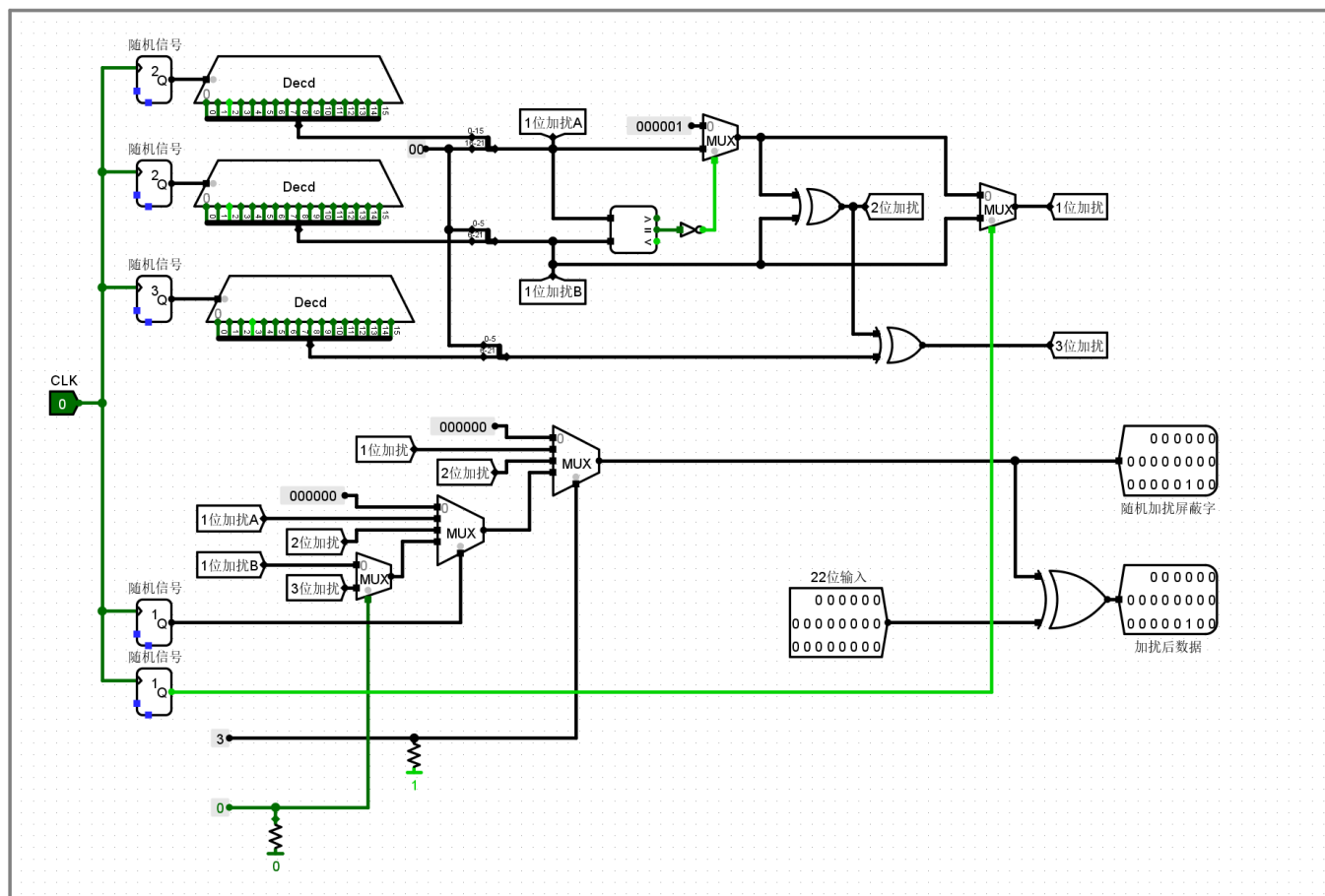
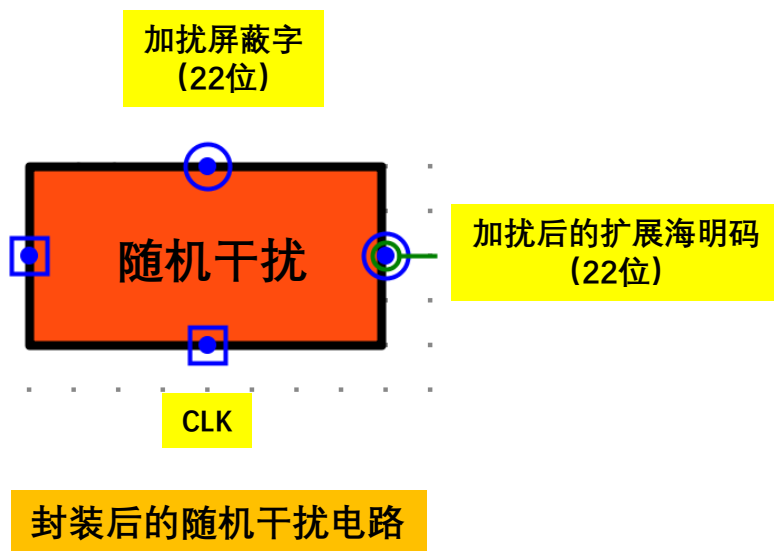
输出：错误模式（无错误、1位错、2位错），纠错后的数据（16位）



见教材图2.19

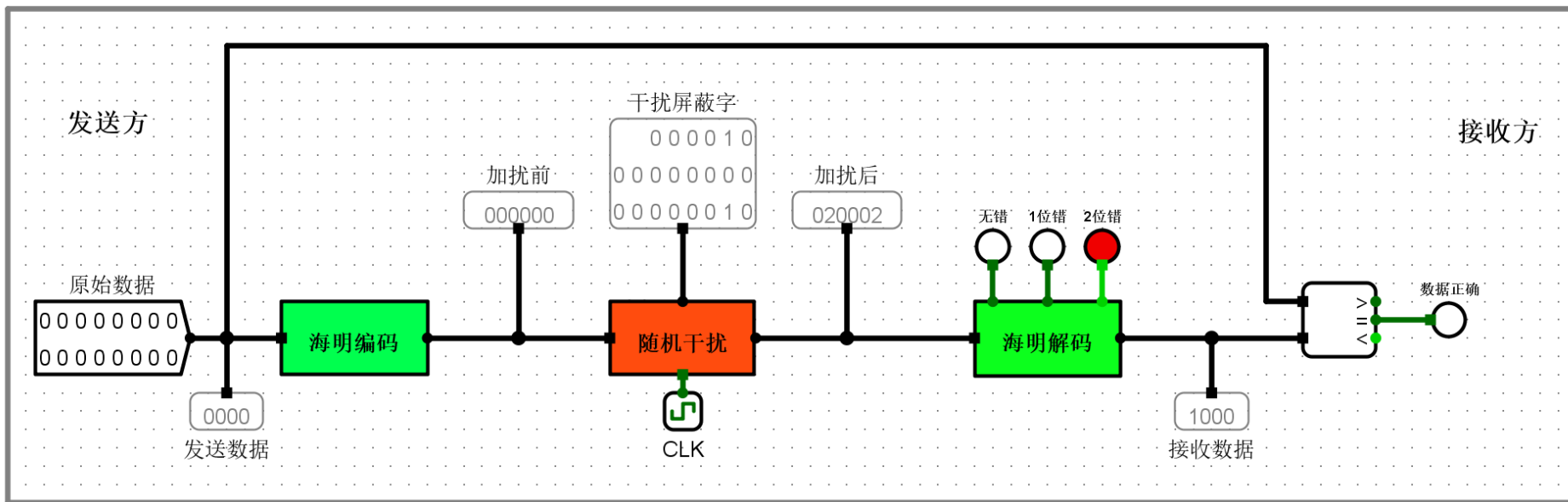
• 3、随机干扰电路

- 输入：22位扩展海明码、时钟（CLK）
- 输出：22位加扰后的扩展海明码、22位加扰屏蔽字
- 每按2次“时钟”按钮将产生不同的加扰屏蔽字



• 4、16位扩展海明码传输测试实验1

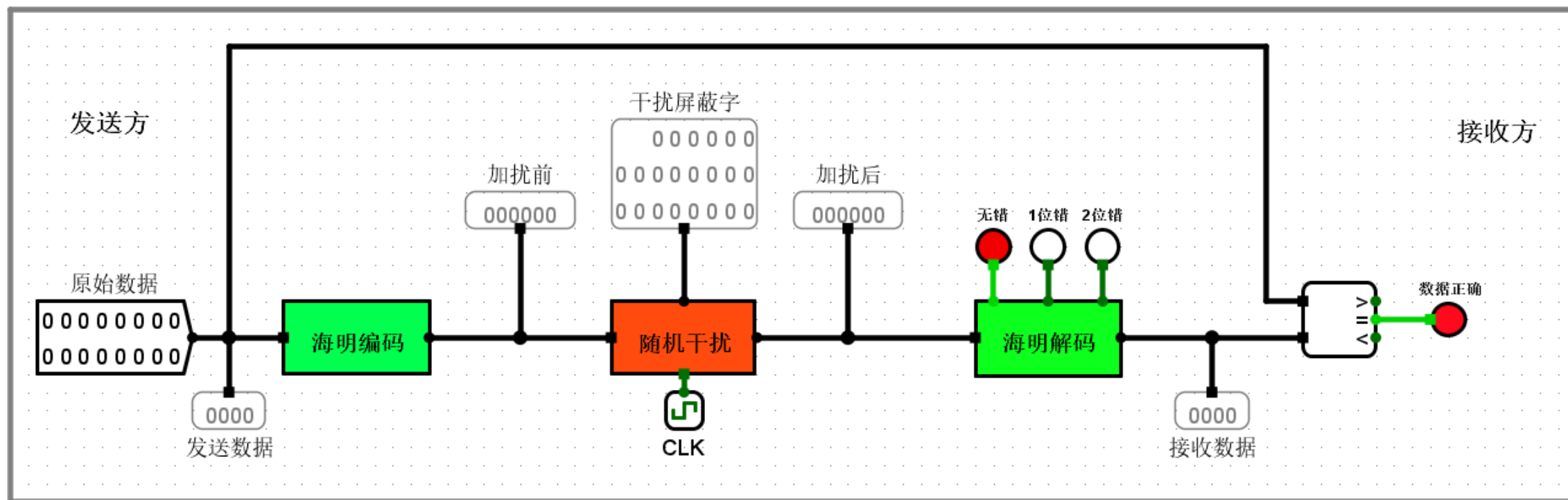
- 输入：16位原始数据（发送数据）、时钟（CLK）
- 输出：16位接收到的数据，4个指示灯（无错、1位错、2位错、数据正确）



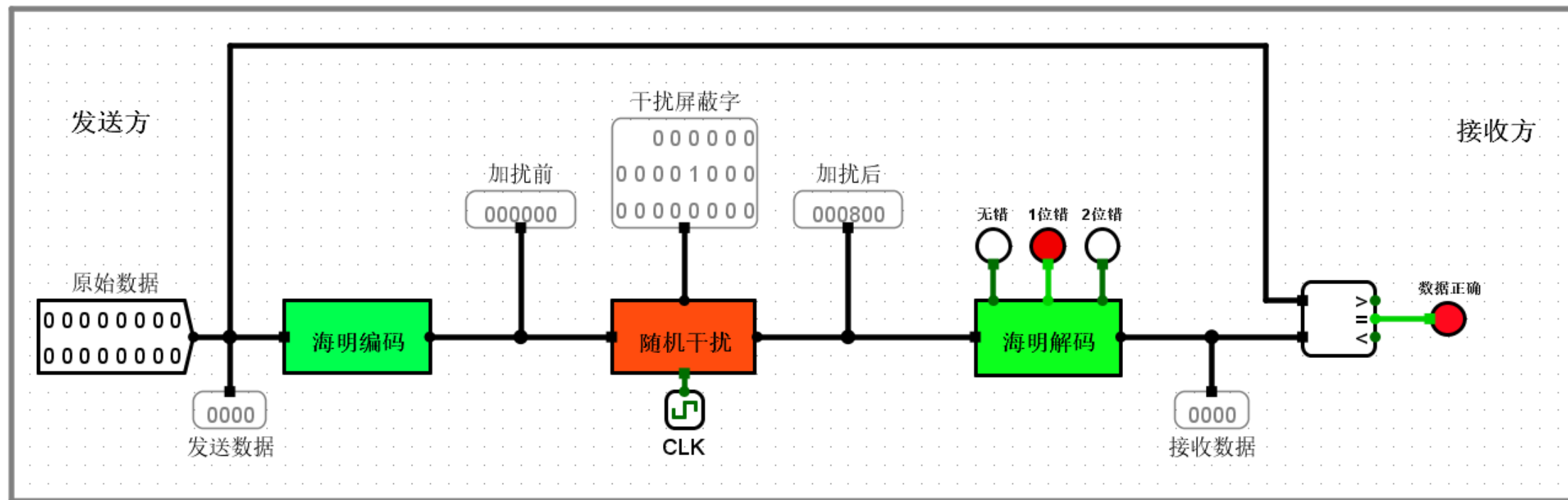
• 实验步骤：每按2次“CLK”按钮，观看4个指示灯，有4种情况：

- (1) 无错：无错指示灯=红色，数据正确指示灯=红色，干扰屏蔽字=0，接收数据=发送数据
- (2) 1位错：1位错指示灯=红色，数据正确指示灯=红色，干扰屏蔽字中有1个1，接收数据=发送数据（可以纠正1位错）
- (1) 2位错：2位错指示灯=红色，数据正确指示灯=白色，干扰屏蔽字中有2个1，接收数据≠发送数据（不能纠正2位错）
- (1) 2位错：2位错指示灯=红色，数据正确指示灯=红色，干扰屏蔽字中有2个1，接收数据=发送数据（此时可以纠正错）

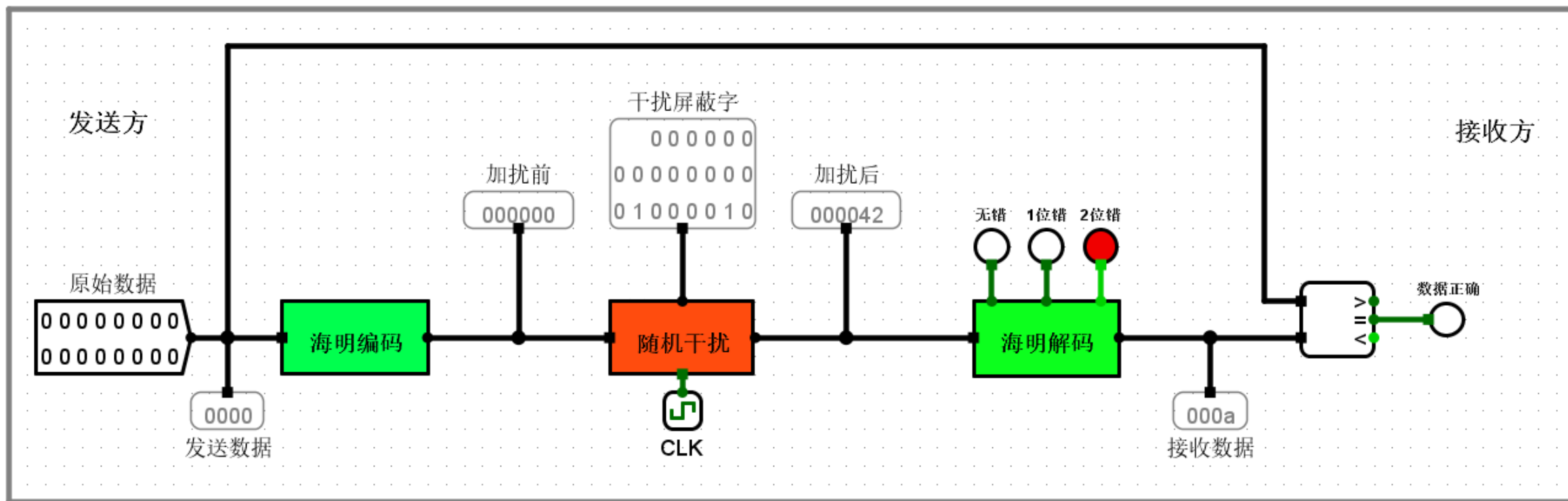
- 第1种情况：加扰屏蔽字=00 0000 0000 0000 0000 0000，**无错**=1（红灯）、数据正确=1（红灯），接收数据与发送数据相同。



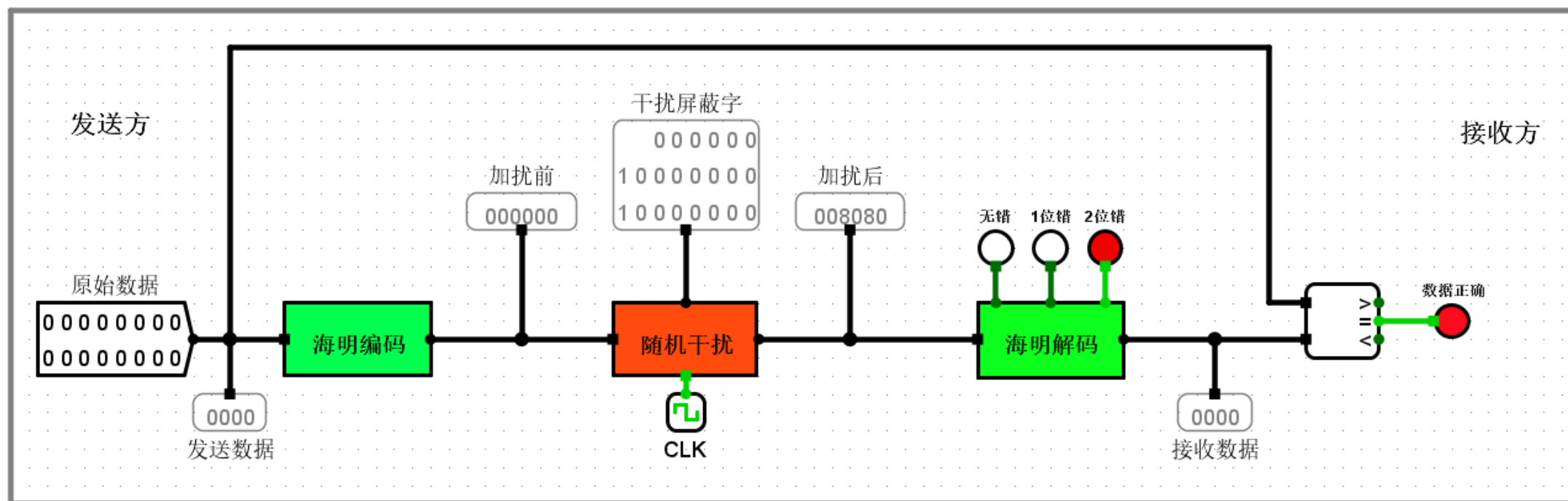
- 第2种情况：加扰屏蔽字=00 0000 0000 1000 0000 0000，1位错=1（红灯）、数据正确=1（红灯），接收数据与发送数据相同，表示扩展海明码可以发现1位错、纠正1位错。



- 第3种情况：加扰屏蔽字=00 0000 0000 0000 0100 0010，**2位错**=1（红灯）、数据正确=0（白灯），接收数据与发送数据不同，表示**扩展海明码可以发现2位错、但不能纠正2位错**。

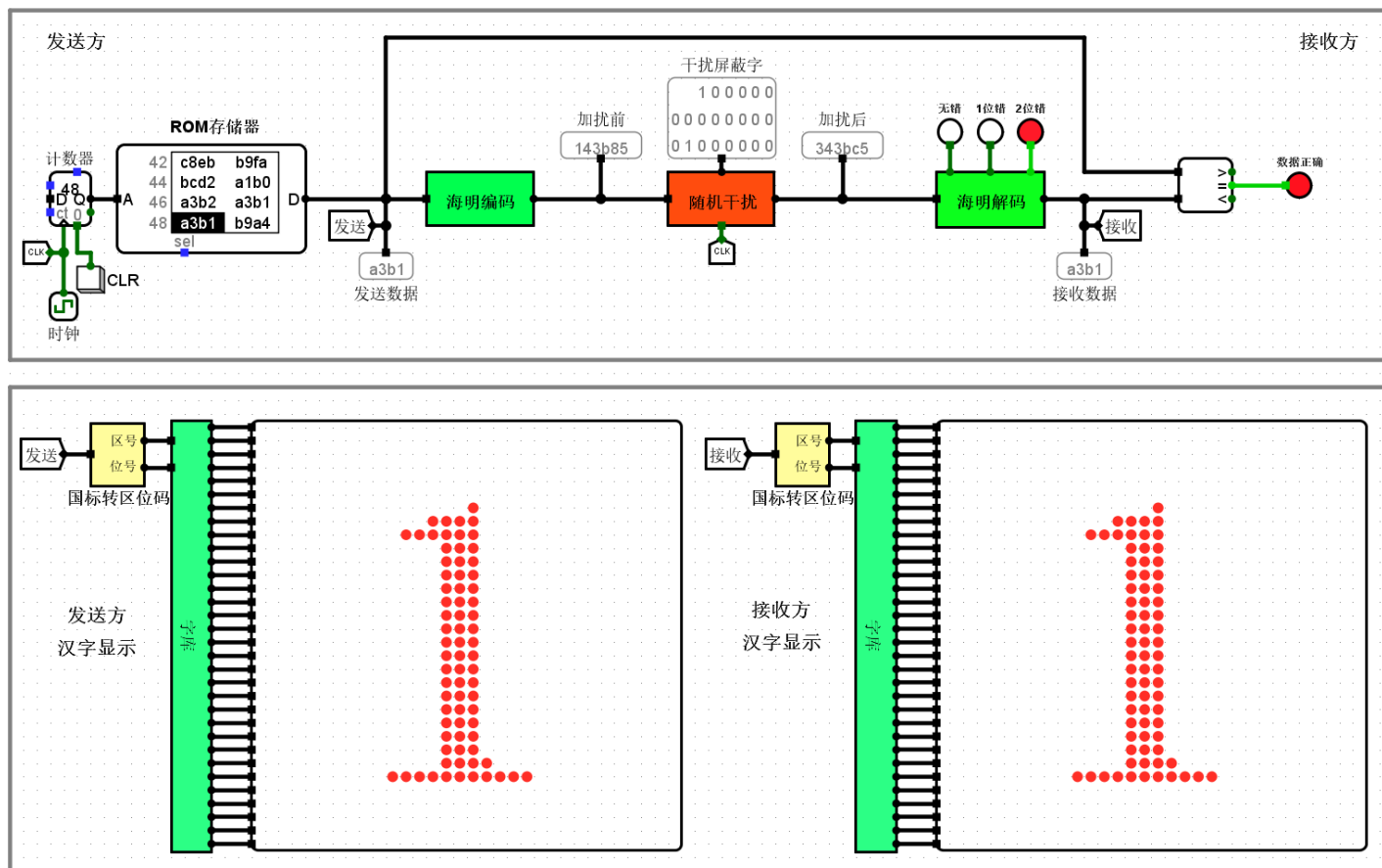


- 第4种情况（需要按多次CLK按钮才会出现这种情况）：干扰屏蔽字=00 0000 1000 0000 1000 0000，2位错=1（红灯）、数据正确=1（红灯），接收数据与发送数据相同；原因是2位错中有1位是P6，或2位错是P4和P5，因此可以纠正。



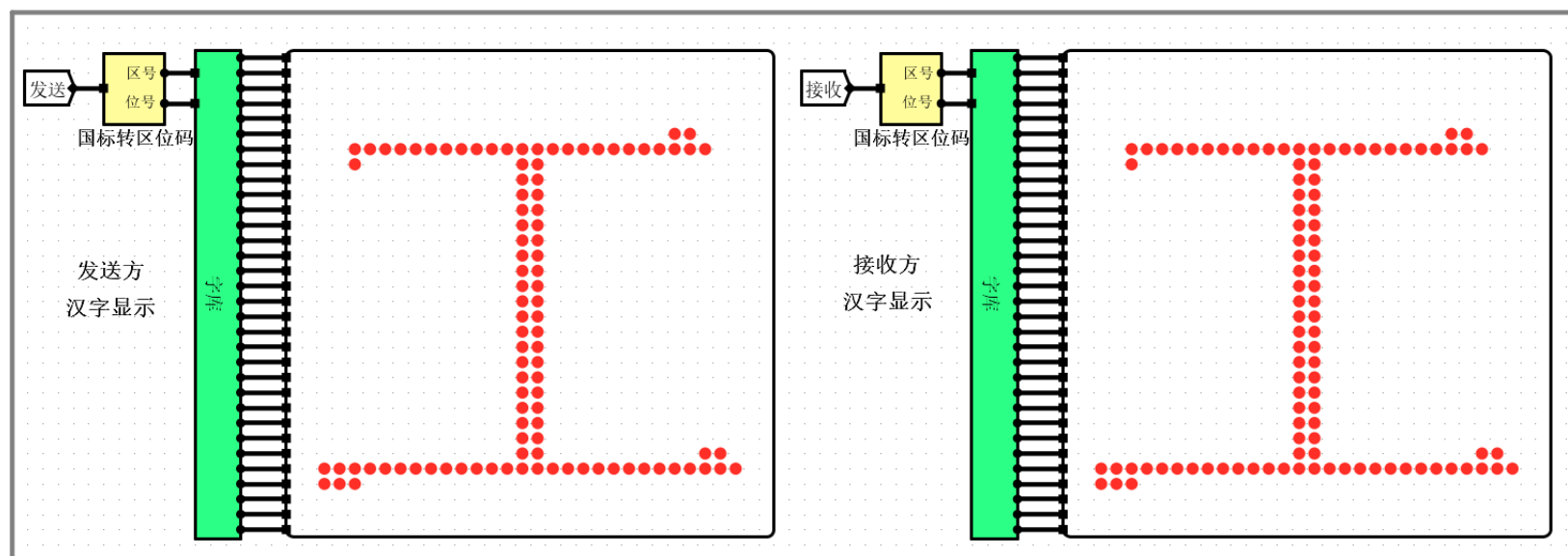
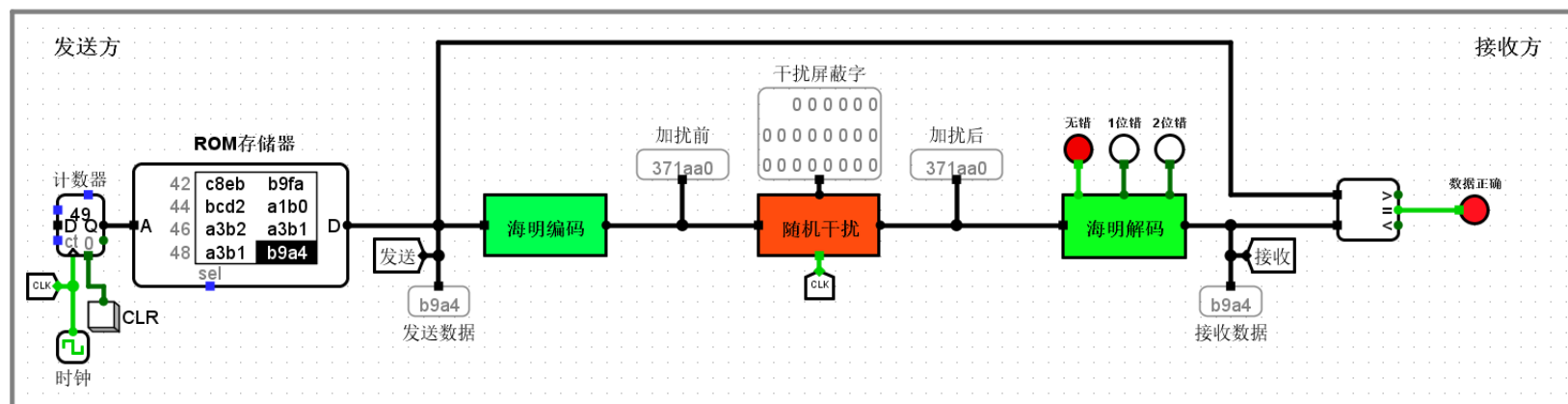
• 5、16位扩展海明码传输测试实验2

- 输入：16位原始数据（发送数据，汉字的国标码，存放在ROM存储器中）、时钟CLK、CLR（清零控制信号）
- 输出：16位接收到的数据（汉字的国标码），4个指示灯（无错、1位错、2位错、数据正确）

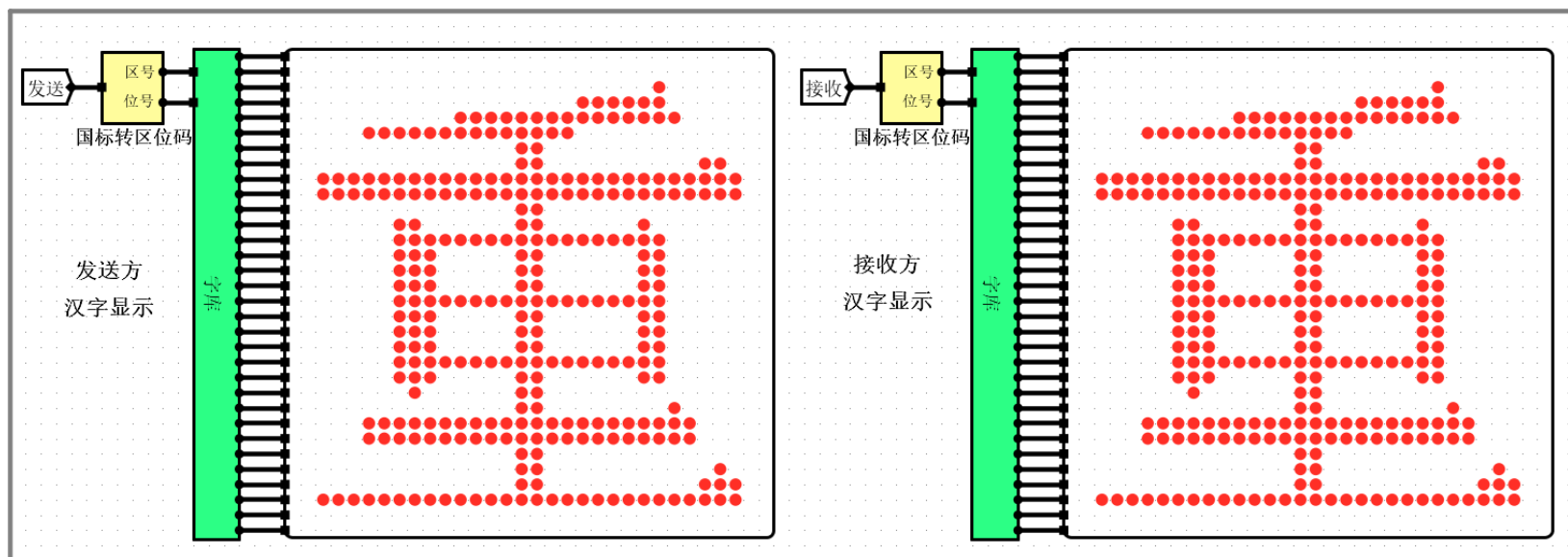
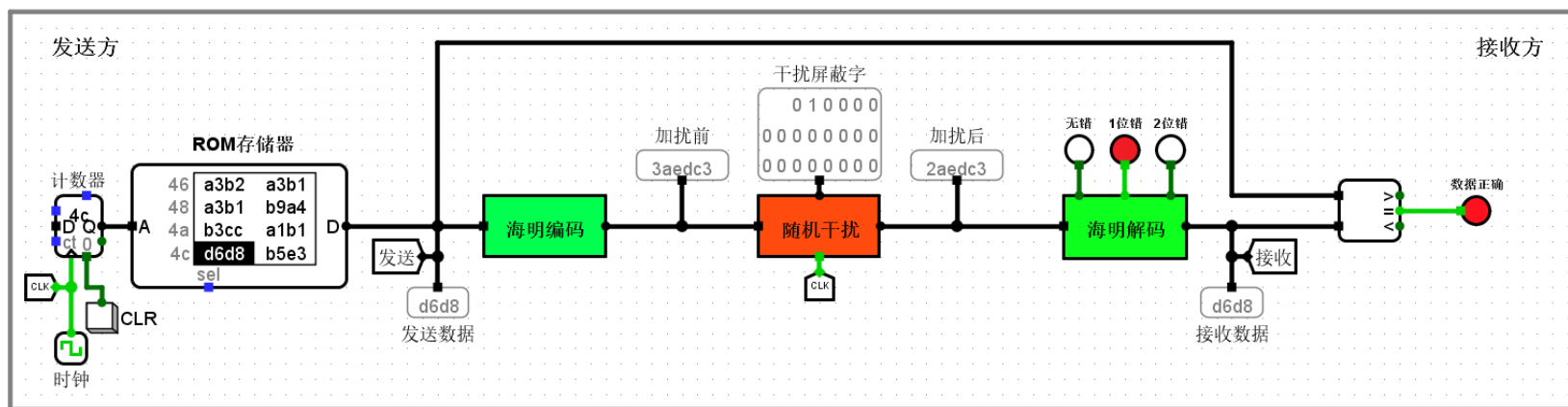


- **实验步骤:** 先按“CLR”按钮，使计数器清零（从头显示汉字）；然后每按2次“时钟”按钮，输出1个汉字的国标码；有4种情况：

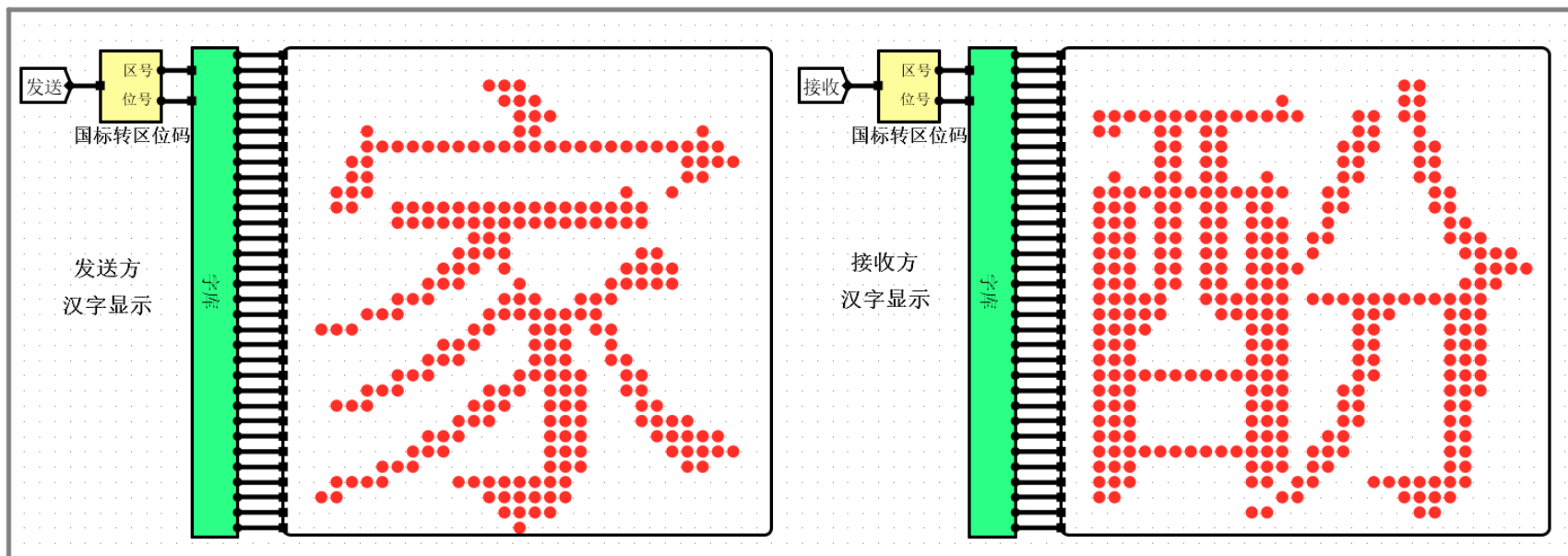
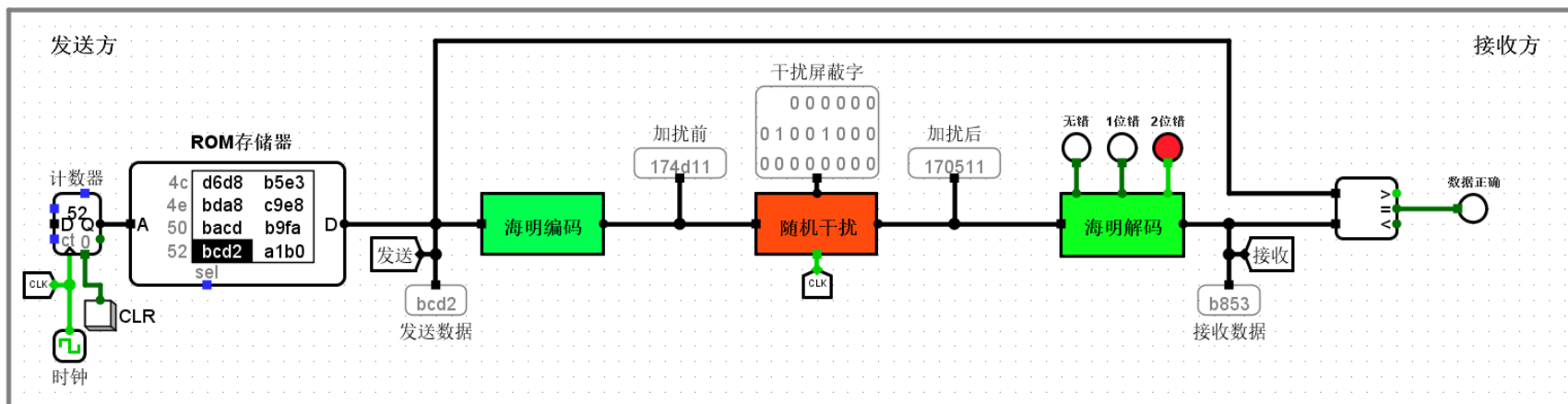
- 第1种情况：加扰屏蔽字=00 0000 0000 0000 0000 0000，**无错**=1（红灯）、数据正确=1（红灯），接收数据与发送数据相同。



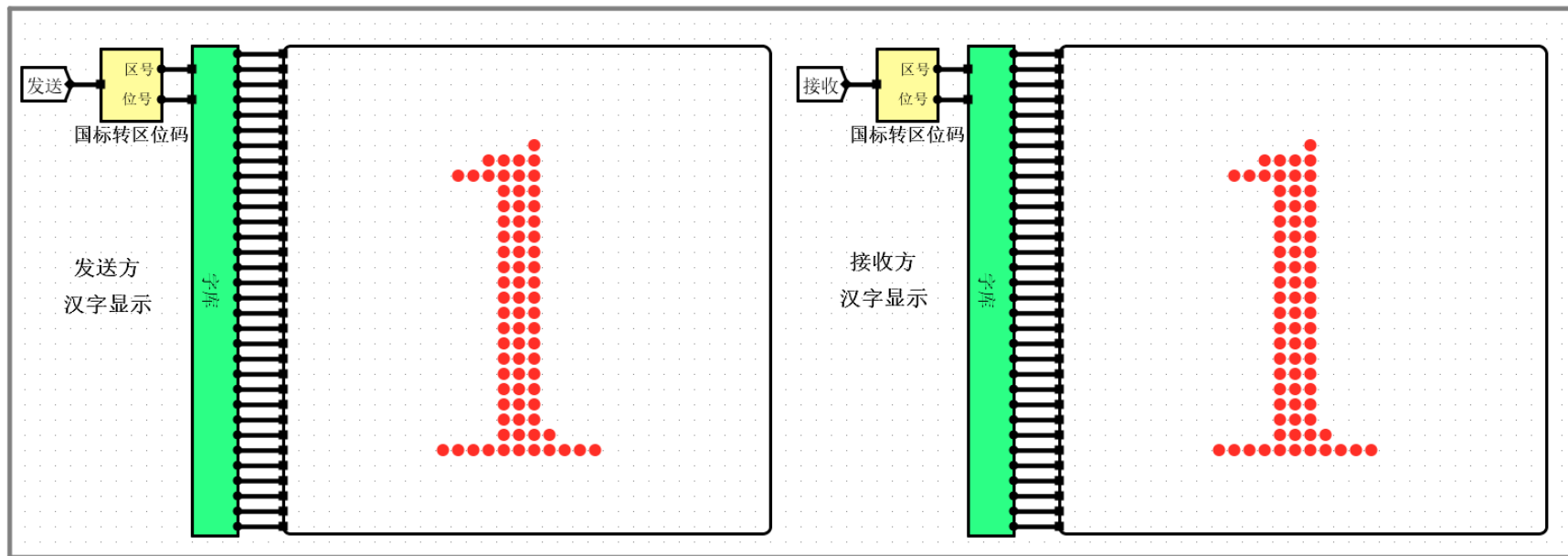
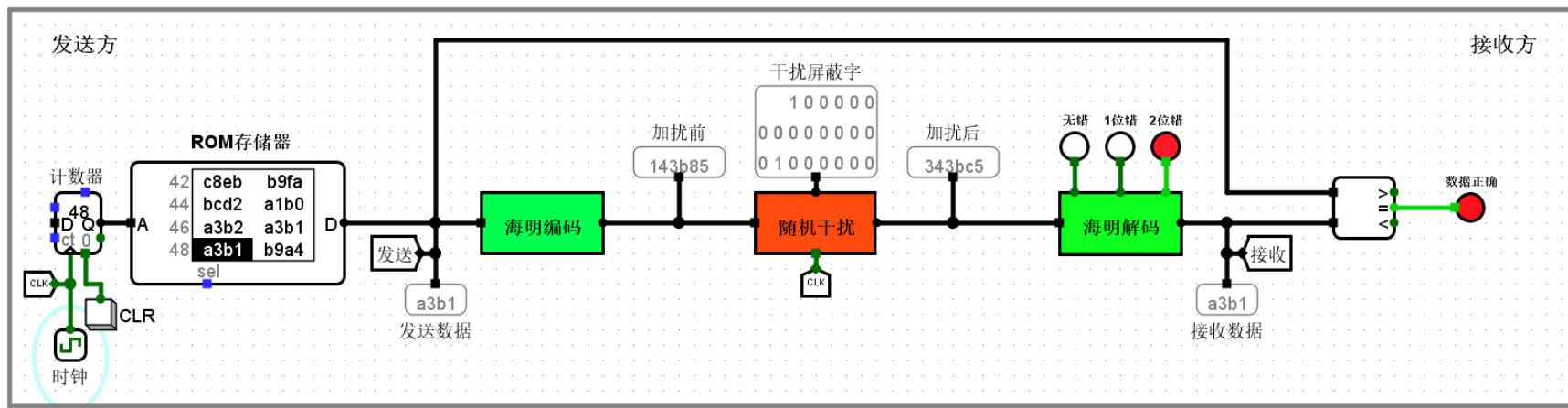
- 第2种情况：加扰屏蔽字=01 0000 0000 0000 0000 0000，**1位错**=1（红灯）、数据正确=1（红灯），接收数据与发送数据相同，表示**扩展海明码可以发现1位错、纠正1位错**。



- 第3种情况：加扰屏蔽字=00 0000 0100 1000 0000 0000，2位错=1（红灯）、数据正确=0（白灯），接收数据与发送数据不同，表示**扩展海明码可以发现2位错、但不能纠正2位错**。

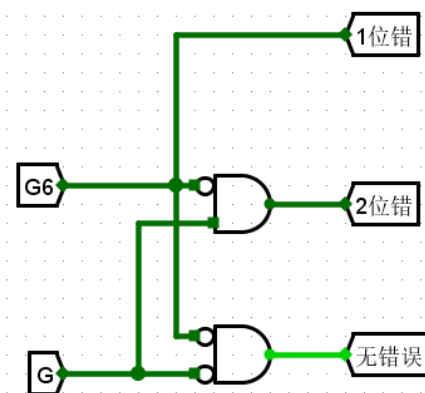
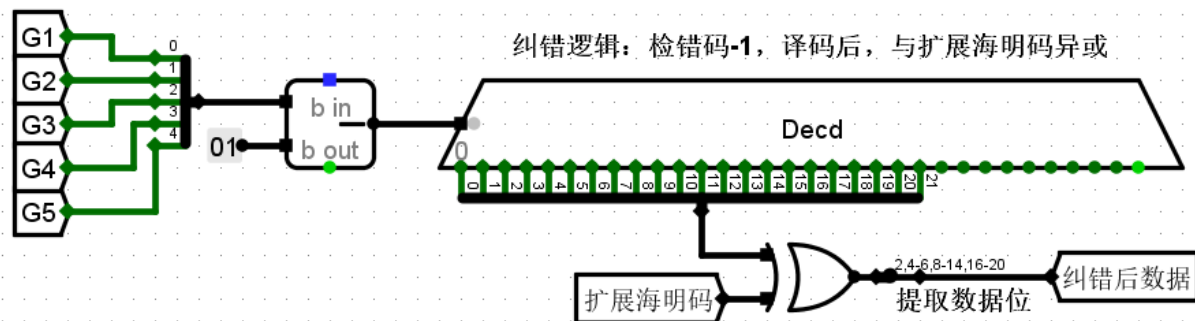


- 第4种情况（需要按多次CLK按钮才会出现这种情况）：加扰屏蔽字=10 0000 0000 0000 0100 0000，2位错=1（红灯）、数据正确=1（红灯），接收数据与发送数据相同；原因是2位错中有1位是P6，或2位错是P4和P5，因此可以纠正错误。

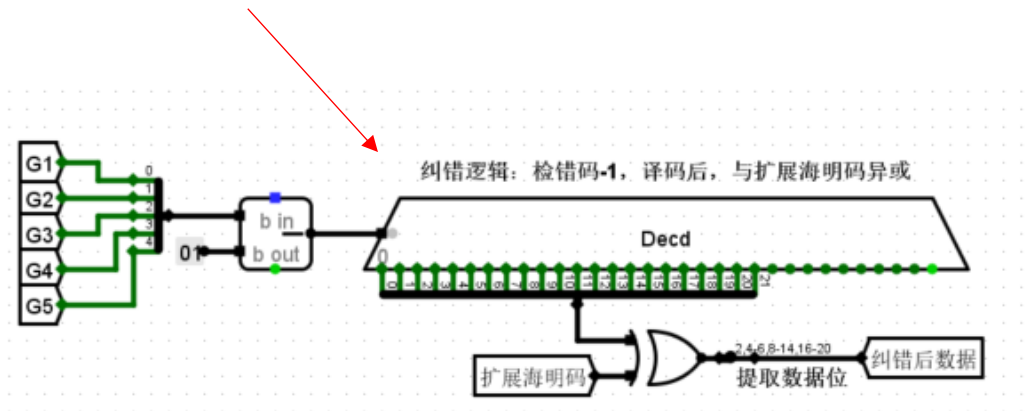


• 本实验的要点：

- 1、由16位原始数据，怎么计算**5位校验码**和**1位总偶校验位**？（扩展海明码编码电路）
- 2、由22位扩展海明码，怎么计算**5位检错码**和**1位总偶检错位**？（扩展海明码解码电路）
- 3、如何得到“1位错”、“2位错”、“无错误”标志位？（教材**图2.19**）
- 4、如何得到纠错后的数据？（**这部分电路**请同学们自行分析）
- 5、为什么会出现**第4种情况**？（有2位错，但是接收数据=发送数据）
- 6、本实验**验证了**：海明码可以纠正1位错；可以发现2位错、但不能纠正2位错；可以纠正特定情况的2位错。



- 请同学们修改“16位扩展海明码传输测试实验1”的电路，使其简化为“8位扩展海明码传输测试实验”（即原始数据=8位）。随机干扰电路已经提供给同学们（13位随机干扰电路.circ）。
- 因为原始数据为8位，因此“8位扩展海明码传输测试实验”中只包括“扩展海明码传输测试实验1”的内容，不包括“扩展海明码传输测试实验2”的内容。
- 修改后的电路命名为“8位扩展海明码传输测试实验.circ”。
- 思路：
 - (1) 根据8位原始数据，按照教材表2.22计算4位校验位（P1、P2、P3、P4），然后计算1位总偶校验位（P5），得到扩展海明码（8位+5位）。
 - (2) 根据扩展海明码，按照教材表2.22计算4位检错位（G1、G2、G3、G4），然后计算1位总偶检错位（G5）。
 - (3) 参考16位扩展海明码解码电路，设计8位扩展海明码获取纠错后数据的电路（这部分是难点）。

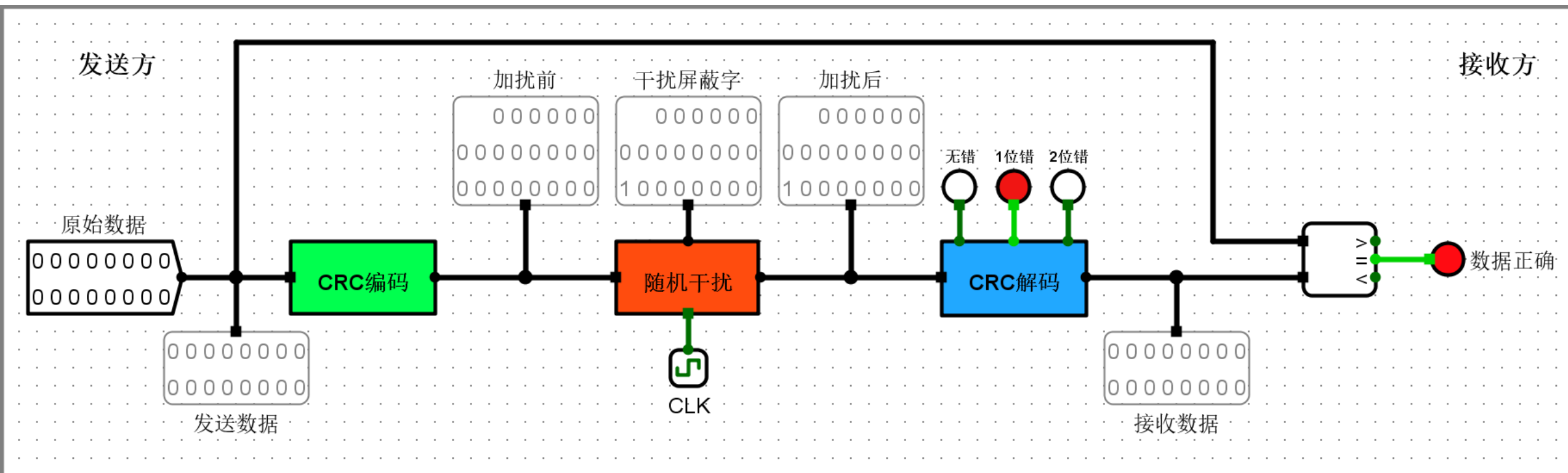


对应教材2.4.4小节“循环冗余校验”

(三) 循环冗余校验码（并行）实验

参考设计文件：循环冗余校验码（并行）实验.circ

- 运行“**Logisim**”，在Logisim中打开“**循环冗余校验码（并行）实验.circ**”。
- 也可以直接双击“**循环冗余校验码（并行）实验.circ**”，打开该设计文件。

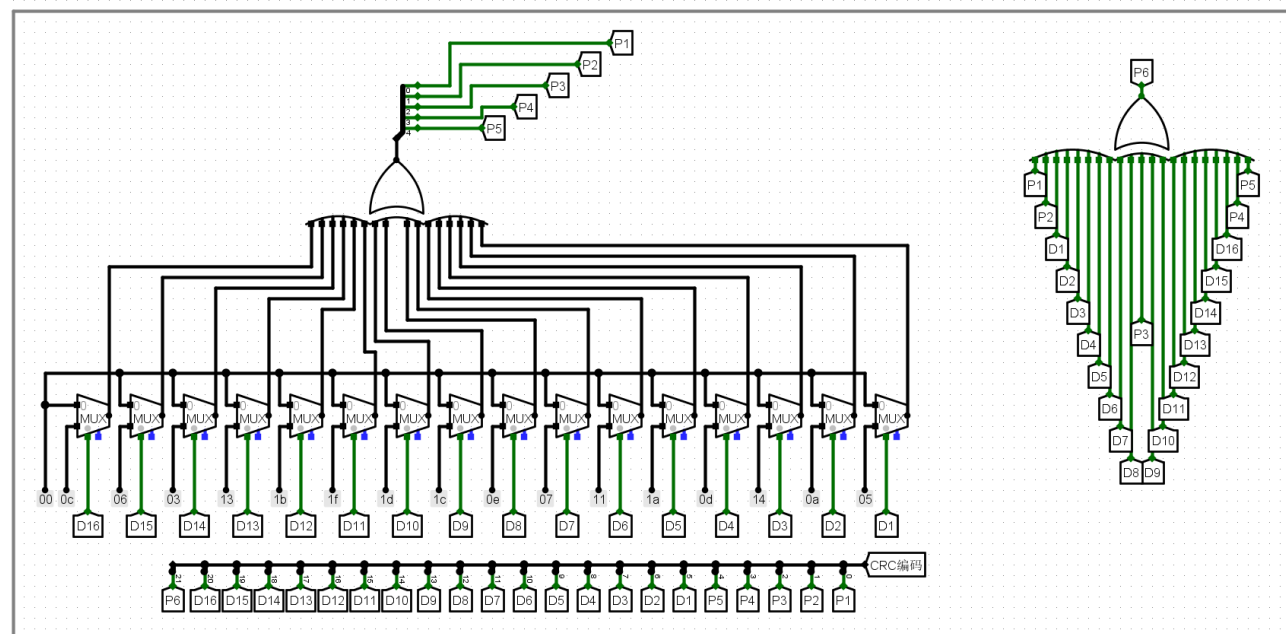


1、CRC并行编码电路

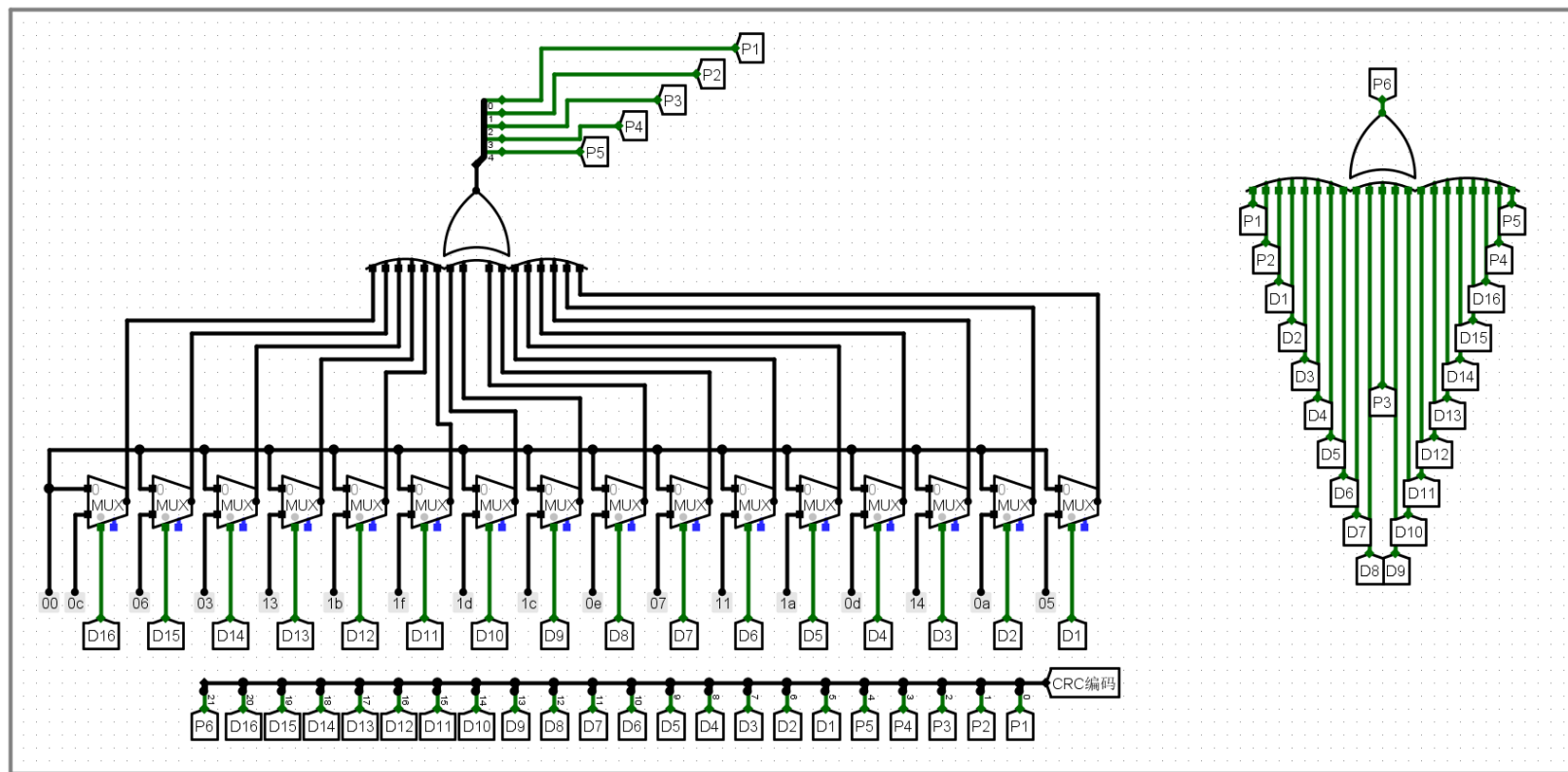
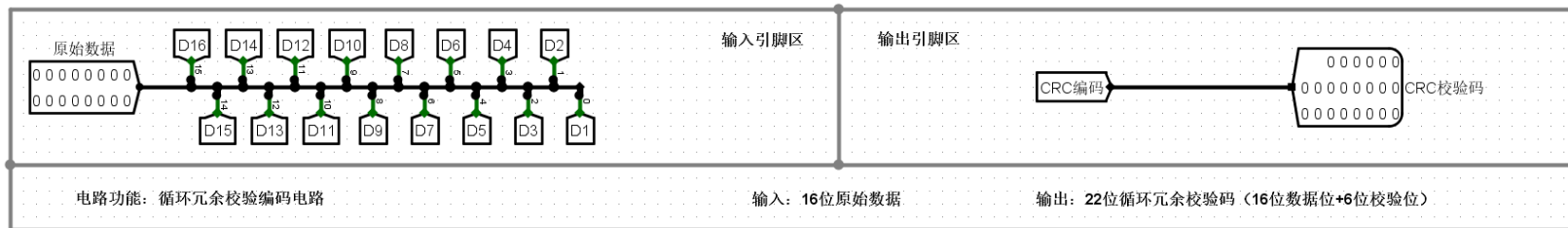
- 输入：16位原始数据
- 输出：22位CRC码
- 22位CRC码=16位原始数据+6位校验位
- 6位校验位=5位余数+1位总校验码
- $n=k+r \leq 2^r - 1$
- $k=16, r=5, n=21 \leq 2^5 - 1=31$
- $k=16, r=4, n=20 > 2^4 - 1=15$
- 生成多项式=100101



封装后的CRC并行编码电路



该电路的原理，请同学们查阅相关的文献！



16位原始数据后面补上5个0 (共21位)，
模2除以生成多项式100101，余数即为
P5P4P3P2P1

P6为16位原始数据与余数 (共21位) 的或
运算

例如：
原始数据=0000 0000 0000 1111

补上5个0后，得到：0000 0000 0000 1111
00000

除以100101，得到余数：10110

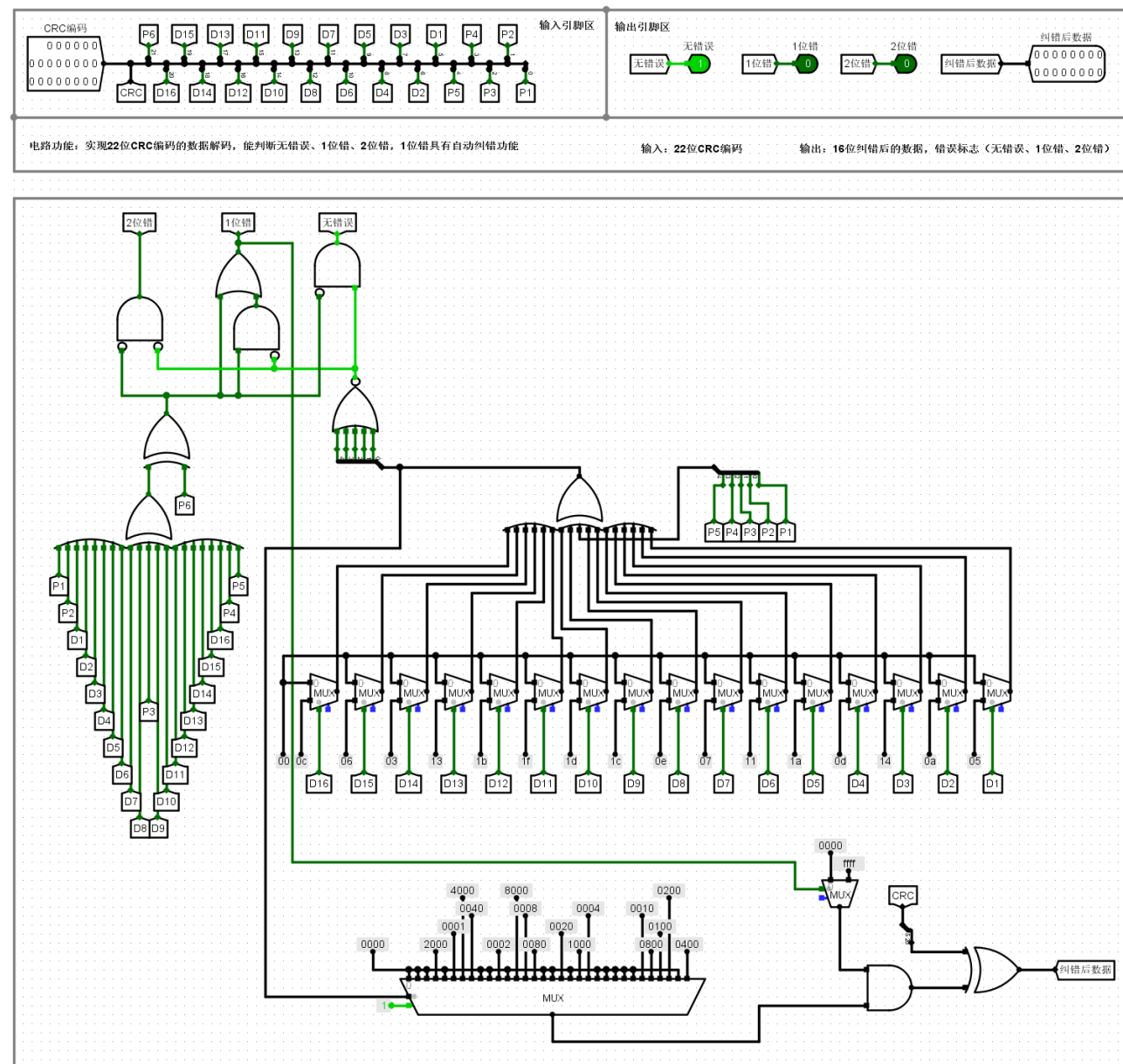
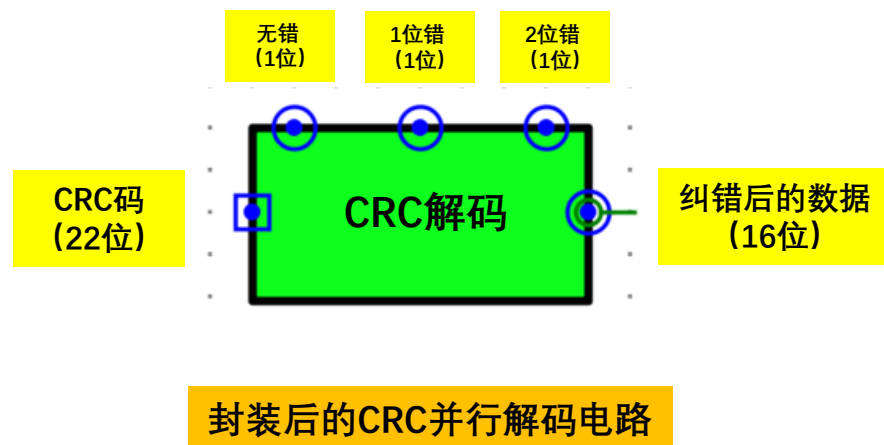
即：P5P4P3P2P1=10110

总校验码P6=0000 0000 0000 1111 10110
的或=1

CRC码=P6D16...D1P5P4P3P2P1=1 0000
0000 0000 1111 10110

• 2、CRC并行解码电路

- 输入：22位CRC码
- 输出：16位纠错后的数据，3个标志位
- 3个标志位：
 - 无错 (1位)
 - 1位错 (1位)
 - 2位错 (1位)



该电路有难度，有兴趣的同学可以查阅相关的文献！

• 3、随机干扰电路

- 输入：22位CRC码、时钟（CLK）
- 输出：
 - 22位加扰后的CRC码
 - 22位加扰屏蔽字

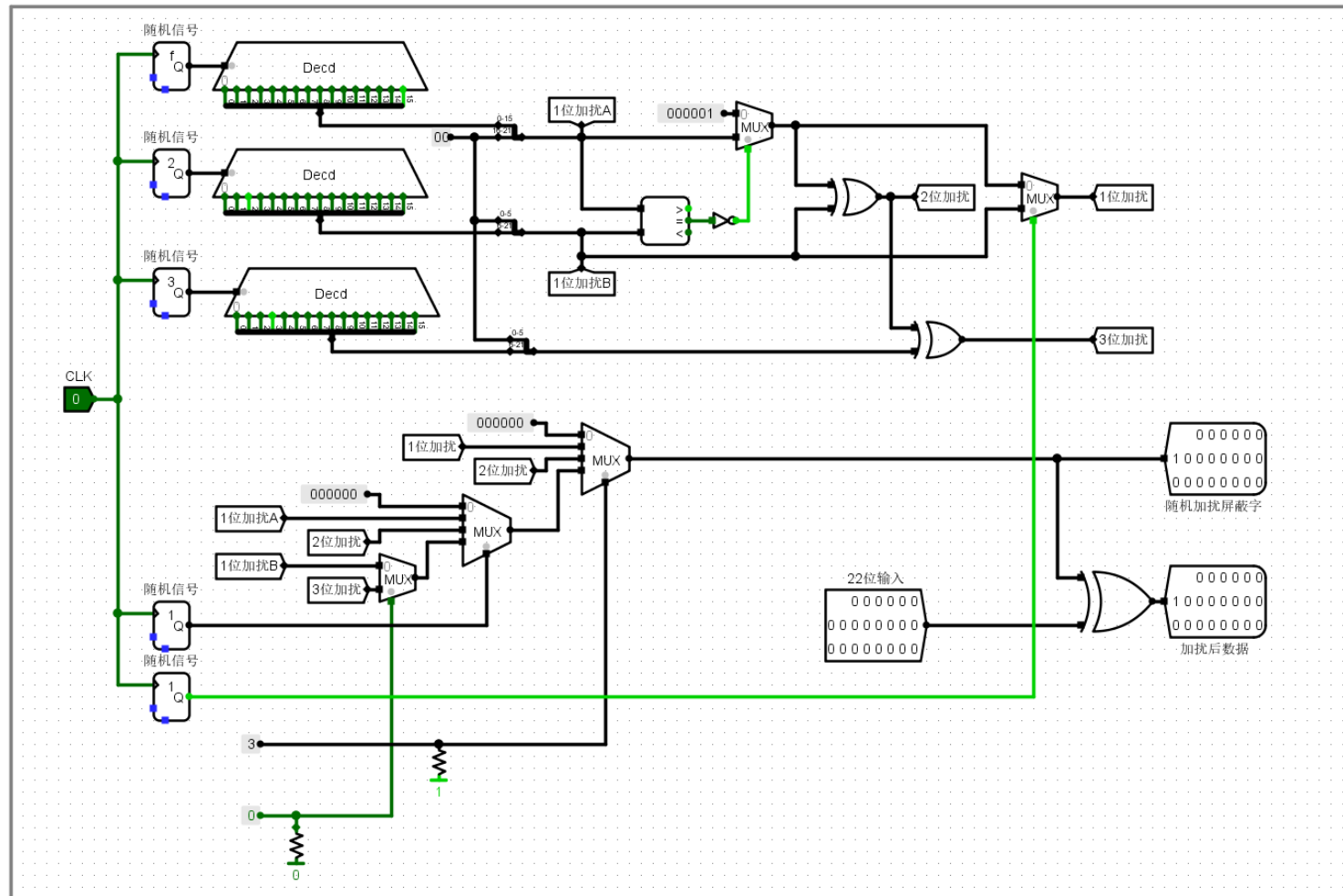
加扰屏蔽字
(22位)

CRC码
(22位)

加扰后的CRC码
(22位)

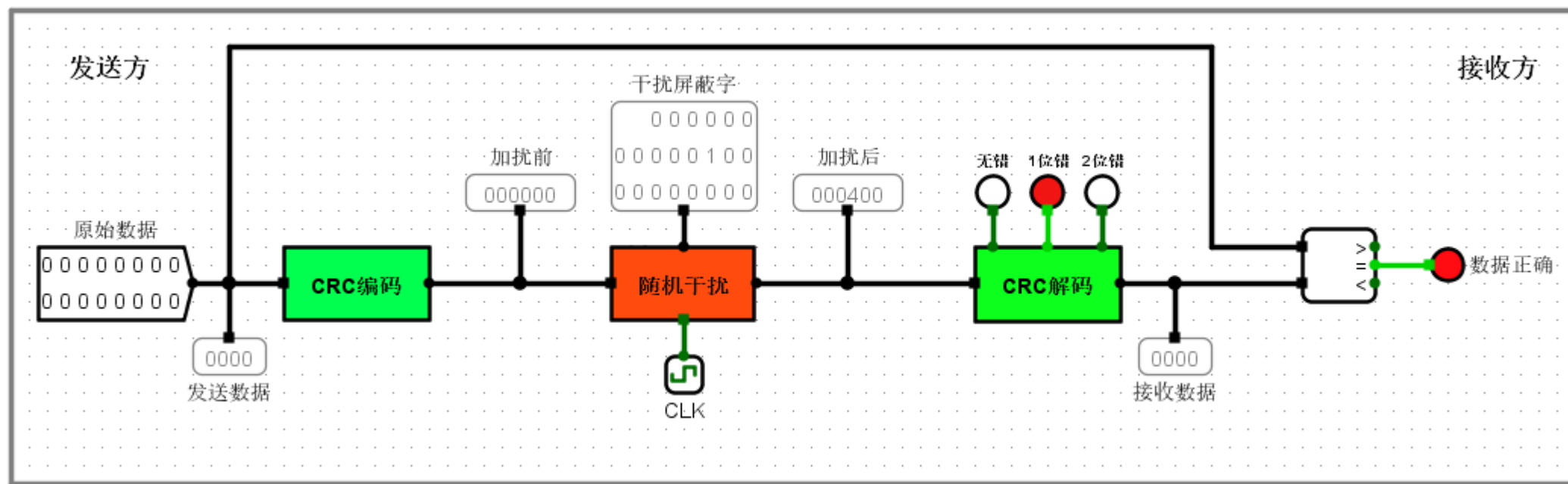
CLK

封装后的随机干扰电路



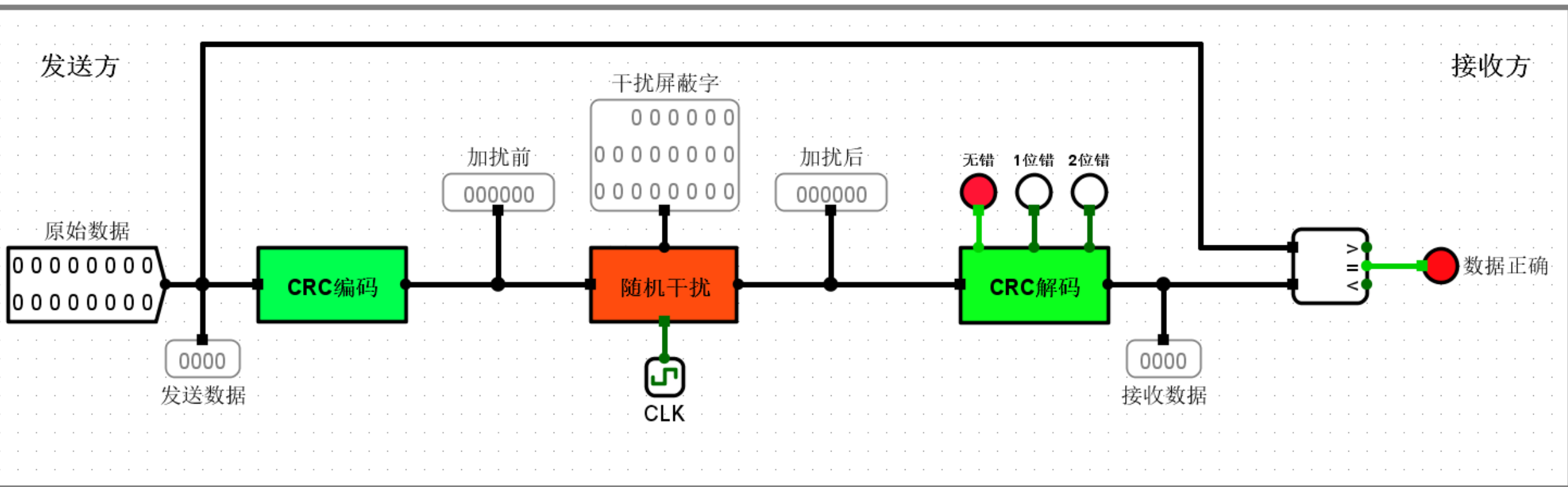
• 4、CRC码传输测试实验1

- 输入：16位发送数据（原始数据），时钟（CLK）
- 输出：16位接收数据，4个指示灯（无错、1位错、2位错、数据正确）

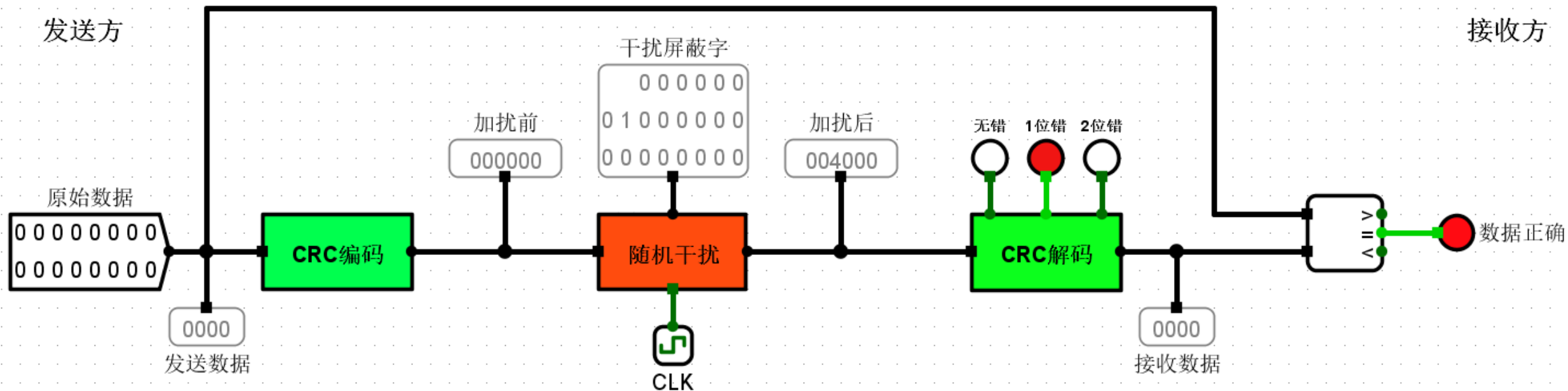


- **实验步骤:** 首先设置16位原始数据，然后每按2次“CLK”按钮，将产生不同的随机干扰，观看4个指示灯，有4种情况：

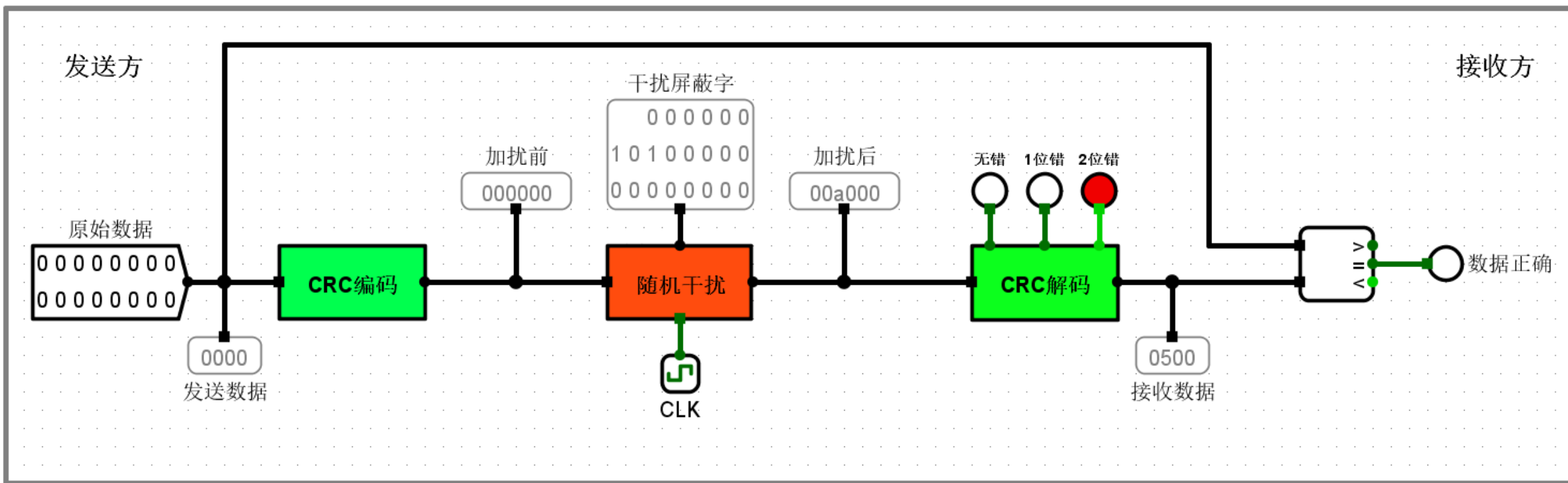
- 第1种情况：无错（接收数据=发送数据），无错指示灯（红色）、数据正确指示灯（红色）



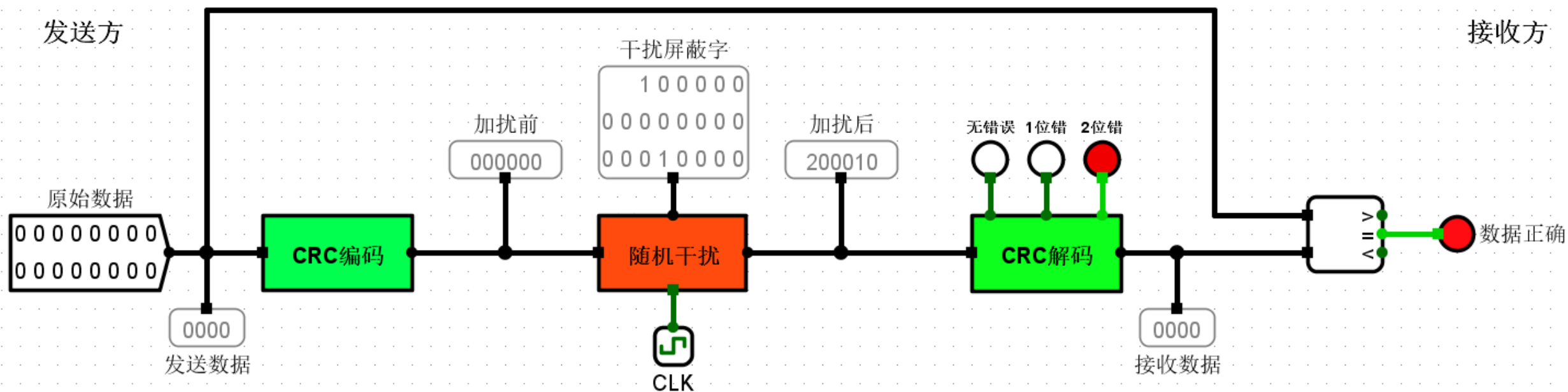
- 第2种情况：1位错（接收数据=发送数据，CRC码可以纠正1位错），1位错指示灯（红色）、数据正确指示灯（红色）



- 第3种情况：2位错（接收数据≠发送数据，CRC码可以发现2位错、但是不能纠正2位错），2位错指示灯（红色）、数据正确指示灯（白色）

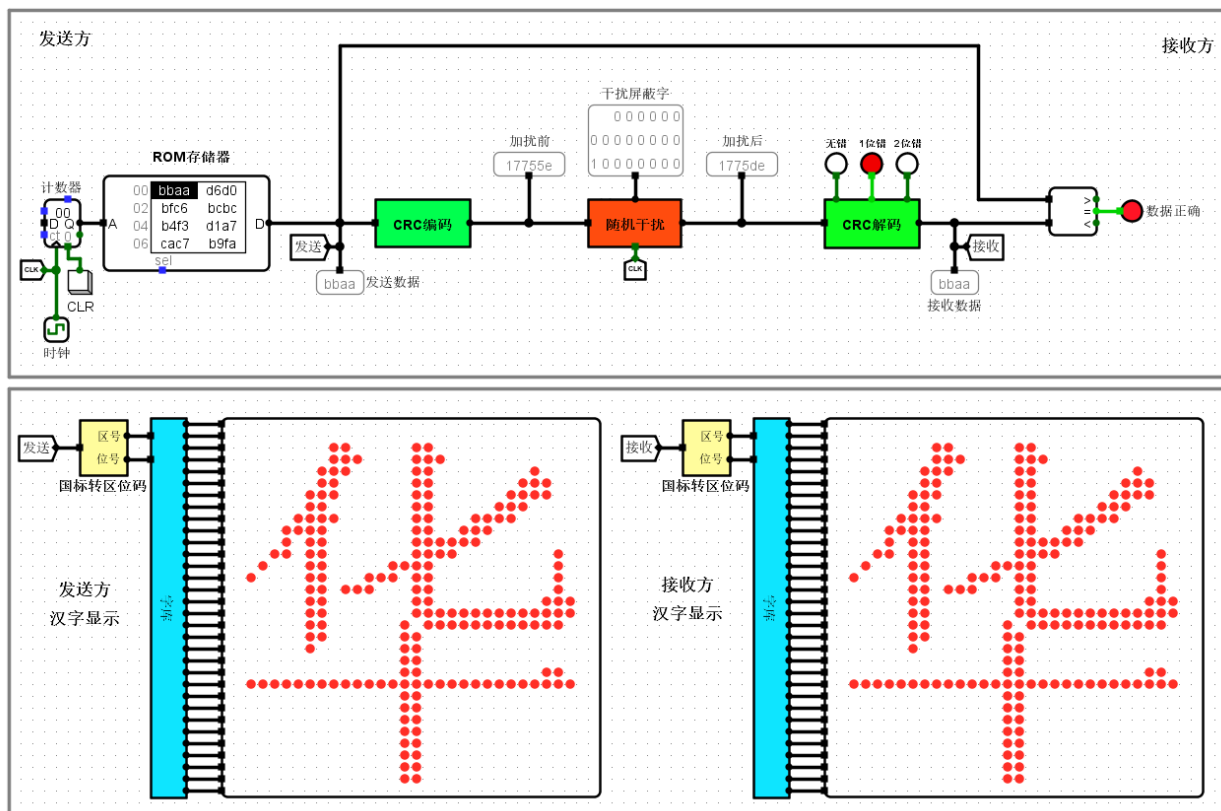


- 第4种情况（需要按多次时钟按钮才会出现这种情况）：2位错（接收数据=发送数据），2位错指示灯（红色）、数据正确指示灯（红色）；原因是2位错均发生在6位校验位上



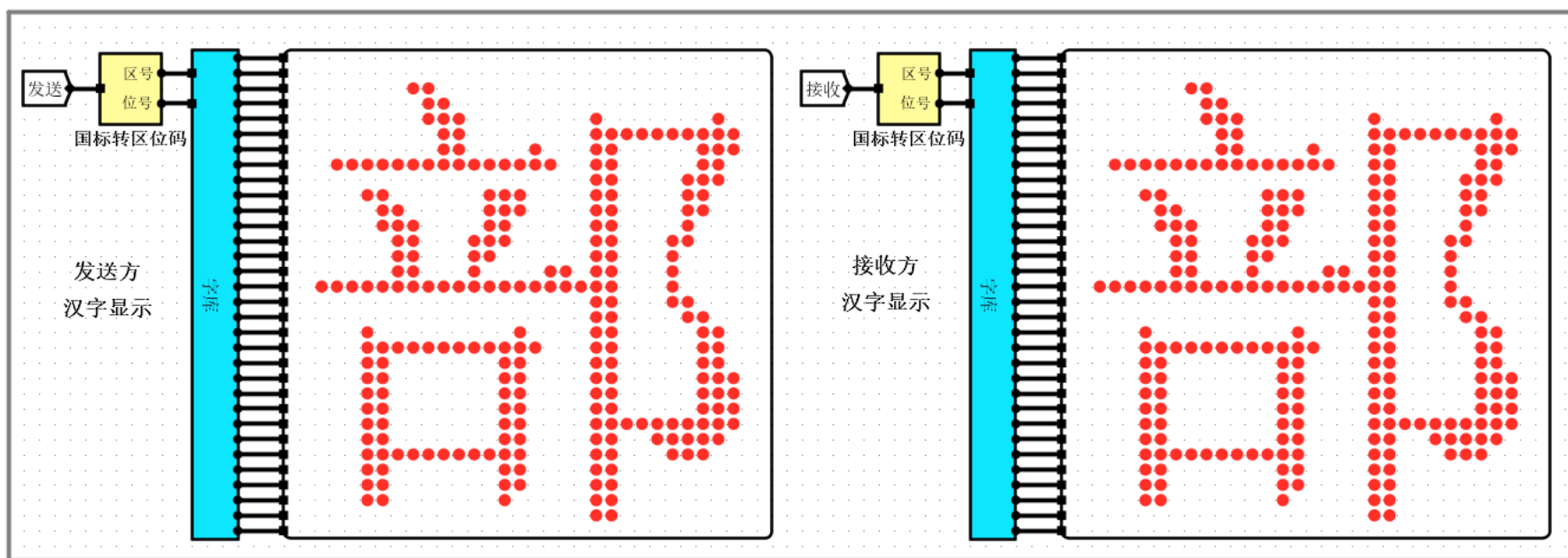
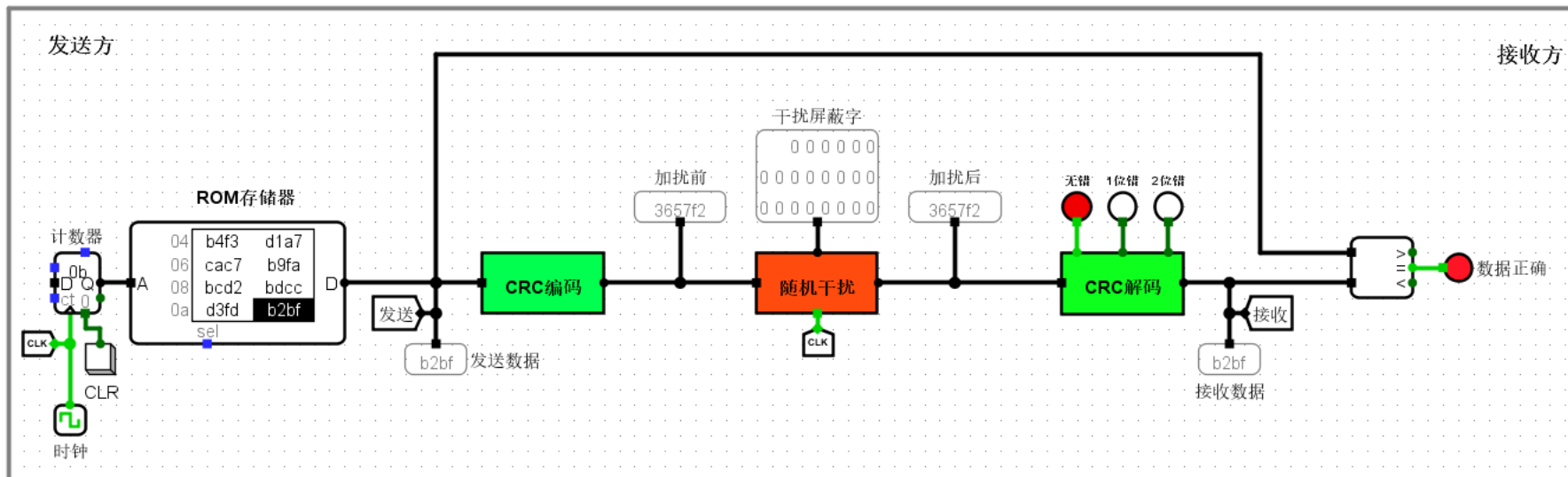
• 5、CRC码传输测试实验2

- 输入：16位汉字国标码（发送数据，在ROM存储器中），时钟CLK，CLR（计数器清零控制信号）
- 输出：16位接收数据（汉字国标码），4个指示灯（无错、1位错、2位错、数据正确），发送方与接收方显示的汉字

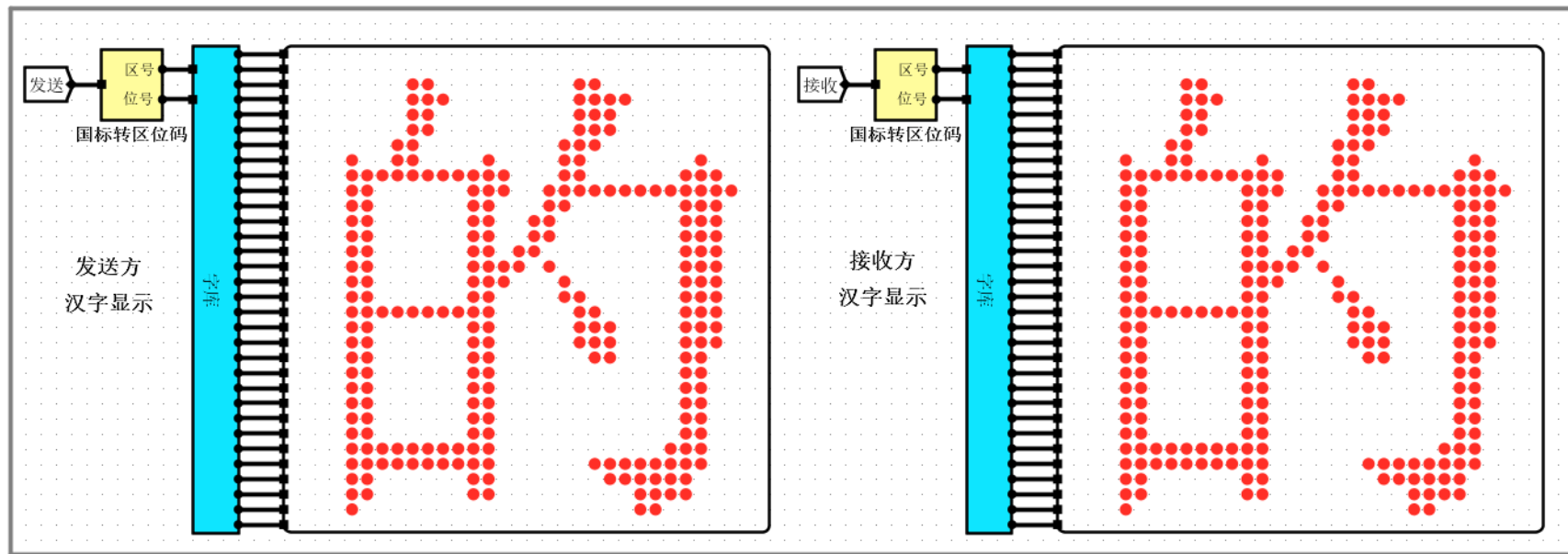
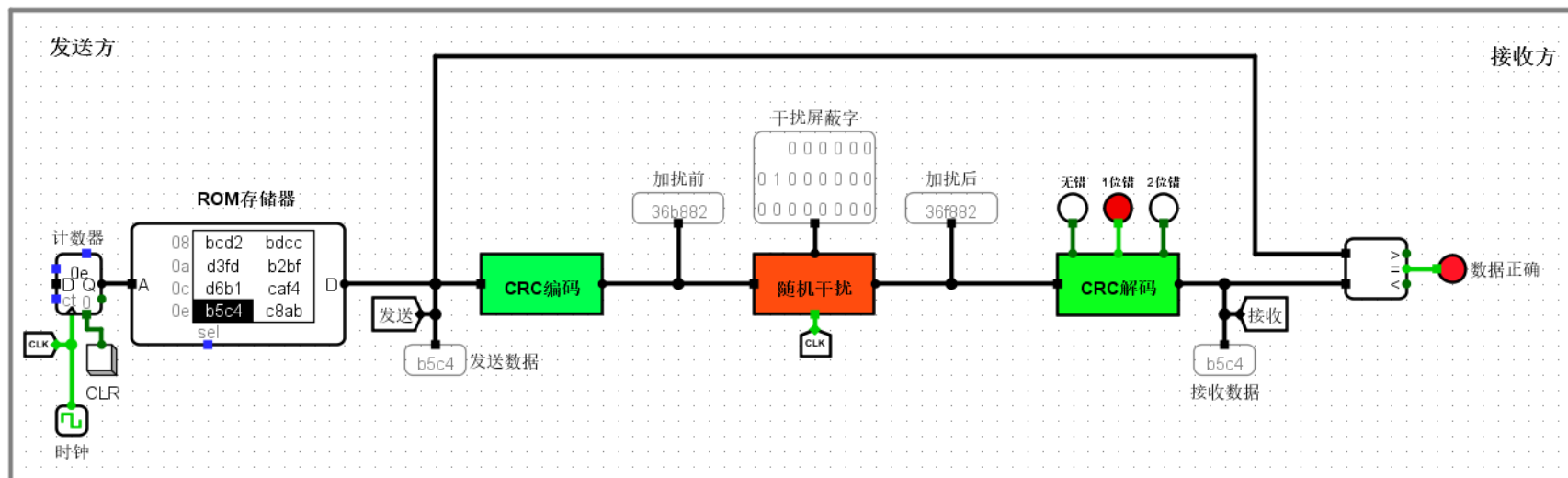


- **实验步骤:** 首先按“CLR”按钮清零计数器（从头显示汉字）；然后每按2次“时钟”按钮，输出1个汉字的国标码，并产生不同的随机干扰；观看4个指示灯，以及发送方和接收方显示的汉字，有4种情况：

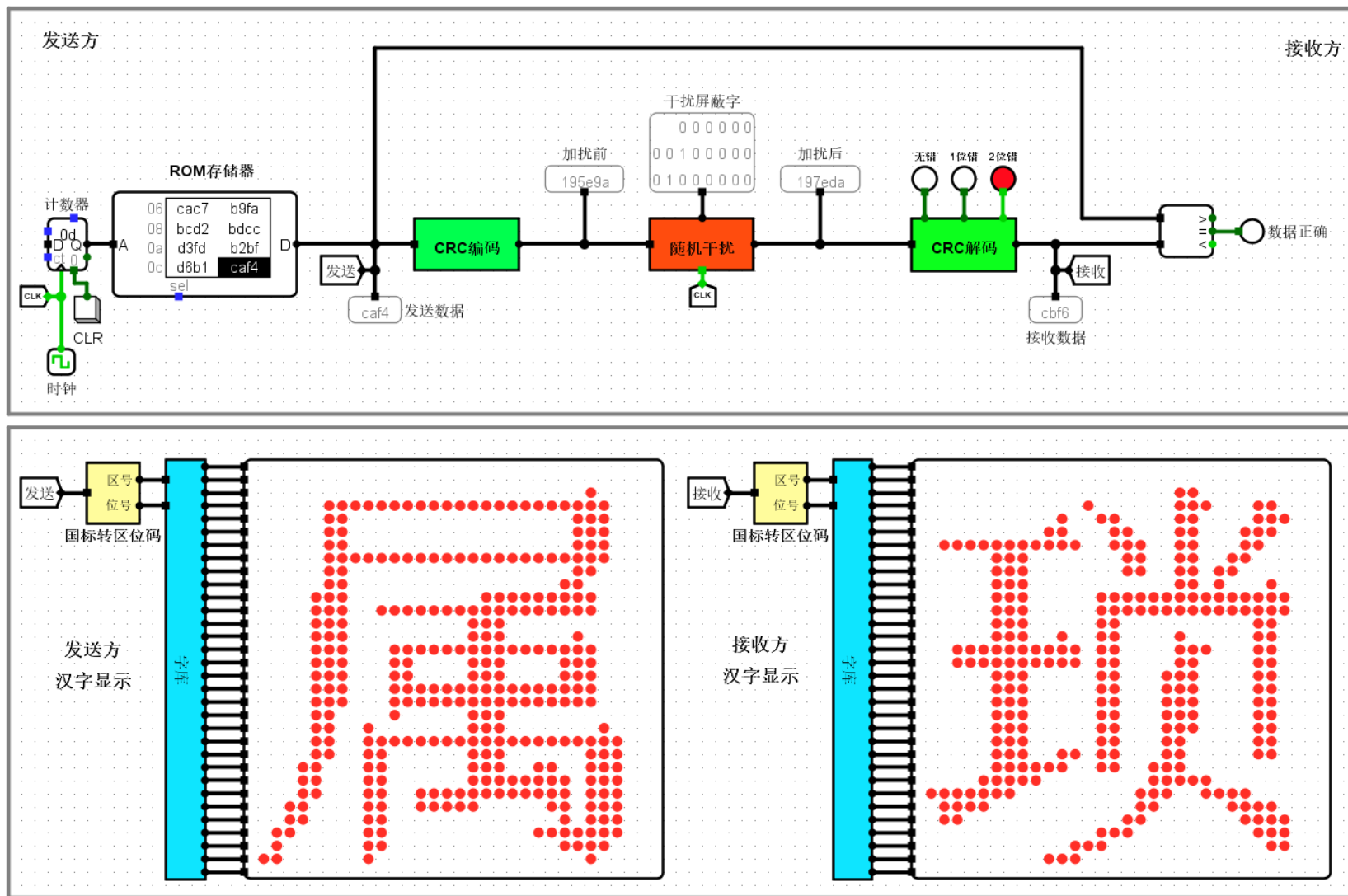
- 第1种情况：无错（接收数据=发送数据），无错指示灯（红色）、数据正确指示灯（红色）



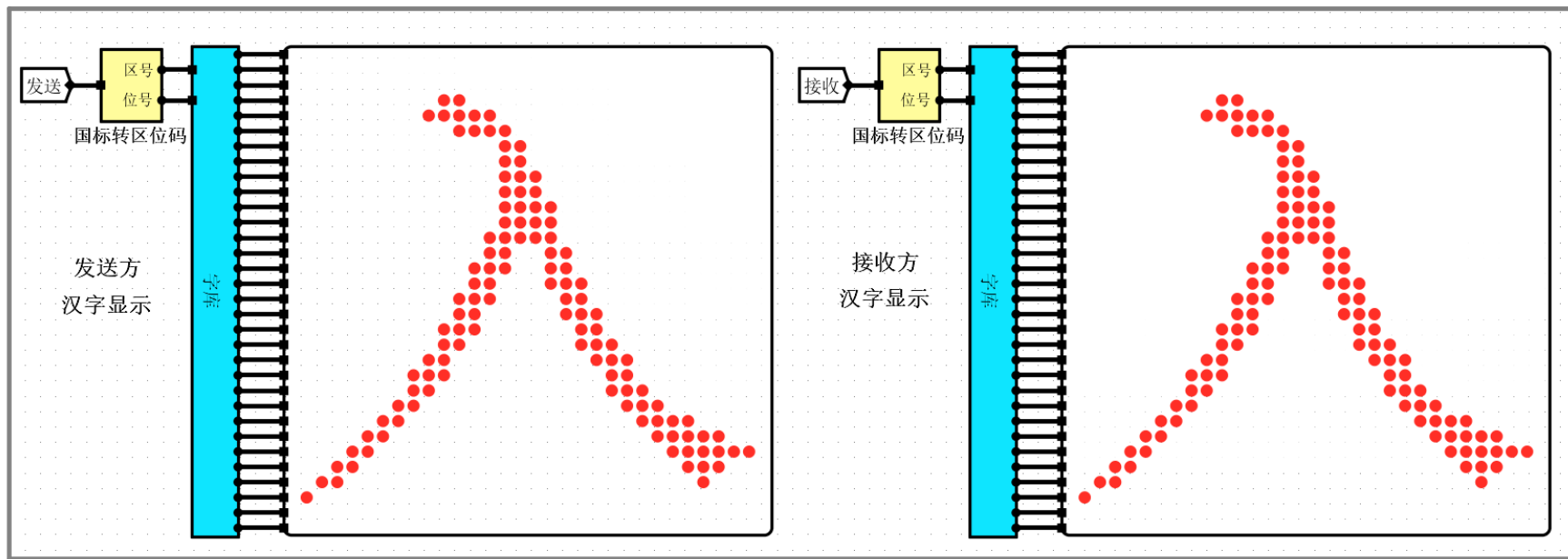
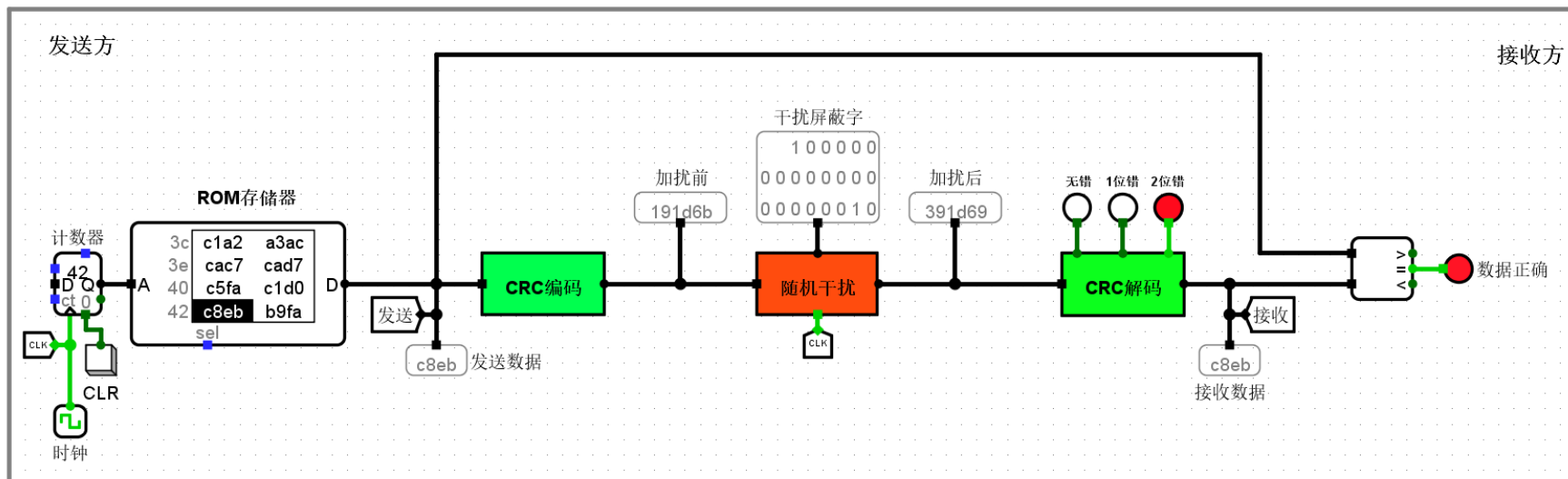
- 第2种情况：1位错（接收数据=发送数据，CRC码可以纠正1位错），1位错指示灯（红色）、数据正确指示灯（红色）



- 第3种情况：2位错（接收数据≠发送数据，CRC码可以发现2位错、但是不能纠正2位错），2位错指示灯（红色）、数据正确指示灯（白色）



- 第4种情况（需要按多次时钟按钮才会出现这种情况）：2位错（接收数据=发送数据），2位错指示灯（红色）、数据正确指示灯（红色）；原因是2位错均发生在6位校验位上



- 本实验的要点：

- 1、已知生成多项式=100101，由16位原始数据，怎么计算5位校验码（P5P4P3P2P1）和1位总校验码（P6）？（CRC并行编码电路）
 - （1）首先将原始数据后面补5个0（共21位），然后模2除以生成多项式，得到的余数即为5位校验码
 - （2）总校验码为原始数据与5位校验码的或运算
 - （3）CRC码（22位）=P6 D16……D1 P5P4P3P2P1
- 2、已知22位CRC码，怎么得到纠错后的数据（16位）？怎么得到3个标志位（无错、1位错、2位错）？（CRC并行解码电路）
 - 这部分电路比较难，有兴趣的同学可以查阅相关的文献
- 3、为什么会出现第4种情况（有2位错，但是接收数据=发送数据）？
- 4、本实验验证了：CRC码可以纠正1位错，可以发现2位错、但不能纠正2位错，可以纠正特定情况的2位错。

- 挑战性实验：

- 请修改前面的“CRC码传输测试实验1”的电路，以及“CRC并行编码电路”、“CRC并行解码电路”，使其变为“8位CRC码传输测试实验”电路，即该电路的输入（原始数据）为8位数据。随机干扰电路采用前面的“13位随机干扰电路.circ”。
- 因为原始数据为8位，因此无法实现“CRC码传输测试实验2”的电路。
- 设计文件命名为“8位循环冗余校验码（并行）实验.circ”。

对应教材2.4.4小节“循环冗余校验”

(四) 循环冗余校验码（串行）实验

参考设计文件：循环冗余校验码（串行）实验1.circ

循环冗余校验码（串行）实验2.circ

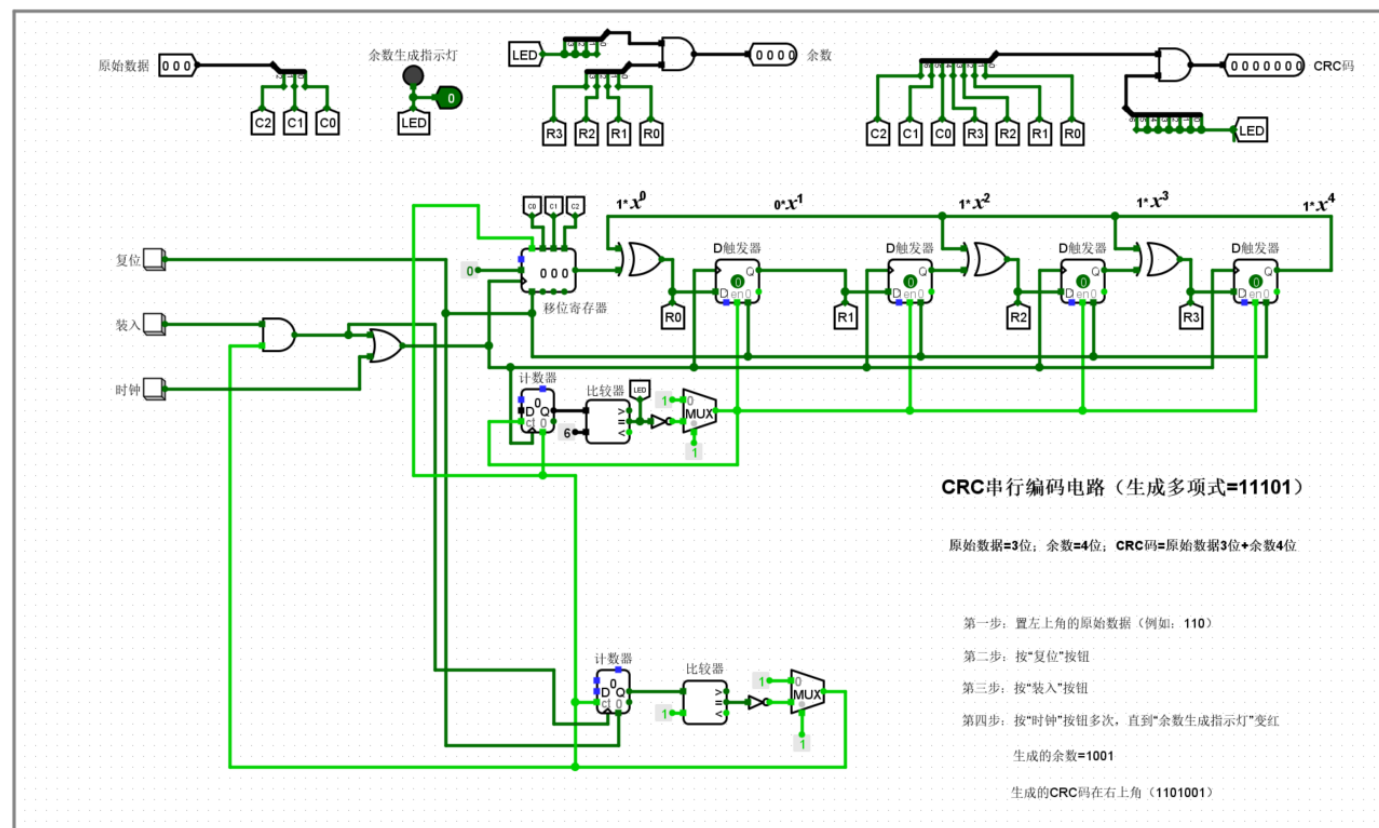
1、CRC串行编码电路

- 输入：3位原始数据
- 输出：4位余数，7位CRC码
- 7位CRC码=3位原始数据+4位余数

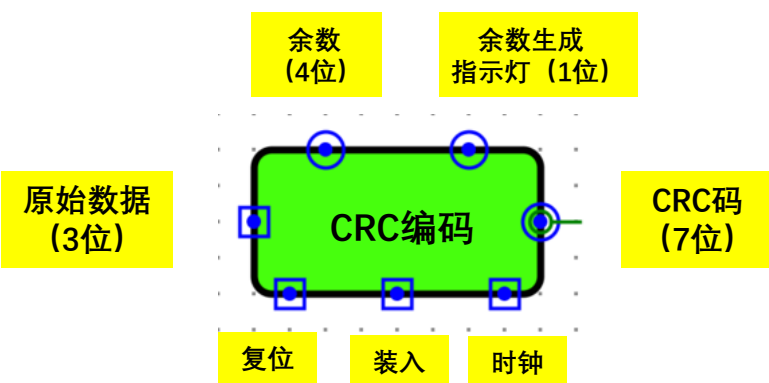
• 生成多项式=11101

• $n=k+r \leq 2^r - 1$

• $k=3, r=4, n=7 \leq 2^4 - 1 = 15$



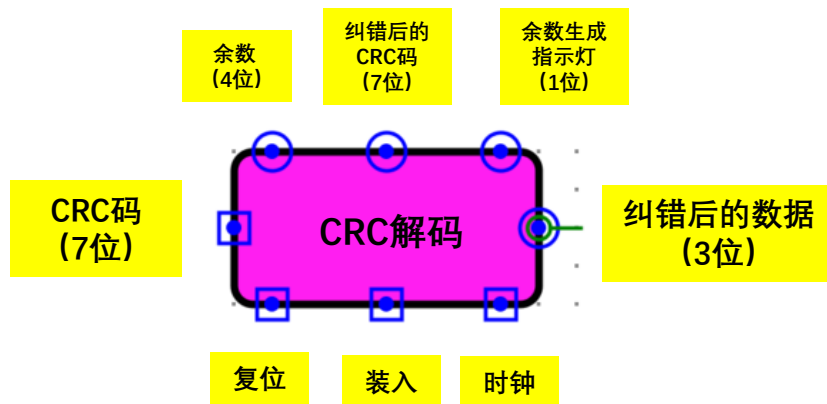
该电路请参考教材上的图2.20



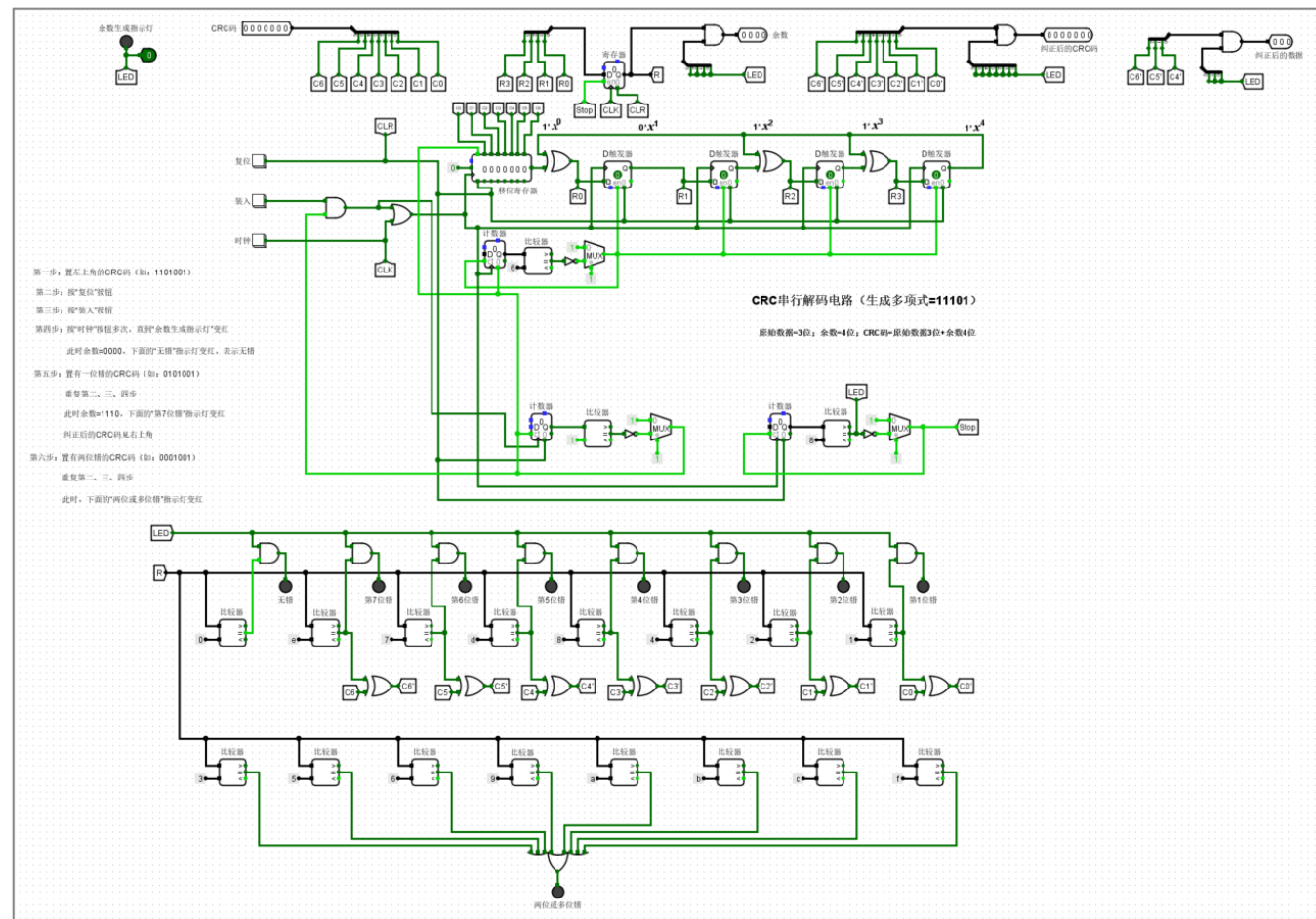
封装后的CRC串行编码电路

• 2、CRC串行解码电路

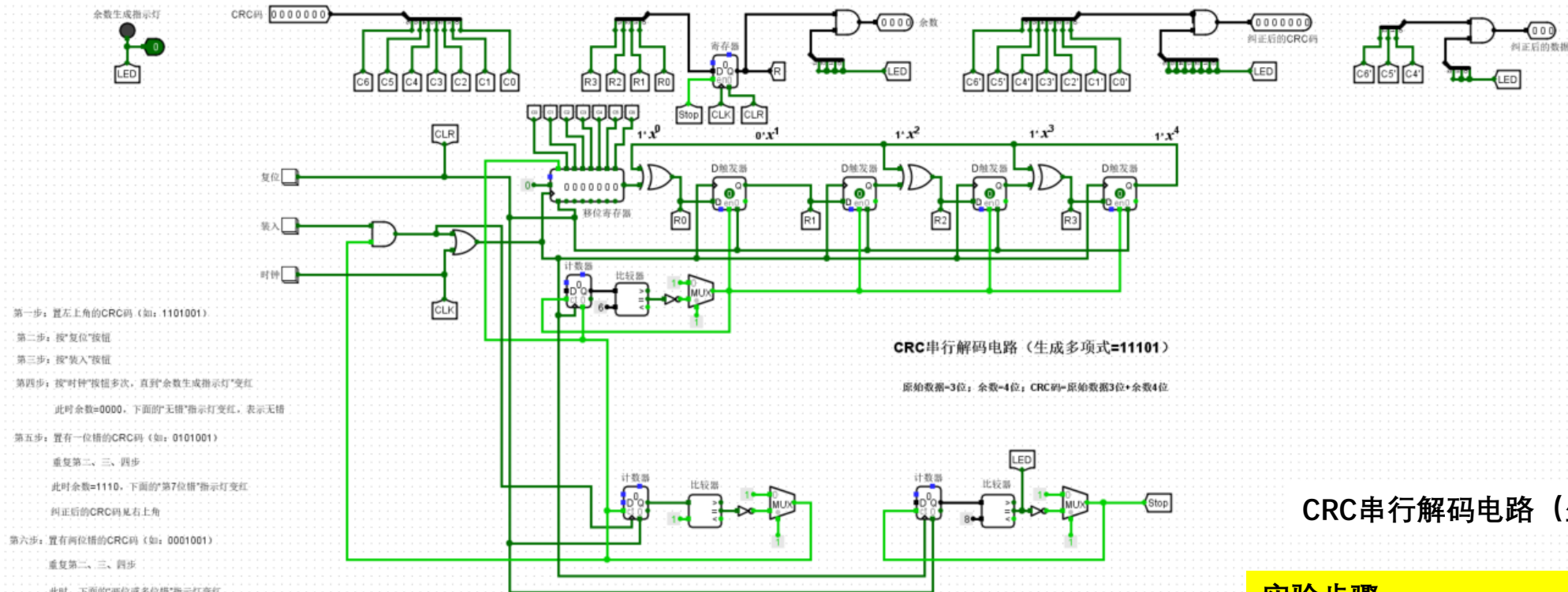
- 输入：7位CRC码
- 输出：4位余数，7位纠错后的CRC码，3位纠错后的数据



封装后的CRC串行解码电路



该电路请参考教材上的图2.20



CRC串行解码电路（生成多项式=11101）

实验步骤：

第一步：置7位CRC码（例如：110 1001）

第二步：按复位按钮

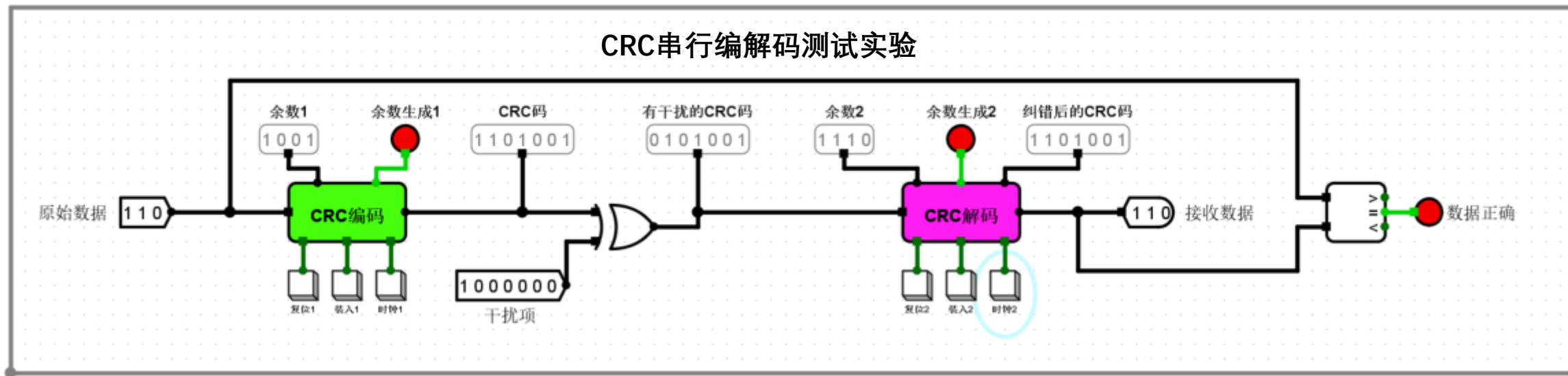
第三步：按装入按钮

第四步：按时钟按钮多次（7次），直到余数生成指示灯变红色

结果：余数=0000，纠错后的CRC码=110 1001，纠错后的数据=110，无错指示灯=红色

• 3、CRC串行编解码测试电路

- 输入：3位原始数据，7位干扰项
- 输出：3位接收数据（纠错后的数据），数据正确指示灯



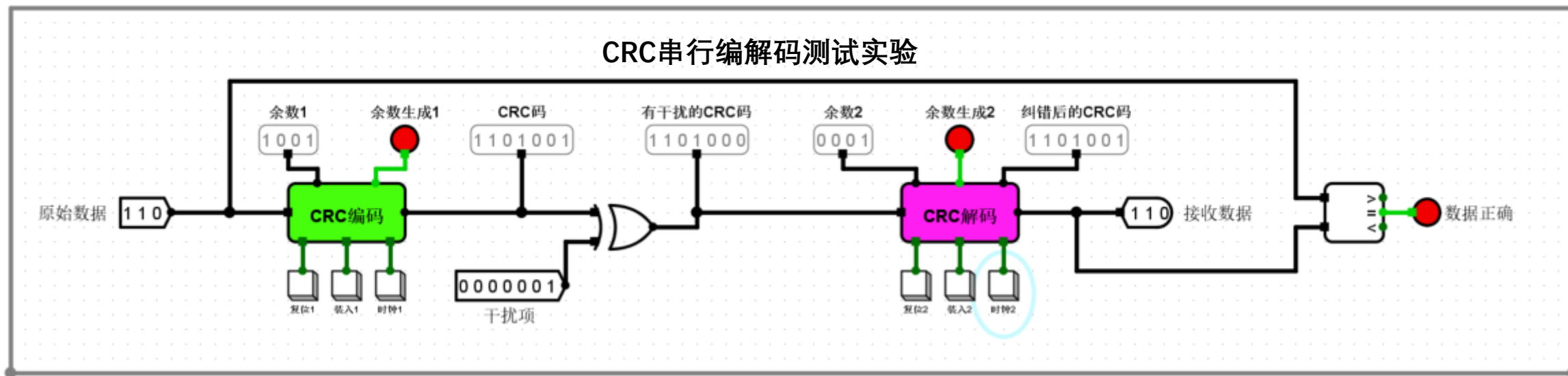
第一种情况（1位错）

第一步：设置3位原始数据（例如：110）；按“复位1”按钮、按“装入1”按钮、按“时钟1”按钮多次（6次），直到“余数生成1”指示灯变红色

此时：余数1=1001，CRC码=110 1001

第二步：设置7位干扰项（只设置1个1，例如：1000000，此时：有干扰的CRC码=010 1001）；按“复位2”按钮、按“装入2”按钮、按“时钟2”按钮多次（7次），直到“余数生成2”指示灯变红色

此时：余数2=1110，纠错后的CRC码=110 1001；数据正确指示灯=红色，接收数据=110=发送数据，表示CRC码可以纠正1位错



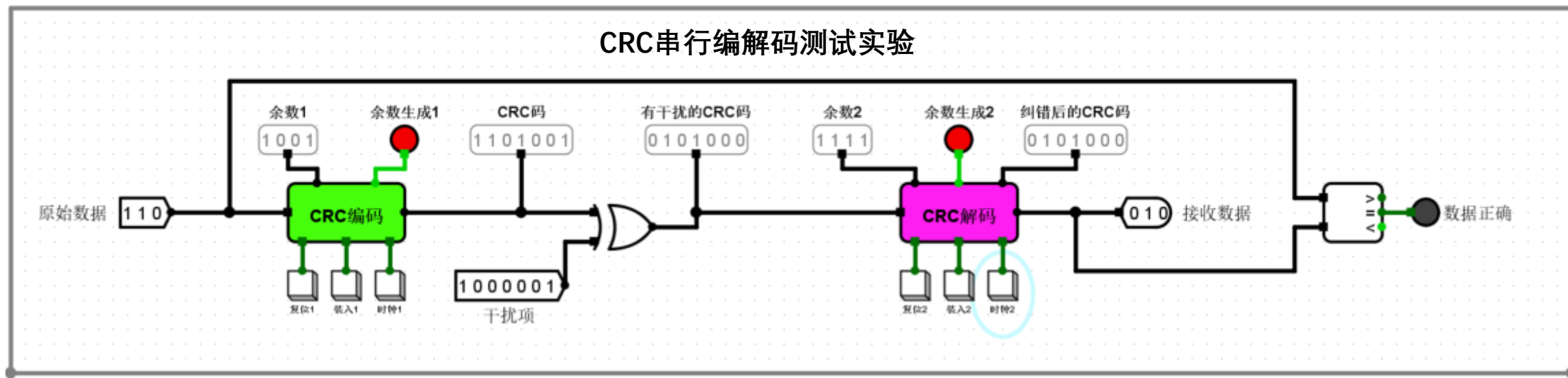
第一种情况（1位错）

重复第二步：设置新的7位干扰项（例如：0000001，此时：有干扰的CRC码=110 1000）；按“复位2”按钮、按“装入2”按钮、按“时钟2”按钮多次（7次），直到“余数生成2”指示灯变红色

此时：余数2=0001，纠错后的CRC码=110 1001；数据正确指示灯=红色，接收数据=110=发送数据，表示CRC码可以纠正1位错

请同学们设置其他5个干扰项=0100000、0010000、0001000、0000100、0000010，并进行验证（1位错）

CRC串行编解码测试实验



第二种情况（2位错误）

重复第二步：设置7位干扰项（包含2个1，例如：1000001，此时：有干扰的CRC码=010 1000）；按“复位2”按钮、按“装入2”按钮、按“时钟2”按钮多次（7次），直到“余数生成2”指示灯变红色

此时：余数2=1111，纠错后的CRC码=010 1000；数据正确指示灯=白色，接收数据=010≠发送数据，表示CRC码不能纠正2位错

请同学们设置包含2个1的不同干扰项= 0100010、0010100、……，并进行验证（2位错）

- 本实验的要点：

- 1、已知生成多项式=11101，由3位原始数据（C2C1C0），怎么计算CRC码？

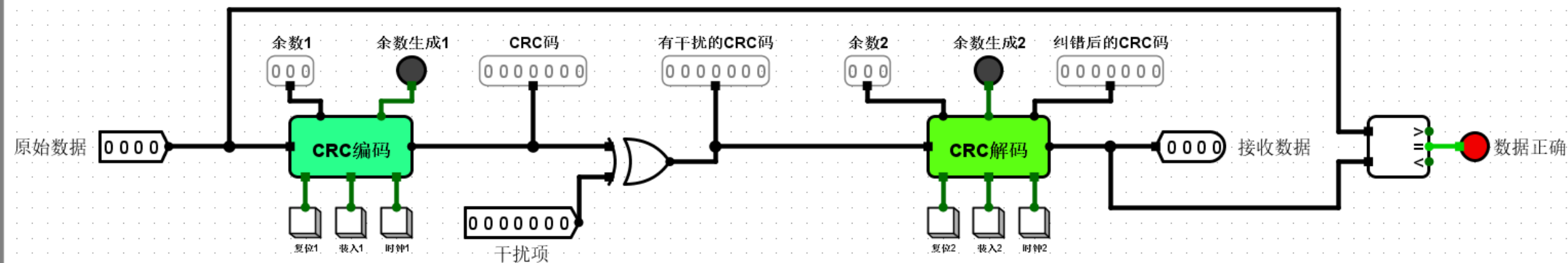
- (1) 参考教材中的图2.20（CRC串行编解码电路）
 - (2) 原始数据（C2C1C0）后面补4个0（共7位），模2除以生成多项式，得到余数（R3R2R1R0）
 - 原始数据=110，补4个0得到110 0000，模2除以生成多项式11101，得到余数=1001
 - (3) CRC码（7位）=C2C1C0 R3R2R1R0
 - CRC码=110 1001

- 2、已知生成多项式=11101，由7位CRC码，怎么得到纠错后的3位数据？

- (1) 参考教材中的图2.20（CRC串行编解码电路）
 - (2) 首先用有干扰的CRC码模2除以生成多项式，得到余数
 - CRC码=110 1001，加入1位干扰后的CRC码=110 1000，模2除以生成多项式11101，得到余数=0001
 - (3) 然后用余数与0、1、2、4、8、13（d）、7、14（e）进行比较（参考教材的表2.24），根据比较结果确定是哪一位出错，然后进行纠错
 - 余数=0001=1，查表2.24，知道是第1位出错，纠错后的CRC码=110 1001

另一个CRC串行编解码电路

- 运行“**Logisim**”，在Logisim中打开“**循环冗余校验码（串行）实验2.circ**”。
- 也可以直接双击“**循环冗余校验码（串行）实验2.circ**”，打开该设计文件。



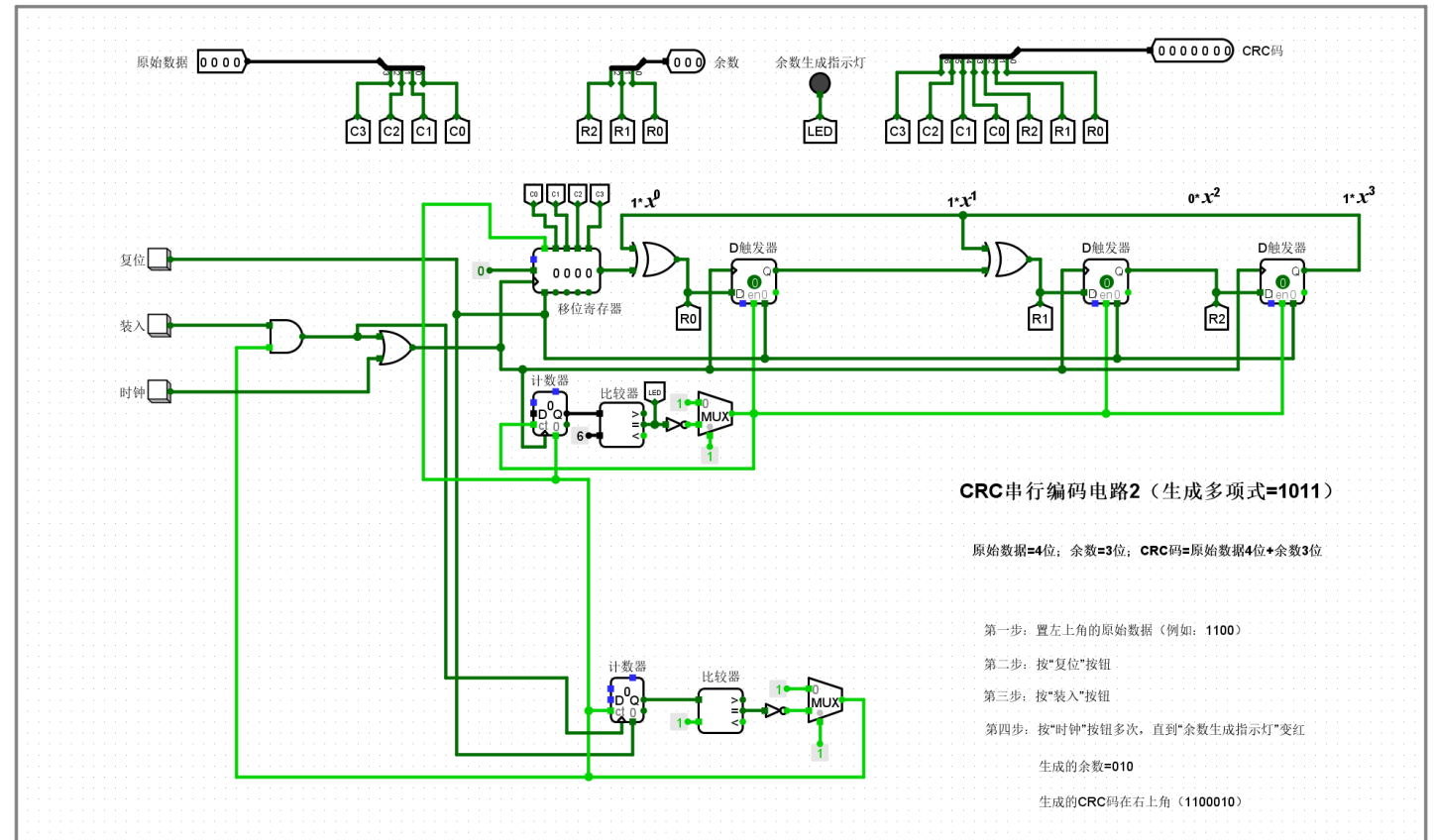
• 1、CRC串行编码电路

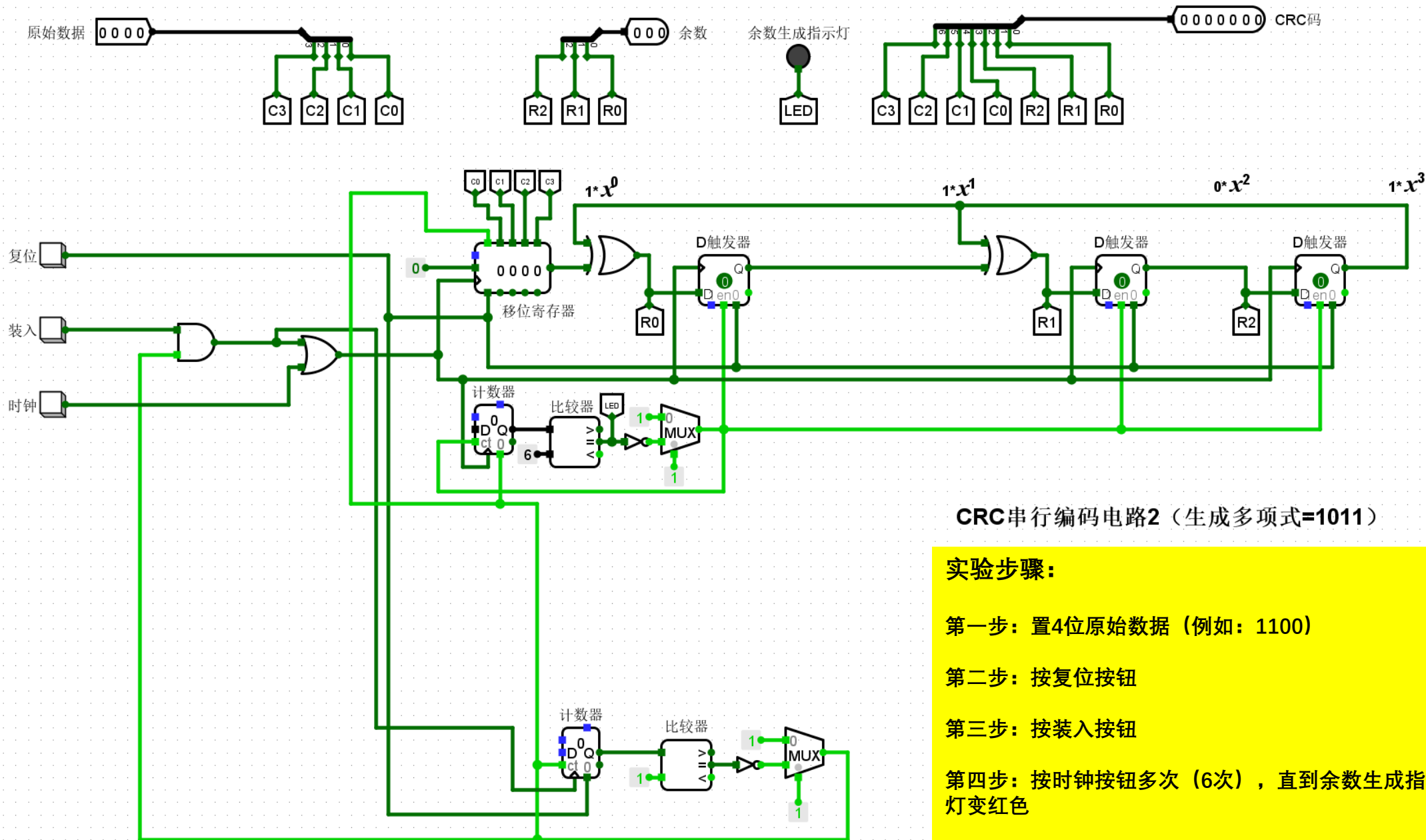
- 输入：4位原始数据
- 输出：3位余数，7位CRC码
- 7位CRC码=4位原始数据+3位余数

• 生成多项式=1011

• $n=k+r \leq 2^r-1$

• $k=4, r=3, n=7 \leq 2^3-1=7$





CRC串行编码电路2（生成多项式=1011）

实验步骤：

第一步：置4位原始数据（例如：1100）

第二步：按复位按钮

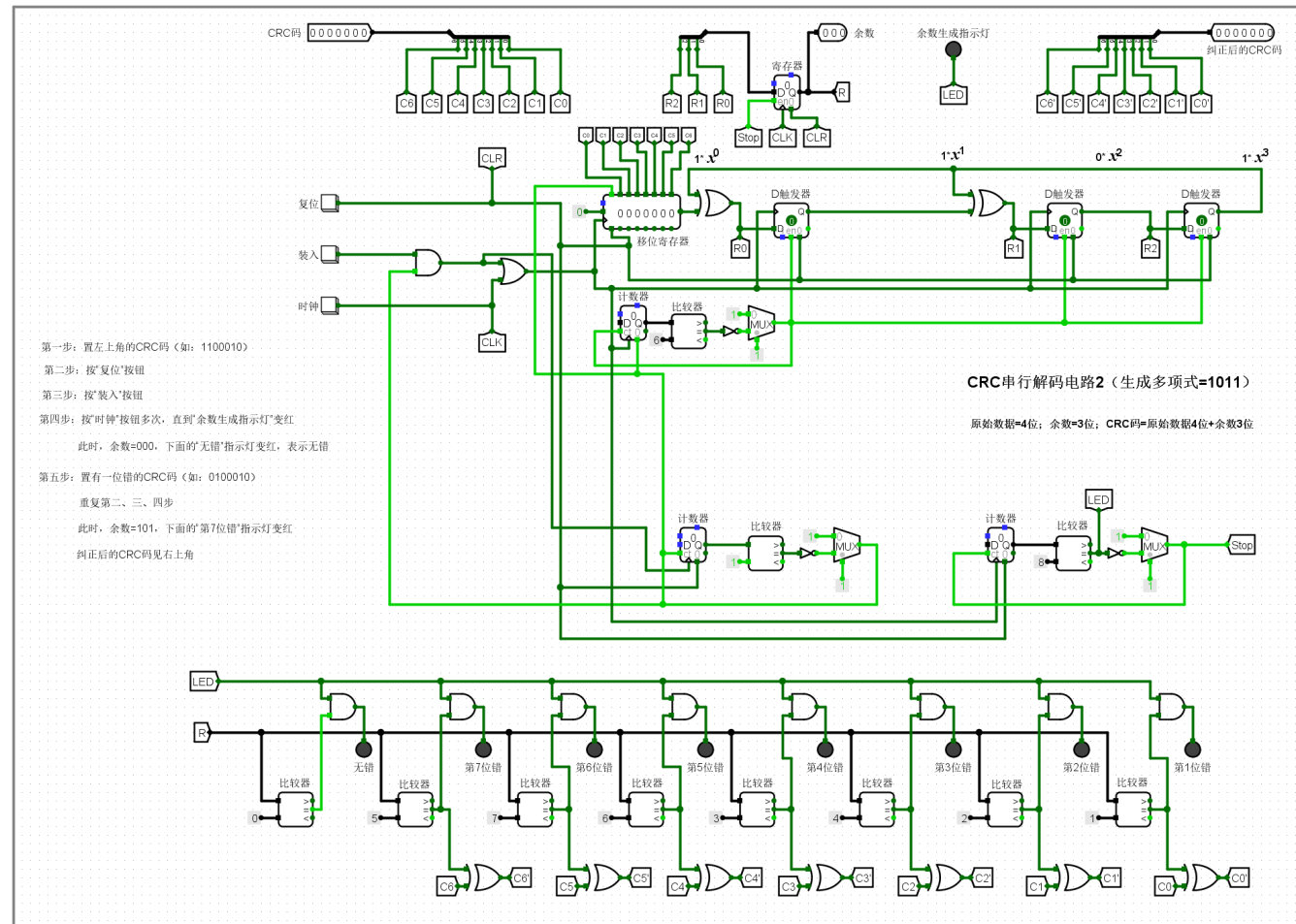
第三步：按装入按钮

第四步：按时钟按钮多次（6次），直到余数生成指示灯变红色

结果：余数=010，CRC码=1100 010

• 2、CRC串行解码电路

- 输入：7位CRC码
- 输出：3位余数，7位纠错后的CRC码，4位纠错后的数据



实验步骤:

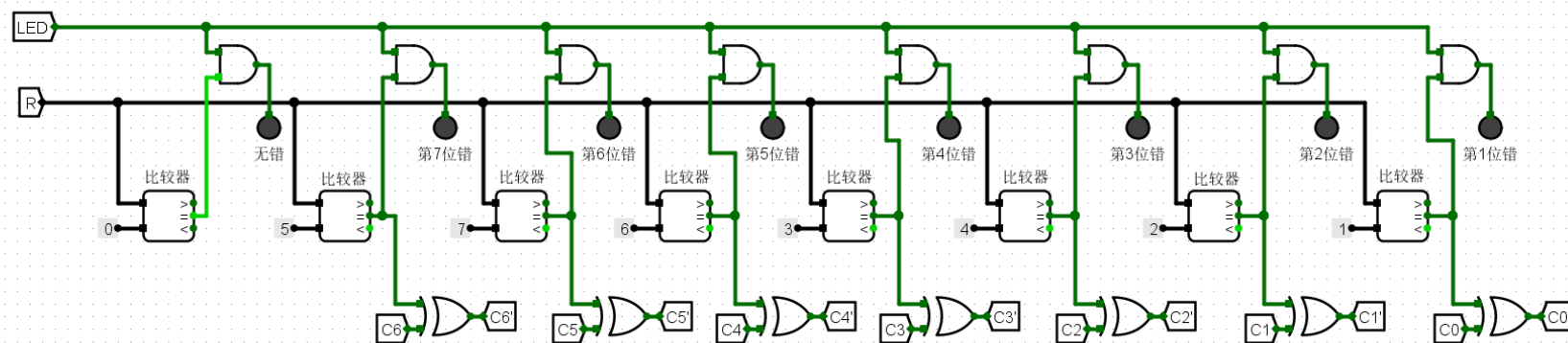
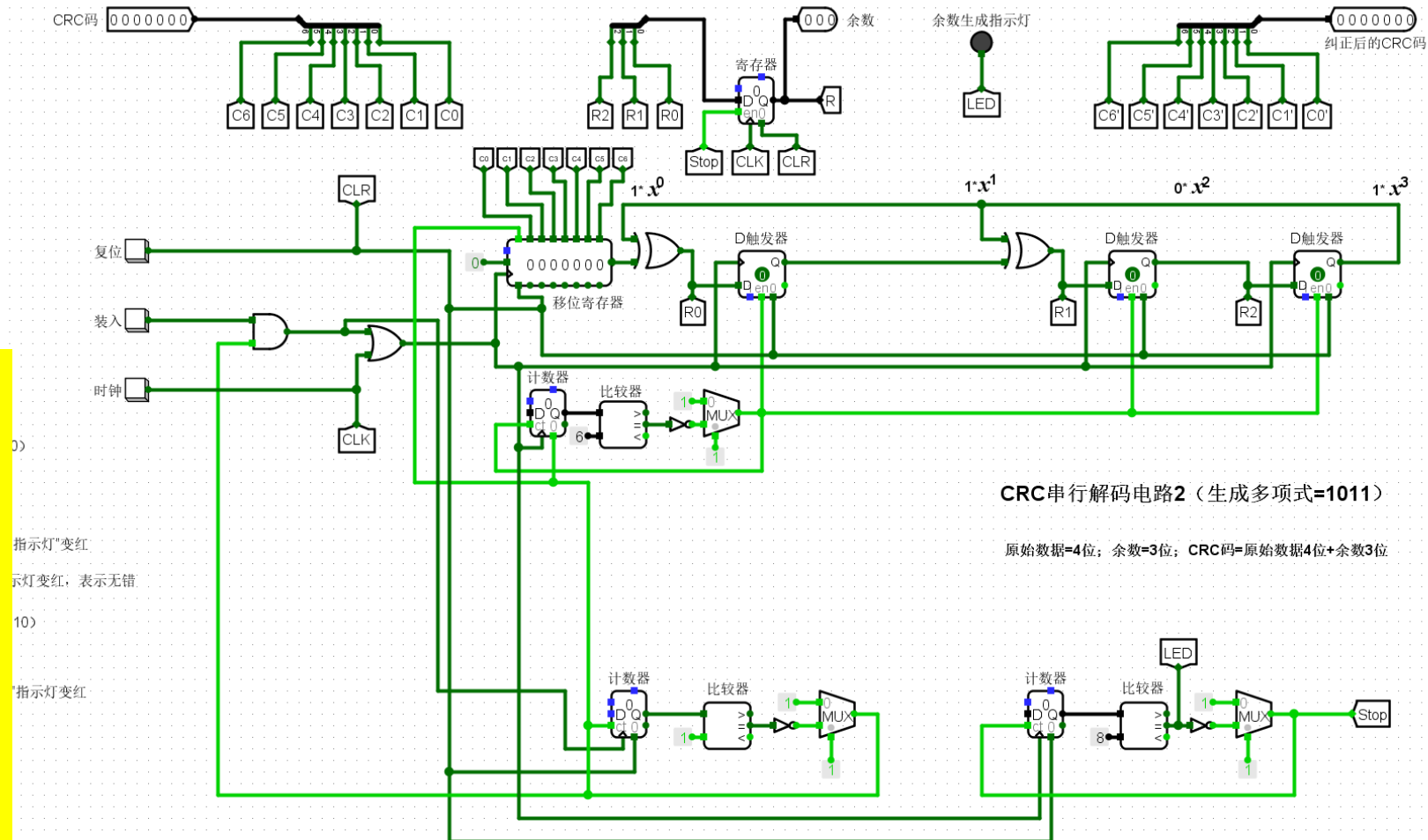
第一步: 置7位CRC码 (例如: 1100 010)

第二步: 按复位按钮

第三步: 按装入按钮

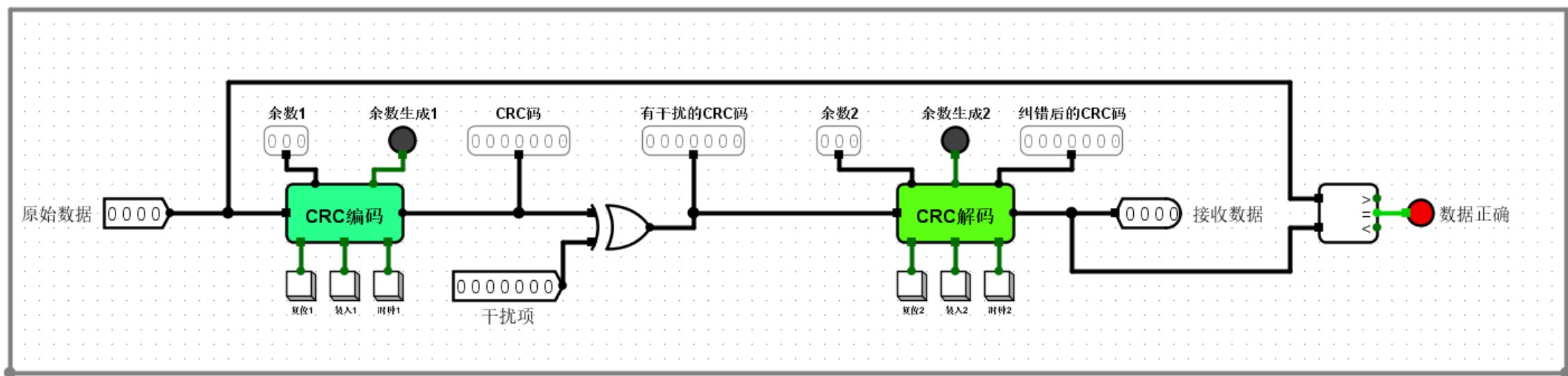
第四步: 按时钟按钮多次 (7次), 直到余数生成指示灯变红色

结果: 余数=000, 纠错后的CRC码=1100 010, 纠错后的数据=1100, 无错指示灯=红色



• 3、CRC串行编解码测试电路

- 输入：4位原始数据，7位干扰项
- 输出：4位接收数据（纠错后的数据），数据正确指示灯



请同学们参考前面的测试方法，完成该电路的测试

第一种情况（1位错），第二种情况（2位错）

实验要求

- 1、在logism上验证“汉字编码实验.circ”，分析其中的“国标码转区位码电路”、“汉字字库电路”，并实现在LED点阵上显示“欢迎加入厦门大学信息学院！”（文本文件命名为：**ROM内容-欢迎加入厦门大学信息学院!.txt**）
- 2、在logisim上验证“16位偶校验传输测试实验.circ”，分析其中的“偶校验编码电路”，“偶校验检错电路”，并实现：
 - （1）修改“16位偶校验传输测试实验”电路，使其实现“**16位奇校验传输测试实验**”电路（设计文件命名为：16位奇校验传输测试实验.circ）
 - （2）修改“16位偶校验传输测试实验”电路，使其实现“**8位偶校验传输测试实验**”电路（设计文件命名为：8位偶校验传输测试实验.circ）
- 3、在logisim上验证“16位扩展海明码传输测试实验.circ”，分析其中的“海明编码电路”、“海明解码电路”，并实现：
 - “**8位扩展海明码传输测试实验**”（即原始数据=8位），采用13位随机干扰电路.circ（已提供）
 - 设计文件命名为：**8位扩展海明码传输测试实验.circ**
- 4、在logisim上验证“循环冗余校验码（并行）实验.circ”，分析其中的“CRC并行编码电路”和“CRC并行解码电路”
- 5、在logisim上验证“循环冗余校验码（串行）实验1.circ”，分析其中的“CRC串行编码电路”和“CRC串行解码电路”
- 6、在logisim上验证“循环冗余校验码（串行）实验2.circ”，分析其中的“CRC串行编码电路”和“CRC串行解码电路”
- 7、**完成挑战性实验（8位循环冗余校验码并行实验），可获得加分，具体参实验报告模板**

实验报告提交要求

- 1、请按照实验报告模板（厦门大学计算机组成原理实验报告样本（第1次实验）.docx），撰写实验报告（Word版本），实验报告命名为：**学号+姓名-1.pdf**
- 2、请将实验报告和4个设计文件（ROM内容——欢迎加入厦门大学信息学院！.txt、16位奇校验传输测试实验.circ、8位偶校验传输测试实验.circ、8位扩展海明码传输测试实验.circ）打包为1个压缩文件，压缩文件命名为：**学号+姓名-1.zip**
- 3、将压缩文件上传到数字化教学平台：**course.xmu.edu.cn**，第1次实验报告提交**截止时间**（2周内）